The Trinity:
Capability-Driven Recursive Self-Improvement for Multimodal Agents

The Trinity Research Team

Project Page GitHub

Abstract Multimodal intelligence can be viewed as human-guided co-evolution between worlds and agents through interaction. However, scaling this process faces two bottlenecks: constructing heterogeneous, verifiable environments is costly, and manual Harness iteration cannot keep pace with growing task diversity and complexity. A key insight is that tasks with different modalities and surface forms can share the same underlying capability requirements. We therefore introduce **The Trinity**, a recursive self-improvement framework coupling world generation with Harness evolution through three roles: Creator, Explorer, and Evolver. Drawing on verified execution trajectories, Evolver shifts the basis of task organization from surface categories to shared requirements, reorganizing the original task tree into a hierarchical capability tree. We introduce **Capability-Driven Real2Sim2Real**: for heterogeneous tasks at the same node, Creator abstracts their common requirements into simplified, verifiable simulated worlds for capability learning, while evaluation on original tasks measures transfer. Along this capability tree, Explorer progressively evolves reusable capability modules through selective inheritance, forming a Harness tree whose validated updates accumulate across iterations within a fixed topology. The resulting Harnesses specialize in different capabilities, motivating a **Capability-Aware Harness Router** that uses public task information to select compatible module configurations from the tree. Our evolved Harnesses improve mean performance by 34 points across 28 training benchmarks and 26 points across 18 unseen benchmarks, including two real-world embodied tasks.

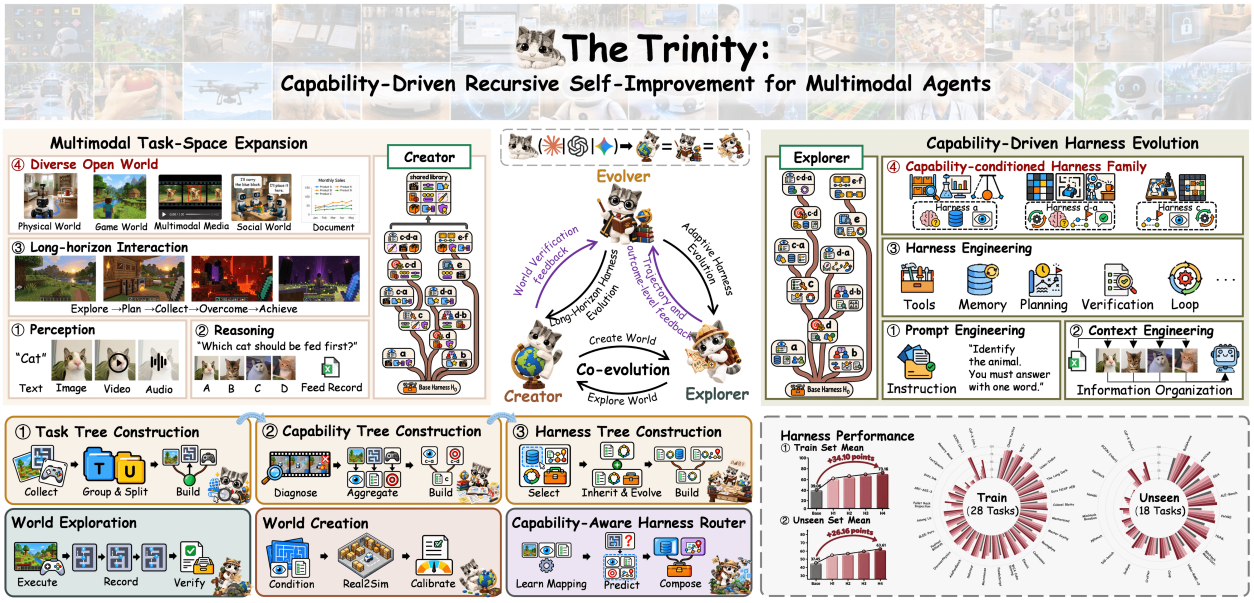

Figure 1 | **Overview of The Trinity.** Creator, Explorer, and Evolver connect multimodal task-space expansion (left) with capability-driven Harness evolution (right) through verified feedback. The lower panels summarize tree construction, world exploration and creation, and capability-aware routing; the charts report mean and benchmark-level Train/Unseen performance across RSI iterations.

Contents

1 Introduction **3**

2 Preliminaries **4**
2.1 Multimodal Agentic Tasks 4
2.2 Agent Harness 4

3 The Trinity **5**
3.1 Framework Overview 5
3.2 Stage 1: Task Tree Construction 5
3.3 Stage 2: Capability Tree Construction 6
3.4 Stage 3: Harness Tree Construction 7
3.5 Recursive Self-Improvement 7
3.6 Capability-Aware Harness Routing 8

4 Experiments **9**
4.1 Experimental Setup 9
4.2 Capability-Driven Organization 9
4.3 Recursive Self-Improvement 11
4.4 Capability-Aware Harness Routing 12
4.5 Capability–Module Relationships 12
4.6 Embodied Transfer of Capability-Evolved Harnesses 13

5 Related Work **14**

6 Conclusion **15**

7 Discussion **15**

8 Authors **15**

References **16**

A Task Tree **25**

B Capability Tree **27**

C Harness Tree **30**
C.1 Explorer Harness Tree 30
C.2 Creator Shared Harness 31

D Evaluation Details **33**

E Visual Record **35**
E.1 Interaction Process 35
E.2 Node Training Levels 46
E.3 Embodied Benchmarks 58

1. Introduction

Multimodal foundation models have rapidly advanced in understanding, reasoning, and generation across text, images, video, and audio [1–12], enabling agents to perceive, decide, and act in open environments [13–15]. We view multimodal intelligence as an evolving loop: worlds provide tasks, humans guide learning, and agents interact with the world to produce verifiable outcomes [16, 17]. As tasks and environments grow increasingly complex, this loop faces two bottlenecks: (1) Constructing large-scale, heterogeneous, and verifiable environments manually is costly, motivating automated world generation; and (2) Agent capability is increasingly constrained by the Harness [18–23], whose manual iteration cannot keep pace with growing task complexity and diversity.

Inspired by Harness recursive self-improvement [24–26] and agent–environment co-evolution [27–29], we ask whether this manual loop can improve continuously. Generated worlds provide adaptive curricula, while verified interaction feedback guides Harness updates.

However, extending this process across heterogeneous text-only and multimodal tasks [30, 31] presents a challenge. Current Harness RSI works focus on text-modality tasks (e.g., coding and math [23, 24, 32, 33]) and adopt task-specific evolution [22, 25, 34, 35]. This approach is suitable when tasks share similar capability requirements, but can lead to Harness overfitting and poor generalization across heterogeneous tasks. Conversely, a global Harness struggles to accommodate conflicting task demands, including performance–efficiency trade-offs in embodied tasks, limiting its efficiency and adaptability.

A key insight is that tasks with different modalities and surface forms can share the same underlying capability requirements. For example, although ARC-AGI-3 [36] and Micromouse [37] differ in task structure, both rely on accurate perception and state tracking: the former requires recognizing grid states and their changes, while the latter requires continuously updating environmental information during maze exploration. Grouping tasks by these shared capability requirements makes the underlying mechanisms a common learning objective, enabling generalization across tasks.

> **Position.** Current RSI training relies heavily on task-specific optimization, limiting capability generalization across tasks. We advocate a shift from task-driven to capability-driven self-improvement, where shared capability requirements across tasks guide the learning and optimization of multiple specialized Harnesses. This shift highlights the role of shared capabilities in cross-task generalization and naturally motivates a capability-aware Router that matches tasks to suitable Harnesses based on their capability requirements.

To operationalize this position, we introduce **The Trinity**, a recursive self-improvement framework coupling environment generation with agent evolution through Creator, Explorer, and Evolver. Using verified trajectories, Evolver identifies task capability requirements and groups tasks sharing them, reorganizing the original task tree into a hierarchical capability tree. Heterogeneous tasks at the same node share a learning objective, motivating **Capability-Driven Real2Sim2Real**: Creator abstracts their common requirements into simplified, verifiable environments, while evaluation on the original tasks measures capability transfer. Along the capability tree, Explorer evolves a Harness tree through selective inheritance and local refinement, while Creator evolves at corresponding nodes and consolidates validated updates into a new Base Harness at the end of each iteration.

The Harnesses specialize in different capabilities, requiring task-dependent selection. For Explorer, our **Capability-Aware Harness Router** uses verified execution experience and public task information to select a matching branch of Explorer's Harness tree. For Creator, overlap within its Harness tree motivates merging complementary modules and removing near-duplicates to form a new Base Harness. Explorer and Creator play distinct roles in RSI processes along the same capability tree.

Contributions. ❶ ***Capability-driven multimodal RSI.*** We make shared capability requirements the organizing principle of Harness evolution. Capability-Driven Real2Sim2Real turns these requirements into compact training worlds, yielding a reusable module-based Harness tree whose specialization motivates capability-aware routing. ❷ ***Capability–module priors for Harness design.*** We connect task-level capability requirements to reusable module configurations, using empirical associations and controlled interventions to examine which module support matters for a task. ❸ ***Efficient evolution and transfer.*** We show that targeting shared decision logic supports effective Harness improvement at low evolution cost, with transfer across heterogeneous tasks, unseen benchmarks, and real-world embodied settings.

2. Preliminaries

2.1. Multimodal Agentic Tasks

Long-horizon multimodal agentic tasks span interactive games [38], GUI manipulation [39], and embodied interaction [13, 14]. They combine perception, understanding, reasoning, and generation with memory, planning, and adaptation across observation–action sequences. Agents must retain evidence, revise hypotheses, and adjust plans as observations arrive. Because later decisions depend on earlier actions and feedback, misread observations or forgotten constraints can propagate across steps. Agent performance thus increasingly depends on effective Harness organization and coordination. Dynamically coordinating capabilities over long interactions and adapting to increasingly large, heterogeneous task sets remain key challenges.

2.2. Agent Harness

A Harness configuration ϕ coordinates a frozen model f_θ through six control-flow protocols:

$$\mathcal{H}_\phi = (\Pi_{\text{er}}, \mathbf{M}, \mathbf{I}, \mathbf{P}, \mathbf{F}, \mathbf{L}), \qquad \mathcal{H}_0 \equiv \mathcal{H}_{\phi_0} \quad \text{(Base)}. \tag{1}$$

Given task τ, observation o_t, public history $\xi_{<t}$, controller state s_t, and available resources C_τ:

$$\begin{aligned} f_t &= \Pi_{\text{er}}(o_t, \xi_{<t}), & v_t &= \mathbf{M}(\xi_{<t}, s_t, f_t), \\ \text{Cand}_t &= \mathbf{I}(\tau, v_t), & d_t &= \mathbf{P}(\tau, s_t, v_t, \text{Cand}_t), \\ C_t &= \mathbf{F}(C_\tau, s_t, v_t, d_t), & (s_{t+1}, e_t, c_t) &= \mathbf{L}(s_t, v_t, d_t, C_t). \end{aligned} \tag{2}$$

Perception extracts facts f_t; **Memory** builds a history-conditioned view v_t; **Inference** maintains compatible candidates; **Planning** ranks or prunes options into directive d_t; **Orchestration** identifies applicable resources; and **Loop** resolves execution e_t and commitment c_t, including withholding action. These roles are independent of task categories and capability-tree nodes. The equations describe functional roles and their data dependencies; individual module bindings are invoked under the runtime policy described in Section 3.4.

Modules are pure functions of supplied public records, without hidden-state access or environment actions. Only Memory may declare explicit `carry`; hypothesis refutation requires observed contradiction. All protocols may emit typed readiness signals, while Loop resolves execution. Orchestration assesses applicability, relevance, and trust within a fixed host-exposed tool set. Evolution updates module configurations with θ fixed.

Capabilities and protocols have a many-to-many relationship: the tree groups shared requirements, while protocols locate module updates. Hypothesis consistency can involve Memory organizing evidence, Inference checking compatibility, and Planning ranking informative tests; the same Memory module can support several requirements. Explorer retains node modules; Creator uses the same tree structure and consolidates its next Base Harness. Evaluation-time skills are prepared separately (Section 4.1); they are not stored, evolved, inherited, or routed in the tree.

3. The Trinity

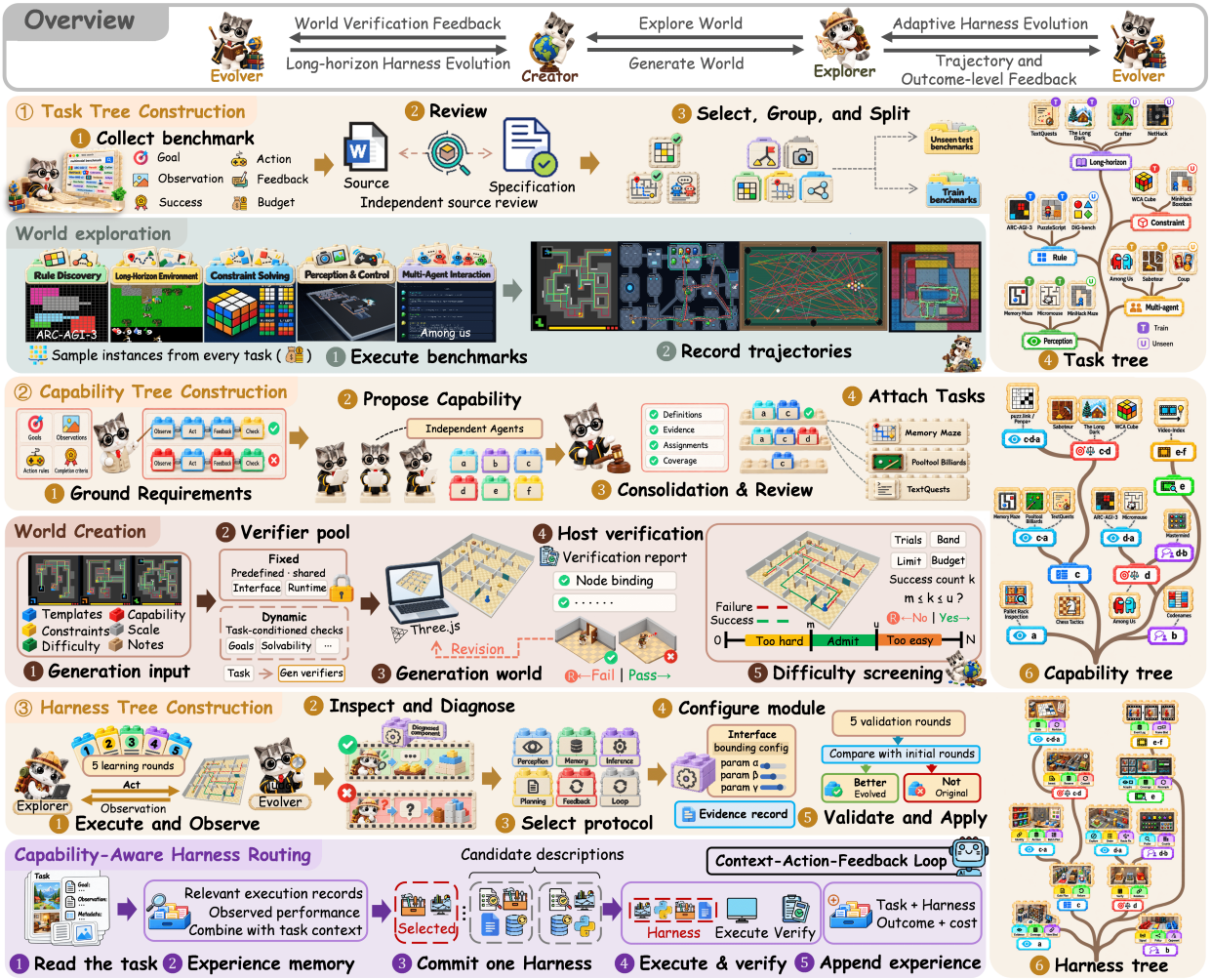

Figure 2 | Detailed pipeline of The Trinity. Three stages build task, capability, and Harness trees. Worlds are verified and screened; Harnesses evolve through selective inheritance and local refinement. Routing selects one compatible Harness and stores verified feedback for later tasks.

3.1. Framework Overview

The Trinity couples environment generation and Harness evolution with a frozen backbone (Figure 2). Explorer collects trajectories; Evolver organizes requirements from specifications and verified trajectories into a capability tree (Sections 3.2–3.3). Through **Capability-Driven Real2Sim2Real**, Creator builds simplified, verifiable worlds; original-task evaluation measures transfer (Section 3.3). Explorer selectively inherits and refines reusable modules, accumulating validated updates in a fixed-topology Harness tree (Sections 3.4 and 3.5). The **Capability-Aware Harness Router** selects a compatible branch using public task information and verified experience (Section 3.6).

3.2. Stage 1: Task Tree Construction

Benchmark Organization. Evolver normalizes public multimodal benchmarks into specifications of goals, observations, actions, feedback, task-completion criteria, and budgets, independently checked

against their sources. A common observation–action interface preserves information access and scoring rules. Benchmarks are grouped into five requirement domains: rule discovery, constrained problem solving, perception and control, multi-agent interaction, and long-horizon environments. The training/unseen split is fixed before capability induction.

World Exploration. Explorer collects trajectories from sampled training instances using $\mathcal{H}_0$ under fixed settings. Only training records support capability-tree construction, world generation, module learning, and Router training (Section 3.6). Unseen capability requirements are assessed only after the tree is fixed, for evaluation. Instance selectors and initial-state hashes are fixed in advance. Diagnostic reruns reset memory; completed runs are retained, while infrastructure failures are marked invalid and retried. Hash-chain checks verify execution records.

3.3. Stage 2: Capability Tree Construction

Evolver derives requirements from task goals, permitted observations, action rules, and task-completion criteria, linking them to rules and trajectory steps satisfying or violating these criteria through rule-based checks. Host-private information supports only retrospective verification.

Requirement Organization. Using training-task specifications and trajectories, independent agents propose **capability categories** $\mathcal{K}$ (Appendix B); a judge consolidates them, and reviewers independently check definitions, evidence, assignments, and coverage. Each training task τ receives a fixed requirement set $K_\tau \subseteq \mathcal{K}$, including requirements already satisfied by $\mathcal{H}_0$.

The tree is constructed from training tasks $\mathcal{B}_{\text{tr}}$ only. A deterministic capability order $\prec$, fixed before construction, defines paths and the tree:

$$p_\tau = \text{sort}_\prec(K_\tau), \qquad \mathcal{T} = \text{Trie}\big(\{p_\tau : \tau \in \mathcal{B}_{\text{tr}}\}\big). \tag{3}$$

The Trie merges **common prefixes** across domains. Each node v supports an Explorer module configuration for its prefix; tasks retain endpoint associations, including at internal nodes. Node requirements define local objectives, with selective parent inheritance for Explorer; the order implies no causal dependencies. Requirements for unseen tasks $\tau \in \mathcal{B}_{\text{un}}$ are identified afterward using the fixed categories, solely for retrospective Router evaluation, without modifying the tree.

World Creation. Once the capability tree is built, each node is linked to training benchmarks through shared requirements. Although the four tasks in Figure 3 differ, all require evidence verification before decisions. This motivates **Capability-Driven Real2Sim2Real**: Evolver designs simplified training levels from node requirements, descendants, and representative tasks' public specifications, excluding target-task layouts, private proofs, and solution traces while preserving core decision processes. Creator implements these levels using a node Harness initialized from its Base Harness or inherited parent modules, omitting aesthetic optimization to focus on task mechanisms. Evolver's host-executed verifier pool checks node binding, execution, state transitions, rendering, and completion. Node identifiers link specifications, trajectories, and module updates, while private verifiers and public feedback guide revisions. Explorer tests verified worlds with a frozen screening Harness, admitting only those within a predefined

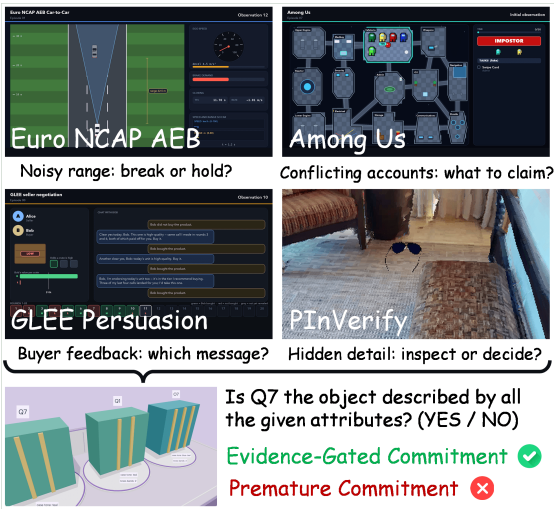

Figure 3 | Capability-driven world creation at node d.

intermediate-performance band. Otherwise, Creator adjusts difficulty along predefined dimensions. Trial counts, performance thresholds, action budgets, and retry limits are fixed in advance; worlds and verifiers remain unchanged during local training. Curricula may be reused or regenerated, while original benchmarks assess capability transfer. Appendix E.2 details the curricula for each node.

3.4. Stage 3: Harness Tree Construction

For Explorer, capability-driven RSI builds a Harness tree aligned with the capability tree. At each node, Explorer executes admitted worlds with inherited and local modules. Evolver compares successful and failed trajectories using public observations, actions, and host-assigned outcomes to identify node-specific improvements. It develops **reusable modules** through host-registered bindings in the six-protocol library (Section 2.2), refining interfaces, bounded configurations, or implementations. Updates record node, predecessor-version, episode, and executable-example references. Before local updates, the host validates evidence, ownership, interfaces, lineage, and typed input–output assertions; ancestor modules remain preserved. Bindings use public context and permitted memory under a fixed runtime policy. Multiple bindings per protocol may coexist, with model calls determining execution order. Admission verifies grounding and executability; evaluation measures performance. Creator evolves only upon verifier-detected failures. At each iteration's end, validated proposals are consolidated with existing Base Harness modules, merging complementary modules and removing near-duplicates. Section 3.5 details persistence, selective inheritance, and consolidation.

3.5. Recursive Self-Improvement

Explorer evolves along the capability tree, with Creator following at corresponding nodes; their Harness trees share the same structure (Appendix C). Explorer selectively inherits node modules; at each iteration's end, Creator deduplicates and merges complementary updates into its next Base Harness. Let $V = V(\mathcal{T})$ and V^+ denote all nodes and non-root nodes, respectively.

Node State. For Explorer, let $\mathcal{H}_{\mathrm{E},v}^{(i)} \equiv \mathcal{H}_{\phi_{\mathrm{E},v}^{(i)}}$ denote the Harness at node $v \in V$ after iteration i. Nodes initialize with $\mathcal{H}_{\mathrm{E},v}^{(0)} = \mathcal{H}_0$ and available training feedback $\mathcal{F}_v^{(0)}$. Top-down iterations retain local modules and keep the synthetic root fixed.

Selective Inheritance. At non-root node $v \in V^+$, compare the previous inherited registry $P_v^{(i)}$ with the updated parent $\widehat{P}_v^{(i+1)}$, holding local modules $L_v^{(i)}$ fixed. Writing the Harness's module configuration as $\mathcal{H}_{\mathrm{E},v}^{(i)} = P_v^{(i)} \oplus L_v^{(i)}$, registries use host-registered binding IDs and right-biased union:

$$(P \oplus L)[m] = \begin{cases} L[m], & m \in \mathrm{dom}(L), \\ P[m], & m \in \mathrm{dom}(P) \setminus \mathrm{dom}(L). \end{cases} \tag{4}$$

Parent exports are bound to parent identifiers and state digests. Ancestor entries compose at recorded versions from root to parent, then local modules; overrides preserve ancestor sources and local ownership/version lineage. Under identical validation tasks, trial count, and budget, the updated parent is retained only with strictly higher host-verified performance, measured by task-completion counts; otherwise, the previous registry is retained.

Continual Refinement. Let $P_v^{(i+1)}$ be the retained inherited registry and $\mathcal{D}_v^{(i+1)}$ the admitted-world curriculum conditioned on the fixed requirement, previous feedback $\mathcal{F}_v^{(i)}$, and prior coverage. Curricula may be renewed or reused. Explorer's trajectories $\mathcal{Z}_v^{(i+1)}$ guide Evolver's local update:

$$\begin{aligned} \mathcal{Z}_v^{(i+1)} &= \mathrm{Rollout}\left(P_v^{(i+1)} \oplus L_v^{(i)}, \mathcal{D}_v^{(i+1)}\right), \\ \widetilde{L}_v^{(i+1)} &= \mathrm{Evolve}\left(L_v^{(i)}, \mathcal{Z}_v^{(i+1)}, \mathcal{F}_v^{(i)}\right). \end{aligned} \tag{5}$$

Trajectories record public observations, actions, host-assigned outcomes, and executed configurations. The host checks update evidence, ownership, interfaces, predecessor versions, and executable contracts. Accepted updates modify local state; rejection preserves it. The retained $L_v^{(i+1)}$ yields $\mathcal{H}_{\mathrm{E},v}^{(i+1)} = P_v^{(i+1)} \oplus L_v^{(i+1)}$; local admission requires no score increase.

Subsequent training-side evaluation of the retained Harness yields $\mathcal{E}_v^{(i+1)}$ and next-iteration feedback

$$\mathcal{F}_v^{(i+1)} = \mathrm{Feedback}\left(\mathcal{Z}_v^{(i+1)}, \mathcal{E}_v^{(i+1)}\right), \tag{6}$$

where Feedback retains verified observations, outcomes, failure evidence, and episode/module-version references for the next iteration. Completed nodes export retained state to children; sibling branches evolve independently. Explorer retains $\mathcal{H}^{(i)} = \{\mathcal{H}_{\mathrm{E},v}^{(i)} : v \in V\}$ for routing.

Creator Base Harness Update. Creator's Base Harness $\mathcal{H}_{\mathrm{C}}^{(i)}$ has module registry $R_{\mathrm{C}}^{(i)}$, initialized with pre-RSI modules at $i = 0$. Creator evolves nodes in top-down order within each iteration. Root children start from this base; other nodes inherit their parent's updated module configuration. Each node retains its own guidance. Verifier-detected failures trigger local evolution, yielding validated proposals $\Delta R_{\mathrm{C},v}^{(i+1)}$. At iteration end, proposals and base modules are consolidated in learning order $\prec_{\mathrm{C}}$:

$$R_{\mathrm{C}}^{(i+1)} = \mathrm{Consolidate}_{\prec_{\mathrm{C}}}\left(R_{\mathrm{C}}^{(i)} \cup \bigcup_{v \in V^+} \Delta R_{\mathrm{C},v}^{(i+1)}\right). \tag{7}$$

The resulting registry configures $\mathcal{H}_{\mathrm{C}}^{(i+1)}$, the Base Harness for the next iteration. Validated revisions replace predecessors; distinct capabilities are retained, partial overlaps are merged to preserve complementary functionality, and near-duplicates retain the earlier-learned module. Each binding retains one active definition and its provenance. Guidance, construction notes, and action budgets remain node-specific. Without accepted updates, the Base Harness remains unchanged. Explorer's evaluation-time skills are prepared separately (Section 4.1).

3.6. Capability-Aware Harness Routing

Routing follows a context–action–feedback loop [40]. Let x_n contain the public goal, initial observation, metadata, and images, and $C \subseteq \mathcal{H}^{(i)}$ be the fixed, versioned Harness pool. Memory $\mathcal{M}_0$ is initialized by importing verified training-side categorization trajectories into empty memory once in file order (Section 3.2). These $\mathcal{H}_0$ executions record public context, the executed configuration, outcomes, and costs; unseen categorization records are excluded. The agent, prompt, candidate modules, retrieval settings, and verifier remain fixed.

For each task, candidate descriptions, retrieved experience R, and observed performance–cost summaries S guide selection:

$$h_n = \pi(x_n, C, R(x_n, \mathcal{M}_n), S(\mathcal{M}_n)). \tag{8}$$

The agent commits h_n with an evidence-based rationale before execution. Retrieval uses cosine similarity ≥ 0.5, at most ten records, and a 20,000-record FIFO memory. Summaries include only matching executed configurations; untried outcomes remain unknown. Tasks are processed once in file order. After terminal receipts are verified against the committed decision and registry hash, outcome and cost f_n update memory for later tasks:

$$\mathcal{M}_{n+1} = \mathrm{Append}(\mathcal{M}_n, (x_n, h_n, f_n)). \tag{9}$$

Unseen task–capability assignments remain excluded from routing context, verification, and memory; they are accessed only for evaluation after the complete prediction ledger is hash-locked. Technical failures remain recorded and score zero; invalid selections receive no fallback.

4. Experiments

4.1. Experimental Setup

We use 28 training and 18 unseen benchmarks (Table 1; Appendix A). Train evaluates a fixed set of representative instances per benchmark over multiple episodes; Unseen uses each benchmark's official evaluation set or a fixed official subset. Five experiments examine capability-driven organization, recursive self-improvement, capability-aware Harness routing, capability–module relationships, and embodied transfer of capability-evolved Harnesses.

Table 1 | Training and unseen benchmarks. *Agent-built from web-searched sources.

Task domain	Train	Unseen
Rule discovery	Eleusis* [41], Mastermind [42], WILT [43], BoxingGym [44], DiscoverPhysics [45], ARC-AGI-3 [36], PuzzleScript [46]	Entity Deduction Arena [47], SciGym [48], DiG-bench [49]
Constrained problem solving	Chess Tactics* [50], puzz.link / Penpa+ [51, 52], WCA Cube Solving* [53], AdaPlanBench [54], Master Keying* [55]	Pencil Puzzle Bench [56], MiniHack Boxoban [57], ALE-Bench [58], BuildArena [59]
Perception and control	Pallet Rack Inspection* [60], PInVerify [61], Memory Maze [62], Micromouse* [37], Euro NCAP AEB* [63], Pooltool Billiards [64, 65], Video-Index [66], SOON (simulation) [67], CaP-X (simulation) [68]	ActiView [69], MiniHack Maze / Corridor [57], PHYRE [70], CaP-X (robot) [68], Video-MME-v2 [71], SOON (robot) [67]
Multi-agent interaction	Among Us [72], Saboteur* [73], Codenames [74], Colonel Blotto [75], GLEE Persuasion [76]	Coup [75], Hanabi [77], Heads-Up NL Hold'em [78, 79]
Long-horizon environments	TextQuests [80], The Long Dark* [81]	NetHack [82], Crafter [83]

All variants share Base Harness $\mathcal{H}_0$ (Prime Agent) and a frozen Claude Opus 5 backbone [84]. Table 8 in Appendix D reports metrics, instance and episode counts, and full results. Token costs are totals in millions (M); Tables 4–5 report inference costs, and – denotes data not required.

Evaluation-Time Skill Preparation. To reflect real deployment and account for each Harness's effect on skill learning, we run five preparation trials per benchmark–Harness pair, summarize their trajectories into skills, and keep these skills fixed for subsequent evaluation on Train and Unseen.

4.2. Capability-Driven Organization

Preliminary Comparison: Modules and Skills. Table 2 compares four settings at Iteration 1. **Base** uses initial modules; **Module Only** evolves them; **Skill Only** adds skills to Base; **Ours** combines both and performs best on Train and Unseen. Skills follow Section 4.1. Organization comparisons use **Module Only** to isolate the effect of Harness organization.

Table 2 | Preliminary comparison of modules and skills. Module denotes evolved modules beyond Base Harness. Combining both improves performance over either alone.

Method	Module	Skill	Train	Unseen
Base ($\mathcal{H}_0$)	×	×	39.06	37.45
Module Only	✓	×	56.95	47.26
Skill Only	×	✓	58.86	48.67
Ours ($\mathcal{H}^{(1)}$)	✓	✓	**63.80**	**53.53**

Table 3 | Harness organization and training curricula. All methods use module-only evaluation without skills. The simulated capability tree achieves the highest Train and Unseen performance.

Method	Curriculum	# H	# Env.	Train	Unseen	Training (M)	Inf. Train (M)	Inf. Unseen (M)
Base ($\mathcal{H}_0$)	–	–	–	39.06	37.45	–	582.54	613.07
Global	Real	1	28	53.04	42.92	582.54	531.81	567.68
A. Task vs. Capability Organization								
Task-Domain	Real	15	15	51.17	40.82	278.25	540.36	577.33
Capability-Domain	Real	6	6	49.25	39.43	136.94	562.32	596.58
B. Real Tasks vs. Capability Simulation								
Capability Tree	Real	10	11	53.44	42.31	211.56	526.53	563.90
Capability Tree	**Sim**	10	**11**	**56.95**	**47.26**	405.70	514.42	549.07

Organization Comparison. Table 3 compares five organizations after one RSI iteration to assess task grouping, hierarchical inheritance, and training curricula. **Global** pools experience from all 28 original environments into one shared Harness. **Task-Domain** learns 15 task-specific Harnesses, whereas **Capability-Domain** uses six flat capability groups.

Capability Tree (Real) and **Capability Tree (Sim)** use the same prefix-based topology, selective parent inheritance, and node-specific local refinement described in Section 3.5. Real trains on original benchmark tasks; Sim uses verified synthetic worlds that preserve their decision processes. Domain/tree variants use one Harness and environment per domain/node. Training counts evolution-stage tokens; Inf. Train/Unseen count execution-only evaluation tokens.

Evaluation. Flat grouping reduces training cost but underperforms Global: Capability-Domain uses the fewest training tokens at 136.94M, with Train/Unseen scores of 49.25/39.43. Capability Tree (Real) gains 4.19/2.88 points over this flat organization, supporting selective inheritance across shared capability prefixes. Sim adds another 3.51/4.95 points, reaching the best module-only scores of 56.95/47.26. Its inference costs are also lowest at 514.42M/549.07M. First-iteration training costs 405.70M, versus 211.56M for Real and 278.25M for Task-Domain.

Training Cost and Environment Reuse. Figure 4 shows that Creator's environment construction dominates the initial training cost. Reusing worlds reduces this overhead in later iterations. Explorer collects the same number of trajectories across iterations with comparable inference-token usage. Gains at fixed trajectory volume suggest that cleaner evolution signals from capability-focused environments support more effective Harness updates. Reuse thus amortizes construction costs as Train performance improves. Full results and the evaluation protocol appear in Appendix D, Table 8.

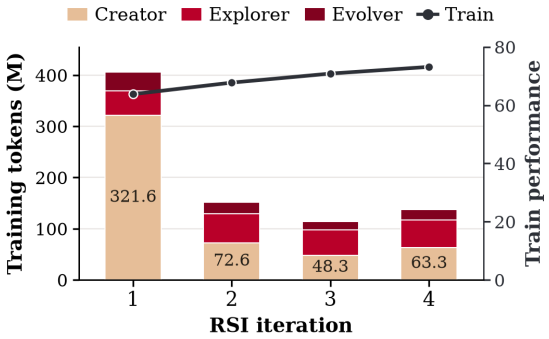

Figure 4 | Training cost and Train performance across RSI iterations.

Finding 1. Capability-driven organization outperforms task-driven specialization. A capability-centric perspective gives rise to a capability tree, where capability-driven Real2Sim2Real achieves stronger capability transfer and lower inference costs with fewer training environments, demonstrating that less is more.

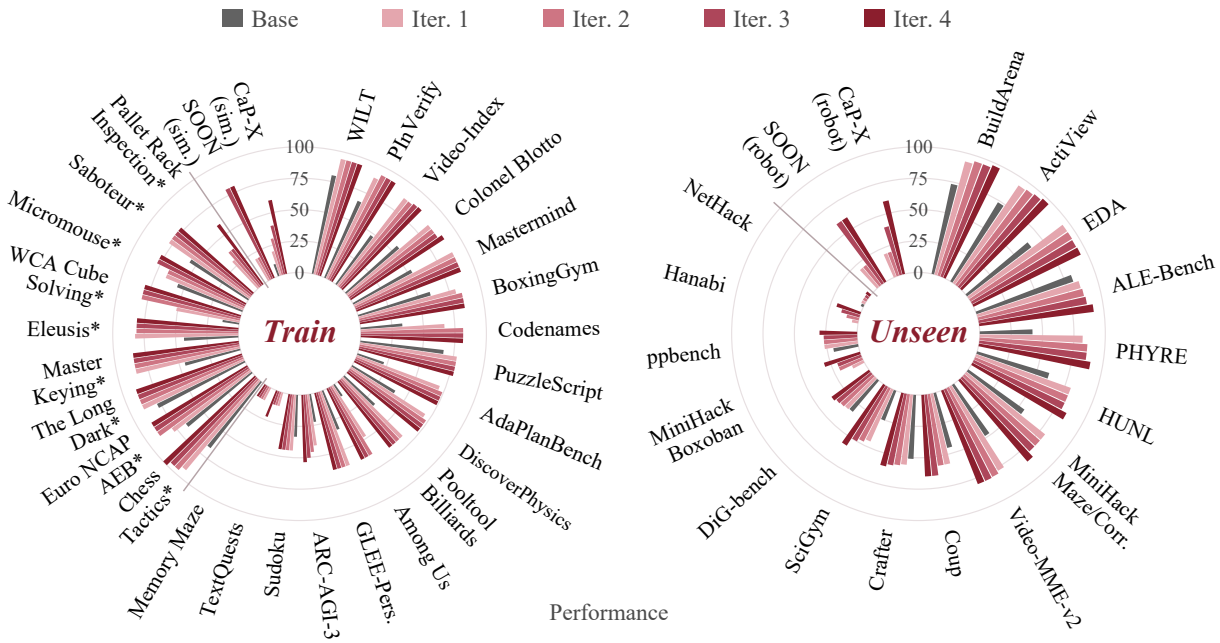

Figure 5 | Benchmark-level Harness evolution. Left: Train; right: Unseen. Radial bars show Base and Iterations 1–4; *Agent-built from web-searched sources.

4.3. Recursive Self-Improvement

We track the RSI loop (Section 3.5) from Base (Iteration 0) through Iteration 4 to test whether improvements continue after the first update. With the benchmark split, capability requirements, tree topology, and backbone fixed, Evolver refines modules from training feedback, Creator renews or reuses curricula, and Explorer collects new trajectories.

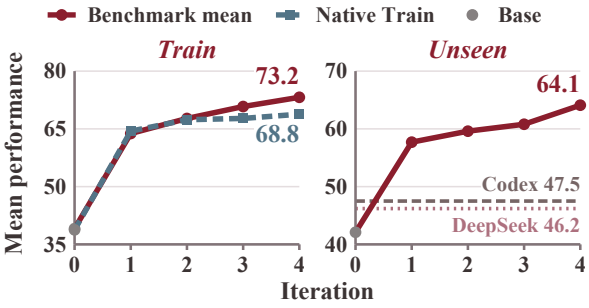

Figure 6 | RSI mean performance across benchmarks. Horizontal lines show Codex and DeepSeek reference means.

Evaluation. Figure 5 reports scores for 28 Train and 18 Unseen benchmarks, with five radial bars per benchmark. Scores retain benchmark-specific metrics on a 0–100 scale; Appendix D gives full results and protocols. Figure 6 averages benchmarks equally. Train covers all 28 benchmarks, with a dashed curve for the 17 native benchmarks; Unseen and its Codex [85] and DeepSeek [86] references cover the 16 non-robot benchmarks. From Base to Iteration 4, Train rises from 39.1 to 73.2 and Unseen from 42.1 to 64.1. The first iteration yields the largest gains, but Iterations 2–4 add another 9.4 Train and 6.4 Unseen points. Native Train also improves from 64.4 to 68.8 after Iteration 1, with smaller later gains than the full Train mean. Progress varies across benchmarks: between Iterations 1 and 4, ARC-AGI-3 rises from 41.0 to 53.6, SciGym from 45.2 to 57.1, and BuildArena from 93.3 to 97.8. Mastermind stays at 88.9, while Eleusis declines from 82.2 to 81.1. The final Unseen mean exceeds the Codex (47.5) and DeepSeek (46.2) references, although Crafter and DiG-bench remain below both comparators (Appendix D).

> **Finding 2. Capability-driven RSI sustains iterative improvement.** Continued gains on a fixed capability tree suggest that simulated environments offer further opportunities for capability refinement, with improvements extending to unseen tasks.

4.4. Capability-Aware Harness Routing

We compare five configurations on the Iteration-4 Explorer Harness tree $\mathcal{H}^{(4)}$ (Section 3.6). **Random** selects one candidate Harness per task at random. **Agent Self-Routing** has three variants: public task information and interface descriptions alone; adding capability definitions, annotated branch cards, and aggregate observed performance–cost summaries; and further retrieving execution experience, yielding the full **Harness Router**. **Oracle** selects the best compatible Harness retrospectively per instance as an upper reference. In Table 4, Cap. and Mem. denote explicit capability definitions and retrieved execution experience; these configuration columns apply only to self-routing variants.

Evaluation. Table 4 pairs benchmark-equal performance with estimated total inference tokens (M) for each split. Full self-routing scores 71.27 on Train and 60.96 on Unseen, compared with Oracle's 73.16 and 63.61, respectively. The token estimates cover skill preparation, routing, and formal inference. Deployable variants share the frozen backbone, episode budgets (including routing calls), initial states, and seeds. Each Harness prepares and freezes skills under Section 4.1 using matched instances and budgets. Neither skills nor preparation records enter routing decisions or memory; evaluation capability labels are also excluded from memory. Unseen routing starts from training memory, and verified feedback updates it only for later instances. Technical failures score zero; invalid selections receive no fallback.

Table 4 | Capability-aware Harness routing. The full self-routing configuration approaches Oracle on both splits.

Method	Cap.	Mem.	Train	Tok. (M)	Unseen	Tok. (M)
Random	–	–	52.74	567.08	46.23	594.25
Agent Self-Routing	×	×	65.79	535.53	52.36	582.29
Agent Self-Routing	✓	×	70.34	510.55	58.27	556.21
Agent Self-Routing	✓	✓	**71.27**	497.89	**60.96**	541.28
Oracle	–	–	73.16	486.53	63.61	528.34

> **Finding 3. Capability priors guide agent routing.** An agent acting as a Router can use capability priors to select suitable Harnesses. The results suggest that the capability tree provides a semantic basis for matching inferred task requirements to Harness configurations.

4.5. Capability–Module Relationships

Capability–Module Correlations. Figure 7 reports Iteration-4 correlations for 29 modules. Rows abbreviate six shared capabilities: **PST**, perception and state tracking; **HCI**, hypothesis and counterpart inference; **CFR**, constraint checking and feedback repair; **GVT**, goal commitment and value trade-off; **VEA**, video evidence acquisition; and **VC**, video comprehension. Columns denote Perception, Memory, Inference, Planning, Orchestration, and Loop (Section 2.2), with coefficients in $[-1, 1]$. The strongest positive association in each row links PST to Memory (0.306), HCI to Inference (0.744), CFR to Planning (0.274), GVT to Loop (0.444), and VEA to Perception (0.347); VC correlates equally with Perception and Memory (0.286). The most negative coefficients link GVT to Perception (−0.361) and PST to Loop (−0.313). These descriptive associations motivate the module interventions below.

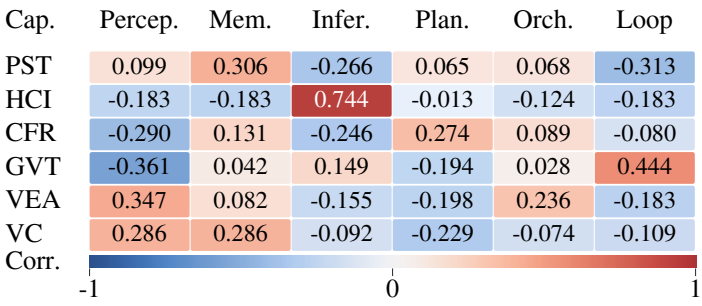

Figure 7 | Capability–protocol correlations across six protocols. Red is positive; blue is negative.

Module Interventions. Table 5 evaluates module interventions on Train (the fixed instance set of Section 4.1) with skills prepared under Section 4.1. Scores are benchmark-equal means on a 0–100 scale; reported inference token totals are in millions (M). For addition, we randomly select one relevant or irrelevant module from the same library and add it to $\mathcal{H}_0$. Candidates pass interface checks and exclude installed modules; additions match protocol class and have comparable description lengths. Relevance follows fixed task requirements; irrelevant modules target capabilities outside both these requirements.

Table 5 | Module interventions. Δ is the difference in displayed scores from each bold baseline.

Method	Train	Δ	Tokens (M)
Base ($\mathcal{H}_0$)	39.06	0.00	582.54
+ irrelevant module	40.08	+1.02	583.37
+ relevant module	58.45	+19.39	551.85
Ours ($\mathcal{H}^{(4)}$)	**73.16**	0.00	496.70
− relevant module	70.42	−2.74	517.26

Adding a relevant module raises the Base score from 39.06 to 58.45 (+19.39), versus 40.08 (+1.02) for an irrelevant module; token totals are 551.85M and 583.37M, respectively, against Base's 582.54M. Removing one relevant installed module from the intact Iteration-4 Harness lowers its score from 73.16 to 70.42 (−2.74) and raises tokens from 496.70M to 517.26M.

Finding 4. Capability–module priors inform Harness design. Capability–protocol associations suggest specialization patterns, while controlled interventions show larger gains from relevant module additions and performance losses after their removal. Together, these results support using task capability requirements to guide module selection and composition.

4.6. Embodied Transfer of Capability-Evolved Harnesses

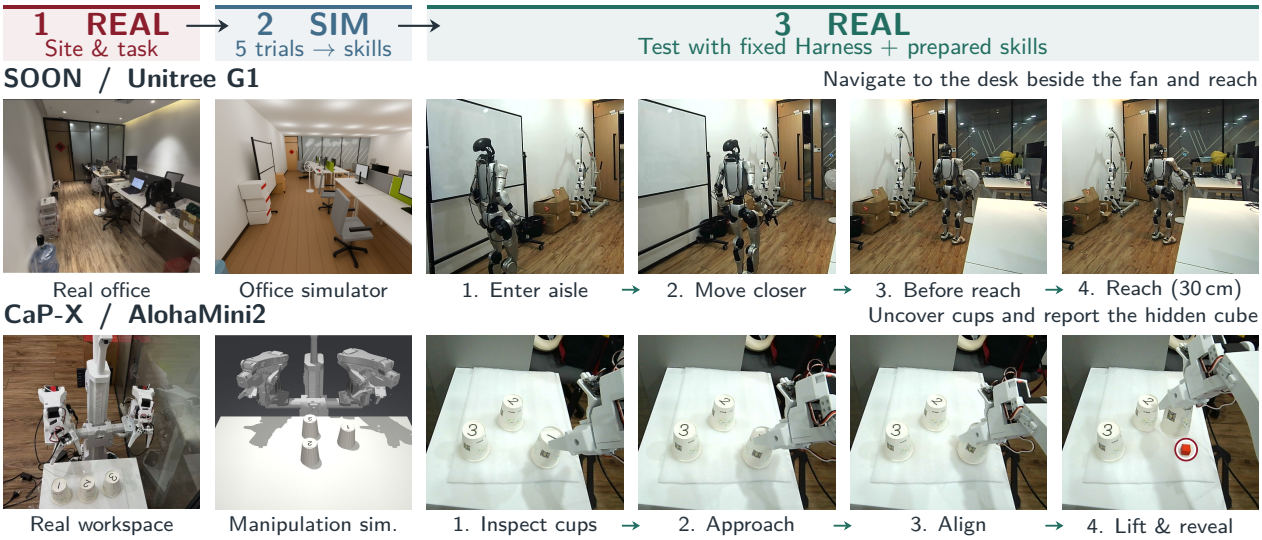

Figure 8 | Embodied Transfer of Capability-Evolved Harnesses. Real scenes, simulators, and robot trials (SOON E01; CaP-X C-01). The circle marks the revealed cube.

Based on their capability requirements, we map the embodied tasks to the $d \to a$ node of the capability tree and adopt the Harness configurations evolved there. We then use **Real2Sim2Real** to learn task-specific skills (Figure 8). For each task–Harness pair, we hold the Harness fixed, run five trial-and-error episodes in the reconstructed simulator, and summarize the trajectories into skills. These skills are frozen for formal simulation and real-robot tests; the five preparation episodes are excluded from test counts and success rates.

Protocol. In **SOON** [67], a Unitree G1 approaches the desk beside a white fan and reaches towards it; real-robot success requires the target desk first ahead of the extended arm, within 30 cm. In **CaP-X** [68], an AlohaMini2 uncovers three inverted cups and reports a cube's presence, colour, and location, or its absence. All five Harness configurations are evaluated with their own prepared skills. Appendix E.3 gives action budgets, observations, and available trajectory records.

Evaluation. Table 6 compares all five Harness configurations after skill preparation. SOON improves from 0 to 80% in simulation and from 0 to 60% on the robot; CaP-X improves from 10 to 60% and from 0 to 60%, respectively. Both robots reach 20% with $\mathcal{H}^{(1)}$. SOON reaches its final real-robot score at $\mathcal{H}^{(3)}$, while CaP-X improves from 40 to 60% between $\mathcal{H}^{(3)}$ and $\mathcal{H}^{(4)}$.

Table 6 | Embodied performance across iterations. Success rates (%) with prepared skills.

Harness	SOON (Sim)	CaP-X (Sim)	SOON (Real)	CaP-X (Real)
$\mathcal{H}_0$	0	10	0	0
$\mathcal{H}^{(1)}$	20	20	20	20
$\mathcal{H}^{(2)}$	20	30	20	20
$\mathcal{H}^{(3)}$	80	40	60	40
$\mathcal{H}^{(4)}$	**80**	**60**	**60**	**60**

Scope. Recorded deployments differ in message handling, interfaces, guidance, and acceptance rules (including lifting versus pushing). These comparisons evaluate complete Harness–skill configurations and do not isolate the effects of module evolution and skill preparation.

> **Finding 5. Capability-Driven RSI supports embodied transfer.** The evaluated Harness–skill configurations improve on both simulated and physical tasks, linking node-level module evolution with task-specific skill preparation.

5. Related Work

Recursive Harness Self-Improvement. Self-improvement spans Gödel machines [87], LLM systems that rewrite their own agent code [24, 88, 89], automated agent design [90–92], prompt optimization [93–95], and learning from self-feedback and reflection [96–98]. With weights fixed, improvement shifts to the harness, which is synthesized from environment feedback [33], repaired from failed trajectories [32, 35], self-revised [19], optimized as code [18, 20], adapted at test time [99] or per task [22], improved at the meta level [100, 101], and shown to lift frozen MLLMs [21]; benchmarks and surveys map this space [23, 102, 103]. Closest to ours are JIT-Agent's harness generator [25], HarnessX's typed primitives [104], HSI's per-task-family harnesses [34], ModularRSI's modules evolved on benchmark-disjoint tasks [26], and the flat capability map of Harness Continual Learning [105] and regime-branched harness tree of Adaptive Auto-Harness [106], both evolved on the task stream. Trinity instead induces a capability tree from training trajectories, shares prefixes across 44 heterogeneous benchmarks with selective module inheritance, evolves Harnesses in Creator-generated worlds that exclude target-task layouts and solutions, and evaluates on held-out benchmarks and real robots. Unlike LLM routers [40], ours routes among capability-specific Harnesses over one frozen backbone.

Co-evolution of Agents and Environments. Open-ended co-evolution [107–109], unsupervised environment design [29, 110], curricula [111, 112], and embodied co-design [28] adapt training to the learner; Voyager grows a skill library under an automatic curriculum [113]. Loops co-evolve language agents with environments [114], optimizers [115], or harnesses [116, 117]; environment generation spans embodied [118, 119], tool-use [120], and terminal settings [121, 122], language world models [123], and game verification [124]. SPADE is closest [27]: one LLM designs environments with verification code and learns roles by reinforcement learning. Trinity keeps the backbone frozen and couples world generation with Explorer and Creator Harness evolution along a capability

hierarchy, execution outcomes refining Explorer and verifier-detected failures refining Creator.

6. Conclusion

We advocate Capability-Driven Recursive Self-Improvement and introduce **The Trinity**, a framework that couples world generation with Harness evolution through Creator, Explorer, and Evolver. By organizing heterogeneous tasks around shared capability requirements, the framework constructs simplified, verifiable worlds whose execution feedback guides the evolution of reusable modules. These modules form specialized Harness configurations, with capability–module correlations and controlled interventions providing empirical support for capability requirements as priors for module design and composition. To apply these evolved capabilities across tasks, capability-aware routing selects suitable Harness configurations using public task information and verified execution experience. Experiments across diverse training and unseen benchmarks demonstrate improved aggregate performance and cross-task generalization, while deployment in two real-world embodied settings further illustrates the transfer of these capabilities to physical environments.

7. Discussion

Limitations. Our framework uses web search for benchmark discovery, but retrieval coverage and result quality may vary across queries. In addition, our current study focuses primarily on self-improvement at the Harness level.

Outlook. Capability-driven learning offers a new paradigm for generalizable agent improvement, where shared capability requirements across tasks guide learning and optimization. Our framework and results at the Harness level demonstrate the promise of this perspective. Looking ahead, we envision dynamically expanding the capability tree to capture emerging capability requirements and extending this paradigm to model parameter training, where capability-aware routing and RSI form a closed loop that jointly drives capability discovery, acquisition, and refinement.

8. Authors

Core Contributors: Weidong Tang[1,2] Ruiheng Wang[3] Jingxuan Zhang[4] Hexin Wang[1]

Contributors: Haoyuan Chen[1] Zhengfeiyang Zhang[1,4] Yixing Ma[1,5] Zichen Jiang[4] Ziyu Guo[6] Haodong Li[6] Zhengbo Jiao[7] Zhiqing Cui[7] Xinxiang Yin[8] Shuo Liang[1,4] Zhenglin Wan[4] Hongzhan Lin[4] Wangbo Zhao[7] Kaipeng Zhang[9]

Corresponding Authors: Pengfei Zhou[1,4] Odin Zhang[6] Yang You[4]

Affiliations: [1] InfRec [2] Xidian University [3] Tongji University [4] National University of Singapore [5] University of California, Berkeley [6] The Chinese University of Hong Kong [7] The Hong Kong University of Science and Technology [8] The University of Hong Kong [9] Shanghai Innovation Institute

References

[1] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katie Millican, Malcolm Reynolds, et al. Flamingo: a visual language model for few-shot learning. In *NeurIPS*, 2022.

[2] OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[3] OpenAI. GPT-4o system card. *arXiv preprint arXiv:2410.21276*, 2024.

[4] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M. Dai, Anja Hauth, Katie Millican, et al. Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.

[5] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *NeurIPS*, 2023.

[6] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, et al. Qwen3-VL technical report. *arXiv preprint arXiv:2511.21631*, 2025.

[7] OpenAI. OpenAI GPT-5 system card. *arXiv preprint arXiv:2601.03267*, 2025.

[8] Weidong Tang, Zhiyuan Liang, Xinyan Wan, Chen Zhu, Zhaopan Xu, Pengfei Zhou, Yan Song, Yang You, and Wangbo Zhao. Efficient video object segmentation and tracking with recurrent dynamic submodel. In *CVPR*, pages 20912–20921, June 2026.

[9] Zhiming Liu, Yujie Wei, Lei Feng, Xiu Su, Xiaobo Xia, Weili Guan, Zeke Xie, and Shuo Yang. Do all individual layers help? an empirical study of task-interfering layers in vision-language models. *arXiv preprint arXiv:2602.01167*, 2026.

[10] Weidong Tang, Xinyan Wan, Siyu Li, and Xiumei Wang. Inference-time scaling for visual autoregressive modeling by searching representative samples. In *PRCV*, pages 406–419, 2026. doi: 10.1007/978-981-95-5699-1_28.

[11] Haodong Li, Chunmei Qing, Huanyu Zhang, Dongzhi Jiang, Yihang Zou, Hongbo Peng, Dingming Li, Yuhong Dai, ZePeng Lin, Juanxi Tian, et al. CoCo: Code as CoT for text-to-image preview and rare concept generation. In *ECCV*, pages 57–75, 2026.

[12] Haodong Li, Tianfei Ren, Xiaoxiao Ma, Chunmei Qing, Zhen Fang, Sipeng He, Ziyu Guo, Haoyu Wu, Juanxi Tian, Yihang Zou, et al. VideoCoCo: Code-as-CoT for physically-consistent video generation via an agentic dual-engine system. *arXiv preprint arXiv:2607.27380*, 2026.

[13] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *CoRL*, pages 2165–2183, 2023.

[14] Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, et al. PaLM-E: An embodied multimodal language model. In *ICML*, pages 8469–8488, 2023.

[15] Zhiming Liu, Yikun Miao, Ying Chen, Hongrui Yin, Fangqi Zhu, Xiaoyi Pang, Quanxin Shou, Zhengyang Yan, Haodong Wang, and Song Guo. World action learning via interaction-centric spectral latent guidance. *arXiv preprint arXiv:2610.03607*, 2026.

[16] Jingwei Shi, Xinxiang Yin, Jing Huang, Shengyu Tao, and Jinman Zhao. CodeHacker: Automated test case generation for detecting vulnerabilities in competitive programming solutions. In *ACL*, pages 2352–2382, 2026. doi: 10.18653/v1/2026.acl-long.108.

[17] Dechen Zhang, Xuan Tang, Xinxiang Yin, Xingwu Chen, Jian Qian, and Difan Zou. VALG: An agentic system for ml theory research, 2026.

[18] Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-Harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*, 2026.

[19] Hangfan Zhang, Shao Zhang, Kangcong Li, Chen Zhang, Yang Chen, Yiqun Zhang, Lei Bai, and Shuyue Hu. Self-Harness: Harnesses that improve themselves. *arXiv preprint arXiv:2606.09498*, 2026.

[20] Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, et al. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses. *arXiv preprint arXiv:2604.25850*, 2026.

[21] Jianglin Lu, Hailing Wang, Xu Ma, Qihua Dong, Mingyuan Zhang, Yizhou Wang, and Yun Fu. MUSE: A unified agentic harness for MLLMs. *arXiv preprint arXiv:2606.03005*, 2026.

[22] Hyunin Lee, Jinglue Xu, Jeffrey Seely, Donghyun Lee, Matei Zaharia, and Yujin Tang. Recursive harness self-improvement. *arXiv preprint arXiv:2607.15524*, 2026.

[23] Yuhao Wu, Jingyuan Zhang, Jiajun Shi, Xinping Lei, Qingshui Gu, Yuxuan Zhang, Zexuan Wang, Chen He, Chen Huang, Maojia Song, et al. HarnessDev: Can LLMs create and evolve their own agent harness? *arXiv preprint arXiv:2609.01437*, 2026.

[24] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Gödel Machine: Open-ended evolution of self-improving agents. In *ICLR*, 2026.

[25] Guibin Zhang, Leo Lu, Fangzhou Xie, Kang Zhu, Junhao Wang, Zhifei Xie, Zhaochen Yu, Zihang Liu, Zhongxiang Sun, Qiankun Li, et al. JIT-Agent: Scaling harness intelligence via just-in-time harness evolution. *arXiv preprint arXiv:2608.25593*, 2026.

[26] Siwei Wu, Jincheng Ren, Yizhi Li, Haau-Sing Li, Chengran Yang, Xudong Ma, Yuxuan Zhang, Weicheng Gu, Jian Yang, Riza Batista-Navarro, et al. ModularRSI: Modular and generalizable recursive harness self-improvement. *arXiv preprint arXiv:2609.14857*, 2026.

[27] Bo Liu, Simon Yu, Yiding Jiang, Ao Qu, Andrew Zhao, Zichen Liu, Junsu Kim, Zijian Zhou, Seungone Kim, Tongzheng Ren, et al. SPADE: Self-play in adaptive synthetic executable environments. *arXiv preprint arXiv:2608.19197*, 2026.

[28] Yuxing Wang, Zhiyu Chen, Tiantian Zhang, Qiyue Yin, Yongzhe Chang, Zhiheng Li, Liang Wang, and Xueqian Wang. Embodied co-design for rapidly evolving agents: Taxonomy, frontiers, and challenges. *arXiv preprint arXiv:2512.04770*, 2025.

[29] Fang Yuan, Junjie Zeng, Qinglun Li, Long Qin, Quanjun Yin, Siqi Shen, Yuxiang Xie, and Junqiang Yang. PACE: Parameter change for unsupervised environment design. In *ICML*, pages 152219–152240, 2026.

[30] Zhiqing Cui, Xinxiang Yin, Yihong Tang, Xinglang Zhang, Yuanzhe Hu, Siru Zhong, Weidong Tang, Yuxuan Liang, Weijia Li, Ming Jin, Shirui Pan, Yuhao Kang, Dingyi Zhuang, and Jinhua Zhao. EarthVerse: Benchmarking scientific agents across dynamic earth systems and natural hazards, 2026.

[31] Weidong Tang, Jierui Li, Yueling Hou, Zihan Mei, Can Zhang, Xinyan Wan, Zhiyuan Liang, Pengfei Zhou, Yang You, and Wangbo Zhao. GroupToM-Bench: Benchmarking group theory of mind and nonlinear social emergence in MLLMs. In *ACL*, pages 40007–40031, 2026. doi: 10.18653/v1/2026.acl-long.1859.

[32] Mengzhuo Chen, Junjie Wang, Zhe Liu, Yawen Wang, Haiming Zheng, and Qing Wang. From failed trajectories to reliable LLM agents: Diagnosing and repairing harness flaws. *arXiv preprint arXiv:2606.06324*, 2026.

[33] Xinghua Lou, Miguel Lázaro-Gredilla, Antoine Dedieu, Carter Wendelken, Wolfgang Lehrach, and Kevin P. Murphy. AutoHarness: Improving LLM agents by automatically synthesizing a code harness. *arXiv preprint arXiv:2603.03329*, 2026.

[34] Tailin Zhou. Hierarchical self-improvement: A framework for task-specific evolvable agent harnesses. *arXiv preprint arXiv:2608.08466*, 2026.

[35] Shuai Shao, Kangning Zhang, Qingyao Li, Shijian Wang, Hao Wang, Wenxiang Jiao, Yuan Lu, Yi Guo, Weiwen Liu, and Weinan Zhang. Harness-R1: Learning to edit executable runtime harnesses from agent failure trajectories. *arXiv preprint arXiv:2608.02276*, 2026.

[36] ARC Prize Foundation. ARC-AGI-3: A new challenge for frontier agentic intelligence. *arXiv preprint arXiv:2603.24621*, 2026.

[37] United Kingdom Micromouse and Robotics Society (UKMARS). UK Micromouse Classic Rules. https://ukmars.org/contests/contest-rules/micromouse-classic/. Accessed 2026-10-26.

[38] Davide Paglieri, Bartłomiej Cupiał, Samuel Coward, Ulyana Piterbarg, Maciej Wolczyk, Akbir Khan, Eduardo Pignatelli, Łukasz Kuciński, Lerrel Pinto, Rob Fergus, et al. BALROG: Benchmarking agentic LLM and VLM reasoning on games. In *ICLR*, 2025.

[39] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *NeurIPS*, 2024.

[40] Pengfei Zhou, Zhiwei Tang, Yixing Ma, Jiasheng Tang, Yizeng Han, Zhenglin Wan, Fanqing Meng, Wei Wang, Bohan Zhuang, Wangbo Zhao, et al. Agent-as-a-Router: Agentic model routing for coding tasks. *arXiv preprint arXiv:2606.22902*, 2026.

[41] John Golden. Eleusis Express. Gathering 4 Gardner, https://www.gathering4gardner.org/wp-content/uploads/2020/04/G4G14-EleusisExpress-Instructions.pdf, 2020. Express variation of Robert Abbott's Eleusis; accessed 2026-10-26.

[42] Jonas Golde, Patrick Haller, Fabio Barth, and Alan Akbik. MastermindEval: A simple but scalable reasoning benchmark. *arXiv preprint arXiv:2503.05891*, 2025.

[43] Eryk Banatt, Jonathan Cheng, Skanda Vaidyanath, and Tiffany Hwu. WILT: A multi-turn, memorization-robust inductive logic benchmark for LLMs. *arXiv preprint arXiv:2410.10998*, 2024.

[44] Kanishk Gandhi, Michael Y. Li, Lyle Goodyear, Agam Bhatia, Louise Li, Aditi Bhaskar, Mohammed Zaman, and Noah D. Goodman. BoxingGym: Benchmarking progress in automated experimental design and model discovery. *arXiv preprint arXiv:2501.01540*, 2025.

[45] Matt L. Wiemann, Lindsay M. Smith, Peter Melchior, Siddharth Mishra-Sharma, Andrew Gordon Wilson, Pavel Izmailov, and Carolina Cuesta-Lázaro. DiscoverPhysics: Benchmarking LLMs for out-of-the-box scientific thinking. *arXiv preprint arXiv:2605.26087*, 2026.

[46] Stephen Lavelle. PuzzleScript: Open source HTML5 puzzle game engine. https://github.com/increpare/PuzzleScript, 2026. Accessed 2026-10-26.

[47] Yizhe Zhang, Jiarui Lu, and Navdeep Jaitly. Probing the multi-turn planning capabilities of LLMs via 20 question games. In *ACL*, pages 1495–1516, 2024.

[48] Haonan Duan, Stephen Zhewen Lu, Caitlin Fiona Harrigan, Nishkrit Desai, Jiarui Lu, Michał Koziarski, Leonardo Cotta, and Chris J. Maddison. Measuring scientific capabilities of language models with a systems biology dry lab. In *NeurIPS Datasets and Benchmarks Track*, 2025.

[49] Ruairidh M. Battleday, Kai Sandbrink, Jimi Cullen-Drohan, Zihan Yan, Timothy Muller, Clare Maguire, Ales Kubicek, Fraser Greenlee-Scott, Sukrit Sumant, Tri Dao, et al. DiG-bench: Discovery in games. *arXiv preprint arXiv:2608.12593*, 2026.

[50] Lichess. Lichess open puzzle database. https://database.lichess.org/#puzzles, 2026. Accessed 2026-10-26.

[51] robx. pzprjs: Playing and editing puzzles by JavaScript. https://github.com/robx/pzprjs, 2026. Accessed 2026-10-26.

[52] swaroopg92. Penpa+: Web application to create and solve sudokus and puzzles. https://github.com/swaroopg92/penpa-edit, 2026. Accessed 2026-10-26.

[53] World Cube Association. WCA regulations. https://www.worldcubeassociation.org/regulations/, 2025. Release of 17 July 2025; accessed 2026-10-26.

[54] Jiayu Liu, Cheng Qian, Zhenhailong Wang, Bingxuan Li, Jiateng Liu, Qing Zong, Heng Wang, Jeonghwan Kim, Yumeng Wang, Bingxiang He, et al. AdaPlanBench: Evaluating adaptive planning in large language model agents under world and user constraints. In *COLM*, 2026.

[55] Graham W. Pulford. *High-Security Mechanical Locks: An Encyclopedic Reference*. Butterworth-Heinemann, 2007. ISBN 9780750684378.

[56] Justin Waugh. Pencil Puzzle Bench: A benchmark for multi-step verifiable reasoning. *arXiv preprint arXiv:2603.02119*, 2026.

[57] Mikayel Samvelyan, Robert Kirk, Vitaly Kurin, Jack Parker-Holder, Minqi Jiang, Eric Hambro, Fabio Petroni, Heinrich Küttler, Edward Grefenstette, and Tim Rocktäschel. MiniHack the planet: A sandbox for open-ended reinforcement learning research. In *NeurIPS Datasets and Benchmarks Track*, 2021.

[58] Yuki Imajuku, Kohki Horie, Yoichi Iwata, Kensho Aoki, Naohiro Takahashi, and Takuya Akiba. ALE-Bench: A benchmark for long-horizon objective-driven algorithm engineering. *arXiv preprint arXiv:2506.09050*, 2025.

[59] Tian Xia, Tianrun Gao, Wenhao Deng, Long Wei, Xiaowei Qian, Chenglei Yu, and Tailin Wu. BuildArena: A physics-aligned interactive benchmark of LLMs for engineering construction. In *ICML*, 2026.

[60] European Committee for Standardization. EN 15635:2008 – steel static storage systems – application and maintenance of storage equipment. Technical report, CEN, 2009. BS EN 15635:2008, British Standards Institution.

[61] Yuhang Jiang. PInVerify: An offline embodied benchmark for active instance verification. *arXiv preprint arXiv:2605.30639*, 2026.

[62] Jurgis Pasukonis, Timothy Lillicrap, and Danijar Hafner. Evaluating long-term memory in 3D mazes. *arXiv preprint arXiv:2210.13383*, 2022.

[63] Euro NCAP. Test protocol – AEB car-to-car systems. Technical Report Version 4.3, European New Car Assessment Programme, December 2023. URL https://cdn.euroncap.com/cars/assets/euro_ncap_aeb_c2c_test_protocol_v43_1e6ed06def.pdf.

[64] Xinrun Xu, Pi Bu, Ye Wang, Börje F. Karlsson, Ziming Wang, Tengtao Song, Qi Zhu, Jun Song, Zhiming Ding, and Bo Zheng. DeepPHY: Benchmarking agentic VLMs on physical reasoning. *arXiv preprint arXiv:2508.05405*, 2025.

[65] Evan Kiefl. Pooltool: A Python package for realistic billiards simulation. *Journal of Open Source Software*, 9(101):7301, 2024.

[66] Enxin Song, Yinuo Xu, Shusheng Yang, Wenhao Chai, and Jiatao Gu. Video-Index: A curated meta-benchmark for video understanding. *arXiv preprint arXiv:2610.00960*, 2026.

[67] Fengda Zhu, Xiwen Liang, Yi Zhu, Qizhi Yu, Xiaojun Chang, and Xiaodan Liang. SOON: Scenario oriented object navigation with graph-based exploration. In *CVPR*, pages 12689–12699, 2021.

[68] Letian Fu, Justin Yu, Karim El-Refai, Ethan Kou, Haoru Xue, Huang Huang, Wenli Xiao, Li Fei-Fei, Guanya Shi, Jiajun Wu, et al. CaP-X: A framework for benchmarking and improving coding agents for robot manipulation. In *ICML*, 2026.

[69] Ziyue Wang, Chi Chen, Fuwen Luo, Yurui Dong, Yuanchi Zhang, Yuzhuang Xu, Xiaolong Wang, Peng Li, and Yang Liu. ActiView: Evaluating active perception ability for multimodal large language models. In *ACL*, pages 7605–7633, 2025.

[70] Anton Bakhtin, Laurens van der Maaten, Justin Johnson, Laura Gustafson, and Ross Girshick. PHYRE: A new benchmark for physical reasoning. In *NeurIPS*, 2019.

[71] Chaoyou Fu, Haozhi Yuan, Yuhao Dong, Yi-Fan Zhang, Yunhang Shen, Xiaoxing Hu, Xueying Li, Jinsen Su, Chengwu Long, Xiaoyao Xie, et al. Video-MME-v2: Towards the next stage in benchmarks for comprehensive video understanding. *arXiv preprint arXiv:2604.05015*, 2026.

[72] Satvik Golechha and Adrià Garriga-Alonso. Among Us: A sandbox for measuring and detecting agentic deception. In *NeurIPS*, 2025.

[73] Frederic Moyersoen. Saboteur: Rules of play, 20 year anniversary edition. AMIGO Spiel + Freizeit GmbH, https://blog.amigo-spiele.de/content/ap/rule/02402-GB-AmigoRule.pdf, 2023. Accessed 2026-10-26.

[74] Matthew Stephenson, Matthew Sidji, and Benoît Ronval. Codenames as a benchmark for large language models. *IEEE Transactions on Games*, pages 1–12, 2025.

[75] Leon Guertler, Bobby Cheng, Simon Yu, Bo Liu, Leshem Choshen, and Cheston Tan. TextArena. *arXiv preprint arXiv:2504.11442*, 2025.

[76] Eilam Shapira, Omer Madmon, Itamar Reinman, Samuel Joseph Amouyal, Roi Reichart, and Moshe Tennenholtz. GLEE: A unified framework and benchmark for language-based economic environments. *arXiv preprint arXiv:2410.05254*, 2024.

[77] Nolan Bard, Jakob N. Foerster, Sarath Chandar, Neil Burch, Marc Lanctot, H. Francis Song, Emilio Parisotto, Vincent Dumoulin, Subhodeep Moitra, Edward Hughes, et al. The Hanabi challenge: A new frontier for AI research. *Artificial Intelligence*, 280:103216, 2020.

[78] Google DeepMind. Game Arena harness. https://github.com/google-deepmind/game_arena, 2026. Accessed 2026-10-26.

[79] Marc Lanctot, Edward Lockhart, Jean-Baptiste Lespiau, Vinicius Zambaldi, Satyaki Upadhyay, Julien Pérolat, Sriram Srinivasan, Finbarr Timbers, Karl Tuyls, Shayegan Omidshafiei, et al. OpenSpiel: A framework for reinforcement learning in games. *arXiv preprint arXiv:1908.09453*, 2019.

[80] Long Phan, Mantas Mazeika, Andy Zou, and Dan Hendrycks. TextQuests: How good are LLMs at text-based video games? *arXiv preprint arXiv:2507.23701*, 2025.

[81] Hinterland Studio. The Long Dark. Video game, https://www.thelongdark.com/, 2017. Accessed 2026-10-26.

[82] Heinrich Küttler, Nantas Nardelli, Alexander H. Miller, Roberta Raileanu, Marco Selvatici, Edward Grefenstette, and Tim Rocktäschel. The NetHack learning environment. In *NeurIPS*, pages 7671–7684, 2020.

[83] Danijar Hafner. Benchmarking the spectrum of agent capabilities. In *ICLR*, 2022.

[84] Anthropic. System card: Claude Opus 5. https://anthropic.com/claude-opus-5-system-card, July 2026. Accessed 2026-10-26.

[85] OpenAI. Codex CLI: Lightweight coding agent that runs in your terminal. https://github.com/openai/codex, 2025. Accessed 2026-10-26.

[86] DeepSeek-AI. DeepSeek Harness: Everything is a plugin. https://github.com/deepseek-ai/deepseek-harness, 2026. Accessed 2026-10-26.

[87] Jürgen Schmidhuber. Gödel machines: Fully self-referential optimal universal self-improvers. In Ben Goertzel and Cassio Pennachin, editors, *Artificial General Intelligence*, Cognitive Technologies, pages 199–226. Springer, 2007.

[88] Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-taught optimizer (STOP): Recursively self-improving code generation. In *COLM*, 2024.

[89] Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, Xiaojun Wan, and William Yang Wang. Gödel Agent: A self-referential agent framework for recursively self-improvement. In *ACL*, pages 27890–27913, 2025.

[90] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *ICLR*, 2025.

[91] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xiong-Hui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, et al. AFlow: Automating agentic workflow generation. In *ICLR*, 2025.

[92] Yu Shang, Yu Li, Keyu Zhao, Likai Ma, Jiahe Liu, Fengli Xu, and Yong Li. AgentSquare: Automatic LLM agent search in modular design space. In *ICLR*, 2025.

[93] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *ICLR*, 2024.

[94] Chrisantha Fernando, Dylan Sunil Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution. In *ICML*, pages 13481–13544, 2024.

[95] Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. TextGrad: Automatic "differentiation" via text. *arXiv preprint arXiv:2406.07496*, 2024.

[96] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-Refine: Iterative refinement with self-feedback. In *NeurIPS*, 2023.

[97] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.

[98] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners. In *AAAI*, volume 38, pages 19632–19642, 2024.

[99] Jun Nie, Yonggang Zhang, Jun Song, Qianshu Cai, Dahai Yu, Yike Guo, Xinmei Tian, and Bo Han. TTHE: Test-time harness evolution. *arXiv preprint arXiv:2607.08124*, 2026.

[100] Zefeng Wang, Minxi Yan, Jinhe Bi, Sikuan Yan, Volker Tresp, and Yunpu Ma. MetaSkill-Evolve: Recursive self-improvement of LLM agents via two-timescale meta-skill evolution. *arXiv preprint arXiv:2607.05297*, 2026.

[101] Zihan Tan, Leixin Sun, Zitong Shi, Yitao Liu, Jiajun Wu, Nathaniel Brooks, Jiaru Qian, Xiaoran Shang, Suyuan Huang, Yi Ding, et al. MetaRSI / RSI2: A meta-recursive self-improving system for recursive self-improving systems themselves. *arXiv preprint arXiv:2609.06396*, 2026.

[102] Yilun Yao, Xinyu Tan, Chao-Hsuan Liu, Yaoming Li, Zhengyang Wang, Zongwei Lv, Wenhan Yu, Zhewen Tan, Yuxuan Tian, Lin Sun, et al. Harness-Bench: Evaluating agent harnesses across models and realistic workflows. *arXiv preprint arXiv:2605.27922*, 2026.

[103] Zhe Ren, Yimeng Chen, Dandan Guo, Guowei Rong, Tonghui Li, R. B. Xiong, Qingfeng Lan, Wenyi Wang, Nanbo Li, Yibo Yang, et al. Self-improvements in modern agentic systems: A survey. *arXiv preprint arXiv:2607.13104*, 2026.

[104] Tingyang Chen, Shuo Lu, Kang Zhao, Weicheng Meng, Hanlin Teng, Tianhao Li, Chao Li, Xule Liu, Jian Liang, Zhizhong Zhang, et al. HarnessX: A composable, adaptive, and evolvable agent harness foundry. *arXiv preprint arXiv:2606.14249*, 2026.

[105] Borui Kang, Jinrui Gu, Junhan Lv, Wenbin Li, Lei Wang, and Yang Gao. Harness continual learning: Continual adaptation beyond model parameters. *arXiv preprint arXiv:2608.19013*, 2026.

[106] Zewen Liu, Zhan Shi, Yisi Sang, Bing He, Minhua Lin, Tianxin Wei, Dakuo Wang, Benoit Dumoulin, Wei Jin, and Hanqing Lu. Adaptive Auto-Harness: Sustained self-improvement for agentic system deployment on open-ended task streams. *arXiv preprint arXiv:2606.01770*, 2026.

[107] Jonathan C. Brant and Kenneth O. Stanley. Minimal criterion coevolution: A new approach to open-ended search. In *GECCO*, pages 67–74, 2017.

[108] Rui Wang, Joel Lehman, Jeff Clune, and Kenneth O. Stanley. POET: Open-ended coevolution of environments and their optimized solutions. In *GECCO*, pages 142–151, 2019.

[109] Rui Wang, Joel Lehman, Aditya Rawal, Jiale Zhi, Yulun Li, Jeffrey Clune, and Kenneth Stanley. Enhanced POET: Open-ended reinforcement learning through unbounded invention of learning challenges and their solutions. In *ICML*, pages 9940–9951, 2020.

[110] Jack Parker-Holder, Minqi Jiang, Michael Dennis, Mikayel Samvelyan, Jakob Foerster, Edward Grefenstette, and Tim Rocktäschel. Evolving curricula with regret-based environment design. In *ICML*, pages 17473–17498, 2022.

[111] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *ICML*, pages 41–48, 2009.

[112] Alex Graves, Marc G. Bellemare, Jacob Menick, Rémi Munos, and Koray Kavukcuoglu. Automated curriculum learning for neural networks. In *ICML*, pages 1311–1320, 2017.

[113] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *TMLR*, 2024.

[114] Yihao Hu, Zhihao Wen, Xiujin Liu, Pan Wang, Xin Zhang, and Wei Wu. SEAL: Synergistic co-evolution of agents and learning environments. *arXiv preprint arXiv:2605.24426*, 2026.

[115] Ziyang Liu, Xinyan Guo, Xuchen Wei, Han Hao, and Liu Yang. Escher-Loop: Mutual evolution by closed-loop self-referential optimization. *arXiv preprint arXiv:2604.23472*, 2026.

[116] Guhong Chen, Yingcheng Shi, Yongbin Li, Binhua Li, Xander Xu, Hu Wei, Shiwen Ni, Min Yang, and Jieping Ye. EvoTrainer: Co-evolving LLM policies and training harnesses for autonomous agentic reinforcement learning. *arXiv preprint arXiv:2606.03108*, 2026.

[117] Prannay Hebbar, Yogendra Manawat, Samuel Verboomen, Alesia Ivanova, Selvam Palanimalai, Kunal Bhatia, and Vignesh Baskaran. SIA: Self improving AI with harness & weight updates. *arXiv preprint arXiv:2605.27276*, 2026.

[118] Abhay Zala, Jaemin Cho, Han Lin, Jaehong Yoon, and Mohit Bansal. EnvGen: Generating and adapting environments via LLMs for training embodied agents. In *COLM*, 2024.

[119] Haoqiang Kang, Xiaokang Ye, Yuhan Liu, Siddhant Hitesh Mantri, Lingjun Mao, James Fleming, Drishti Regmi, and Lianhui Qin. SimWorld Studio: Automatic environment generation with evolving coding agent for embodied agent learning. *arXiv preprint arXiv:2605.09423*, 2026.

[120] Guanting Dong, Junting Lu, Junjie Huang, Wanjun Zhong, Longxiang Liu, Shijue Huang, Zhenyu Li, Yang Zhao, Xiaoshuai Song, Xiaoxi Li, et al. Agent-World: Scaling real-world environment synthesis for evolving general agent intelligence. *arXiv preprint arXiv:2604.18292*, 2026.

[121] Kanishk Gandhi, Shivam Garg, Noah D. Goodman, and Dimitris Papailiopoulos. Endless terminals: Scaling RL environments for terminal agents. *arXiv preprint arXiv:2601.16443*, 2026.

[122] Zihao Cheng, Hongru Wang, Zeming Liu, Xinyi Wang, Xiangrong Zhu, Yuhang Guo, Wei Lin, Jeff Z. Pan, and Yunhong Wang. Terminal-World: Scaling terminal-agent environments via agent skills. *arXiv preprint arXiv:2605.20876*, 2026.

[123] Yuxin Zuo, Zikai Xiao, Li Sheng, Fei Huang, Jianhong Tu, Yuxuan Liu, Tianyi Tang, Xiaomeng Hu, Yang Su, Qingfeng Lan, et al. Qwen-AgentWorld: Language world models for general agents. *arXiv preprint arXiv:2606.24597*, 2026.

[124] Chaobo Jia, Ruipeng Wan, Ting Sun, Weihao Tan, Borui Wan, Yuxuan Tong, Guangming Sheng, and Hong Xu. GameGen-Verifier: Parallel keypoint-based verification for LLM-generated games via runtime state injection. *arXiv preprint arXiv:2605.07442*, 2026.

[125] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5026–5033, 2012.

A. Task Tree

The benchmark inventory comprises 44 benchmarks organized into five domains and 15 subclasses (Figure 9): 26 training benchmarks, 16 unseen benchmarks, and two embodied benchmarks, SOON [67] and CaP-X [68], each evaluated both in simulation and on a real robot (Appendix E). The five domains are Latent Rule & Mechanism Discovery; Constraint Solving, Puzzles & Design; Perception, Navigation & Control; Multi-Agent Strategy & Social Interaction; and Long-Horizon Open-Ended Environments. Counting the simulation version of each embodied benchmark as a training benchmark and its real-robot version as an unseen benchmark gives the 28 training and 18 unseen benchmarks of Section 4.1. Every subclass contains at least one training and one unseen benchmark.

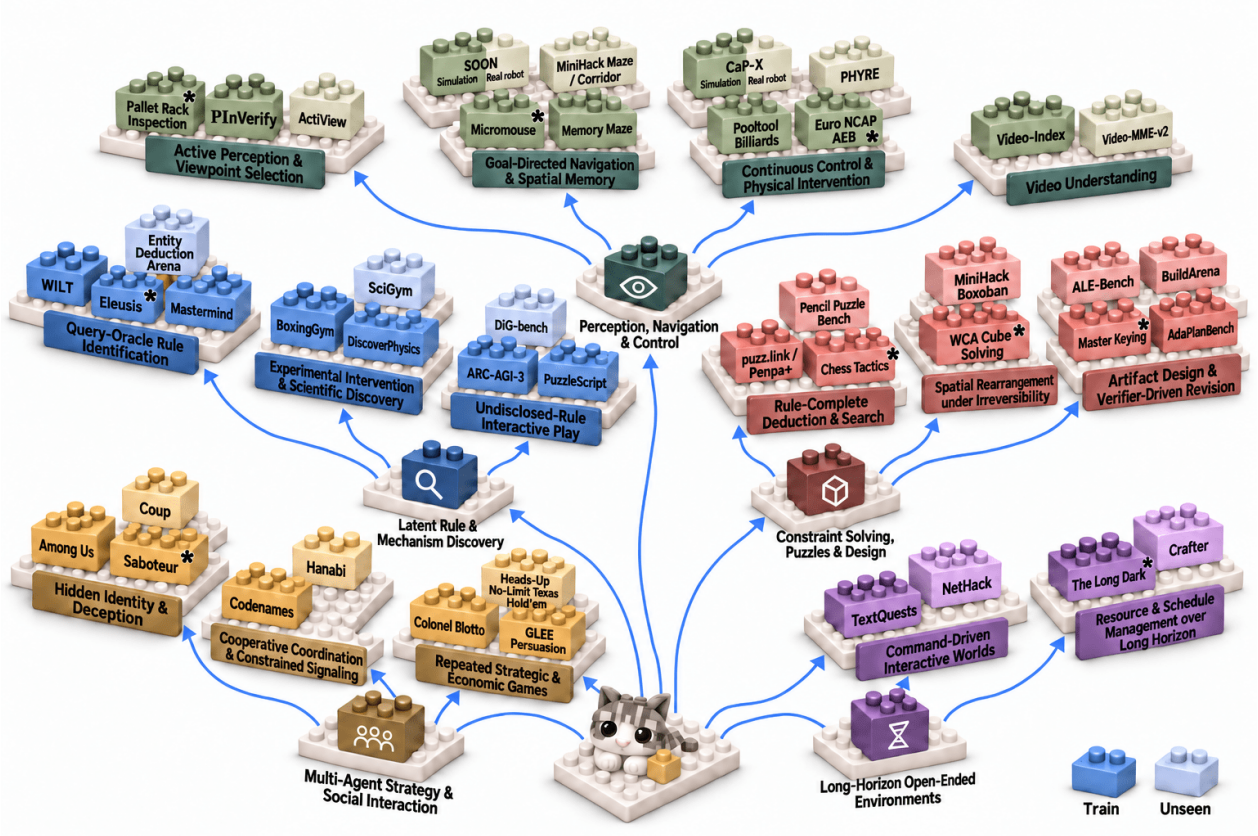

Figure 9 | Task tree. The 44 benchmarks are grouped into five domains and 15 subclasses. Training benchmarks take the colour of their domain and unseen benchmarks a lighter shade of it; SOON and CaP-X show their simulation and real-robot versions; * marks environments rebuilt from published rules.

Candidate benchmarks were collected through agent web search over five types of source: research benchmarks released with code or data, public AI competitions, human contests and testing standards, published games and puzzle communities, and professional practice. Because a multimodal agent must perceive its world through whatever interface that world exposes, the inventory spans every input modality in which agentic tasks are posed, and each benchmark retains the observation protocol of its source. Each modality probes a different part of an agent's capability. Image input (photographs, rendered scenes, puzzle images, and robot camera views) tests perception from pixels, viewpoint selection, and the grounding of decisions in what is seen. Video input tests evidence acquisition over time: selecting frames within a budget, tracking events across them, and understanding what

happened. Boards, grids, and maps rendered as text test spatial reasoning and state tracking over a symbolic visual state; with pixel-level perception removed, a failure can be attributed to reasoning rather than to seeing. Plain text tests the capabilities a multimodal agent relies on downstream of perception: rule discovery, experimental design, planning, negotiation, and strategic interaction. Keeping all four modalities in one inventory lets visual and text benchmarks share nodes in the capability tree (Appendix B).

Nine of the training sources publish their rules but provide no agent-facing environment: Chess Tactics [50], Eleusis [41], Euro NCAP AEB [63], Master Keying [55], Micromouse [37], Pallet Rack Inspection [60], Saboteur [73], The Long Dark [81], and WCA Cube Solving [53] (marked * in Figure 9). We rebuilt each of them through agent web search from the published rules and examples, preserving the mechanics, information access, and scoring of the source; the rebuilt environments present their state to the agent as images or symbolic text. Every unseen benchmark other than the two real-robot versions runs on its official environment or evaluation code, e.g., the BALROG harness [38], the PHYRE simulator [70], the DiG-bench server [49], the ALE-Bench judge [58], the Video-MME-v2 scorer [71], and the Besiege game of BuildArena [59].

B. Capability Tree

Independent agents propose capability categories from the task specifications and verified trajectories of the training benchmarks, a judge consolidates them, and reviewers independently check definitions, evidence, assignments, and coverage. The result is six capabilities and 30 defect types (Table 7): a Perception & State Tracking (PST), b Hypothesis & Counterpart Inference (HCI), c Constraint Checking & Feedback Repair (CFR), d Goal Commitment & Value Trade-off (GVT), e Video Evidence Acquisition (VEA), and f Video Comprehension (VC). Each defect type is an observable failure pattern that evidences a gap in its capability. Every training record is used for capability induction, tree construction, level generation, module evolution, and the Router's training memory. Unseen benchmarks are recorded only after all Harness training has finished, from their runs with H_0; these records go through the same categorization procedure and are anchored to the fixed taxonomy without changing it.

Table 7 | Capability and defect taxonomy. Columns count the training and unseen benchmarks whose trajectories expose each defect type; unseen counts come from H_0 runs recorded after all Harness training.

		Training	Unseen
a	**Perception & State Tracking**		
F01	Host action-protocol and output-format fidelity	4	5
F02	Observation-to-state grounding	3	4
F03	Action and parameter effect calibration	4	3
F04	Instruction-to-action grounding	2	0
F05	Outcome ledger: no re-issue of refuted or inert items	5	5
F06	Re-acquisition of out-of-window facts	7	0
F07	Exploration coverage and route reuse	4	4
b	**Hypothesis & Counterpart Inference**		
F08	Evidence-consistent hypothesis maintenance	3	1
F09	Discriminating probe design	3	2
F10	Data-to-model inference with assumption checks	1	1
F11	Counterpart modelling from revealed history	4	2
F12	Receiver-aware signalling	3	1
c	**Constraint Checking & Feedback Repair**		
F13	Pre-commit one-step constraint check	6	5
F14	Multi-step constraint propagation to forced moves	1	0
F15	Reachability lookahead before irreversible action	2	1
F16	Feasibility-checked complete plan	1	2
F17	Feedback-localized repair	2	2
F18	Non-convergence detection and method switching	9	9
d	**Goal Commitment & Value Trade-off**		
F19	Evidence-gated commitment	4	5
F20	Goal-discrepancy-directed action selection	4	0
F21	Goal-completing opportunity recognition	3	1
F22	Goal-predicate verification before terminal action	4	1
F23	Budget pacing and terminal-action reservation	5	4
F24	Scored-objective valuation of options and policies	6	6
e	**Video Evidence Acquisition**		
V01	Question-guided evidence acquisition	1	1
V02	Coverage-adequate frame sampling	1	0
V03	Non-visual channel access	1	1
f	**Video Comprehension**		
V04	Frame-grounded perception and order reading	1	1
V05	Motion, counting and 3D dynamics across frames	1	1
V06	Cross-frame event integration and inference	1	1

The six capabilities are parallel and non-overlapping, with no causal relation among them; ordered by the breadth of the domains each capability spans, the fixed order c, d, a, b, e, f defines the paths of the capability tree, and most benchmarks require more than one capability. The flat capability tree (Figure 10) shows this directly: each capability lists every benchmark that requires it, so a benchmark with several requirements appears under each of them.

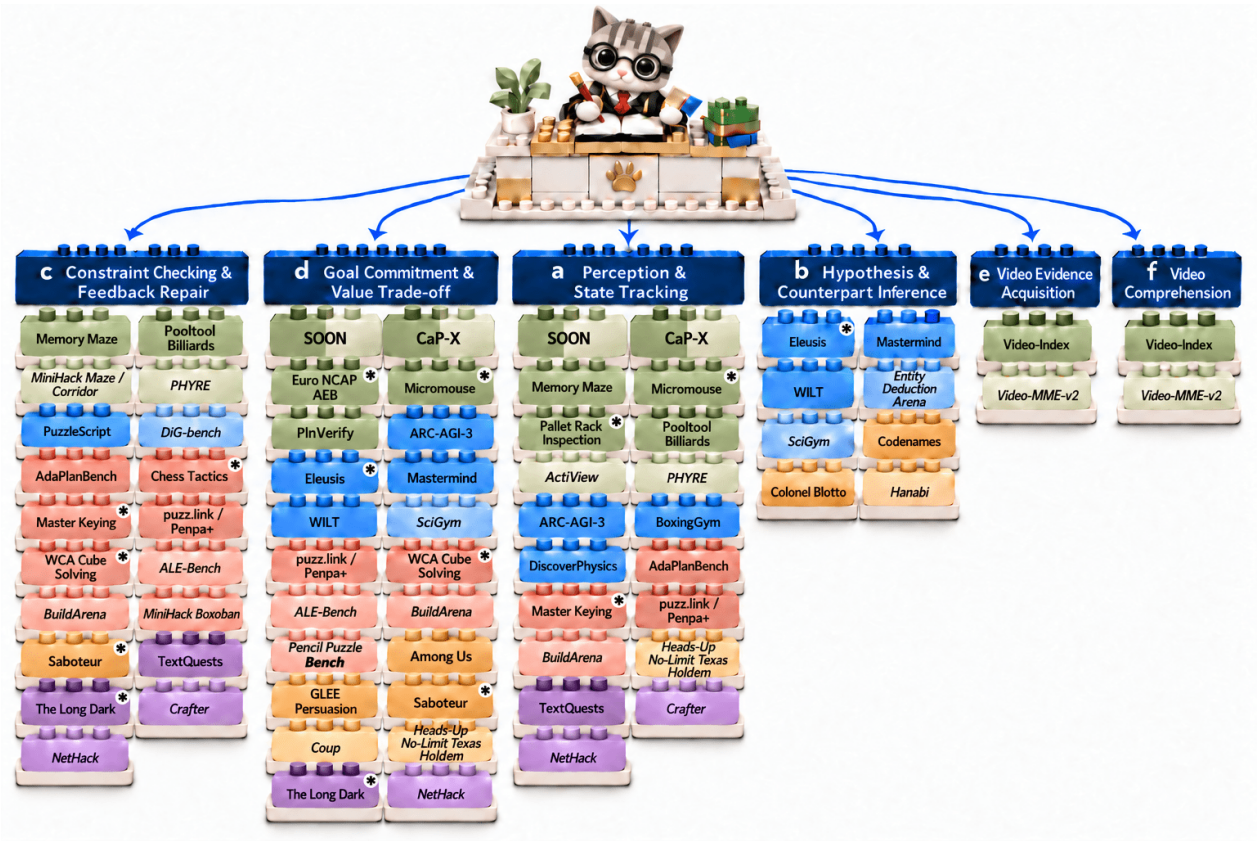

Figure 10 | Flat capability tree. Bricks keep the colours of Figure 9; unseen benchmarks are in italics; * marks rebuilt environments.

The capability tree (Figure 11) organizes the same assignment by combination: the requirement set of each training benchmark, sorted in this order, forms a path from the root, and these paths are merged along their common prefixes. Each node stands for one combination of capabilities, read along the branch from the root: for example, node c holds the benchmarks that require constraint checking alone, its child c→d those that also require goal commitment, and c→d→a those that further require perception and state tracking. A benchmark sits at the node of its own combination, regardless of its domain. AdaPlanBench, Master Keying, Memory Maze, Pooltool Billiards, and TextQuests, for instance, come from three domains and differ widely in content, from adaptive planning and lock-system design to maze navigation, billiard shots, and text adventures; yet they share the same capability requirements and therefore all sit at c→a. An edge adds a capability; it does not imply any dependency or causal relation between the two capabilities. The 11 non-root nodes define the shared structure of the Explorer and Creator Harness trees, and both evolve along the paths and structure of the capability tree (Appendix C). Unseen benchmarks are then placed at the nodes of their combinations without modifying the tree.

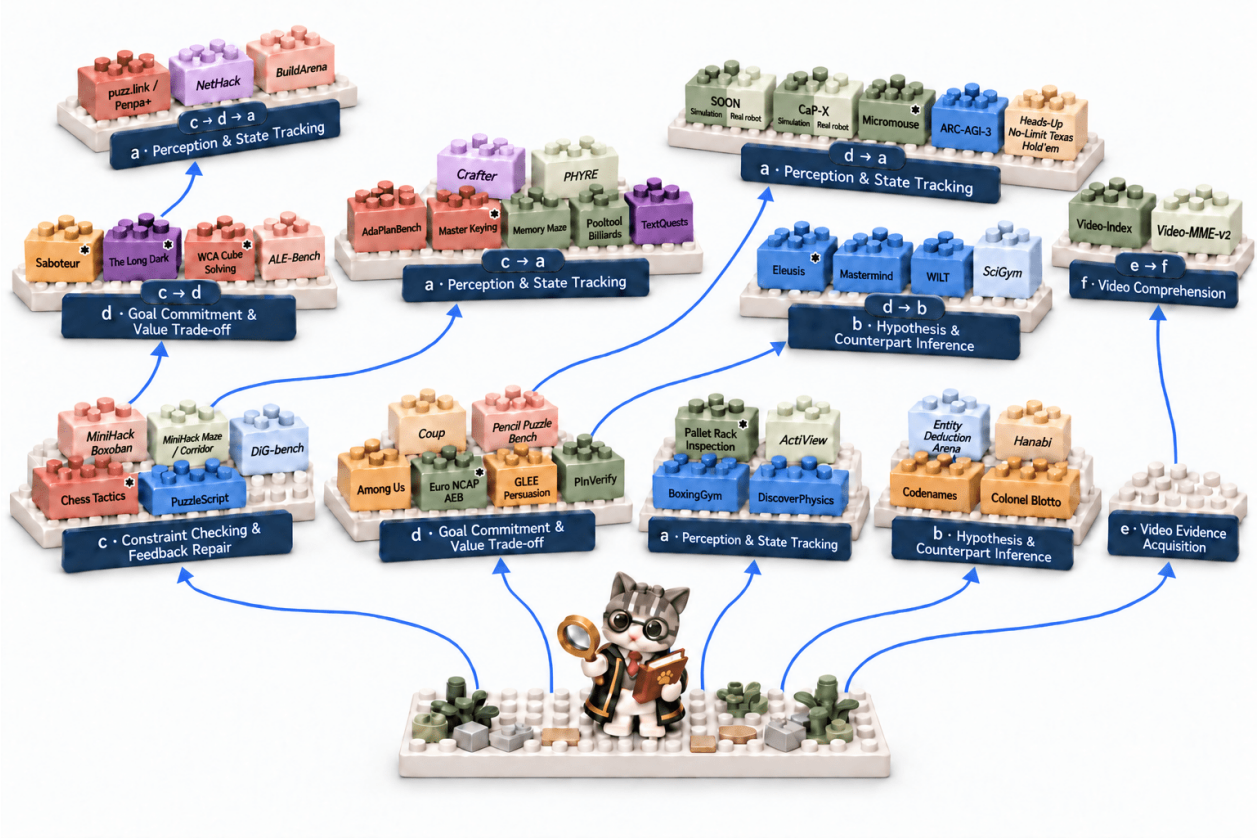

Figure 11 | Capability tree. Each node is one combination of capabilities, read along the branch from the root, and each benchmark sits at the node of its combination. Bricks keep the colours of Figure 9; unseen benchmarks are in italics; * marks rebuilt environments.

C. Harness Tree

Explorer and Creator both start from H_0 and evolve along the same capability tree, but they retain their learned modules differently. Explorer keeps a Harness at every node except e: each node selectively inherits its parent's modules and refines its own local modules, and the resulting Harness tree is retained for routing. Creator keeps a single shared library: within an iteration, its node Harnesses inherit updated parent modules, and at the end of the iteration the modules proposed at all nodes are consolidated with the current library into the Base Harness of the next iteration. Explorer's trajectories drive Evolver's local module revisions, whereas worlds rejected by the verifiers drive Creator's module proposals. Every module is a versioned function behind a prompt-facing interface and is installed only after the host checks its evidence, interface, and version lineage.

C.1. Explorer Harness Tree

Explorer plays every benchmark with the frozen backbone (Appendix D), and each instance uses the best-scoring compatible node Harness under the Oracle setting. Every node Harness starts from H_0, which has no learned module, and combines the inherited modules at their retained versions with the node's local modules; a benchmark at c→d→a therefore uses the modules of c, c→d, and c→d→a, subject to selective inheritance (Figure 12). At each node, Explorer plays the admitted training levels with these modules, and Evolver compares successful and failed trajectories to create or revise local modules; the host validates evidence, ownership, interfaces, and version lineage before every update, and ancestor modules are preserved. An updated parent is inherited only if it achieves strictly more host-verified task completions than the previously inherited modules under the same validation tasks, trial count, and budget; otherwise the previous modules are kept. A local module overrides an inherited module bound to the same identifier. Module calls take public inputs and return structured advice without advancing the environment, with at most two calls per step and none forced.

The catalog below lists the 29 modules M01–M29 of the Explorer Harness tree after the final iteration. Each was generated and revised during RSI; the modules are grouped by capability family: **a Perception & State Tracking:** Object Evidence Ledger and Rule Classification; Complete Inspection Coverage and Report Reconciliation; Identity Binding Across Reversed Views; Relational Orders, Identity Binding and Position Transforms; Sealed Observation Archives and Unordered Collection Tours; Ordered Prefix Validation and Capacity-Constrained Batch Planning; Execution Receipt Verification and State-Difference Memory; Stage Revision, Deduction Retraction and Execution Memory; Target Collection, Current-Phase Routing and Frontier Exploration; Product-State Routing for Ordered Collection; Phase-Dependent Edge Repair and Static Corridor Reuse. **b Hypothesis & Counterpart Inference:** Parity Reversal and Two-Period Signal Selection; Contingent Policy Search with Endogenous Signals; Opponent Inference, Allocation Scoring and Archive Value; Candidate Ledger, Dynamic State Alignment and Dual-Objective Probing; Decomposing Repeated Symbols into Multiplicity and Position. **c Constraint Checking & Feedback Repair:** Joint Gate Planning, Proposal Validation and Milestone Scheduling; Return Reachability Under Asymmetric Gates. **d Goal Commitment & Value Trade-off:** Investment Timing and Scored Value Trade-offs; Resource Reserves and Delivery Opportunity Trade-offs; Goal Verification, Stopping Decisions and Plan Value; Evidence Verification, Counterexample Stopping and Submission Budget; Relation Verification Within One Object and Coordinate Frame; Fixed Goal Conditions, Evidence Ledger and Bounded Commit Review. **e Video Evidence Acquisition:** Adaptive Frame, Region and Audio Evidence Acquisition; Delivered Evidence Coverage and Acquisition Budget Verifica-

tion; Evidence Gap Acquisition and Unproductive Retry Suppression. **f Video Comprehension:** Handoff Episode Aggregation, Event Ledger and Endpoint Audit; Candidate Identity Association Across Frames.

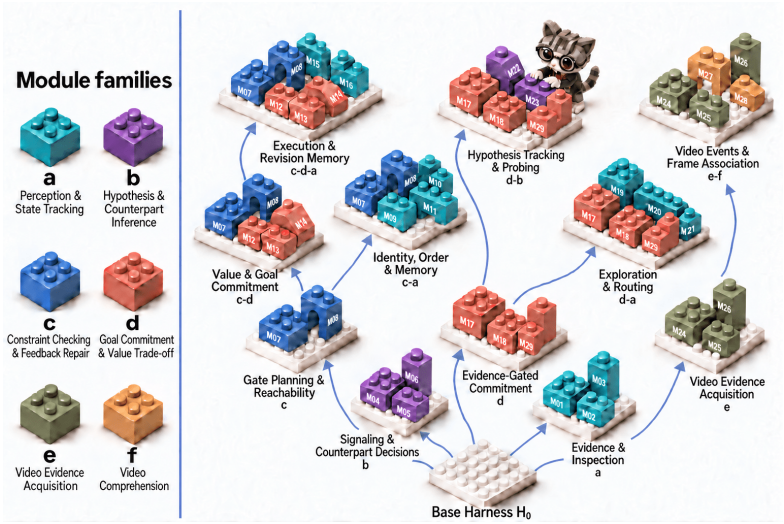

Figure 12 | Explorer Harness tree. Every node Harness starts from H_0 and combines the modules it selectively inherits with its own local modules. Brick colours mark capability families, and labels give module IDs.

C.2. Creator Shared Harness

Creator builds the simplified training worlds of each node with a coding harness, created per node and iteration, with a workspace for the world and an evidence directory outside it. Its Base Harness at the first iteration is H_0 extended with the pre-RSI modules listed below. Within each iteration, Creator visits the nodes top-down: the children of the root start from the Base Harness, and every other node inherits its parent's updated modules. Each node keeps its own guidance, one construction note, and one public action budget per iteration. Verifier-detected failures trigger local evolution, and at the end of the iteration the validated proposals of all nodes are consolidated with the current library in learning order (Equation 7): validated revisions replace their registered predecessors; among new modules, distinct capabilities are retained, partial overlaps are merged to preserve complementary functionality, and near-duplicates keep the earlier-learned module and discard the later one. The consolidated library becomes the Base Harness of the next iteration; it is shared by all nodes and exposed as a single tool that runs one module per call. The host, not the model, supplies the inputs of each call: the build digest, the submission allowance, and, for a rule module, a measurement of the build. Fields that the host does not supply remain unknown, and missing evidence never passes. Before submission, the host reruns the applicable audits and blocks any missing, stale, or conflicting result; Creator returns errors in the frozen task design or verifier contract to their owner instead of rewriting them.

In the module names, a witness is a recorded route run on the current build; the commitment cost is the number of extra actions that a wrong commitment costs over the direct route; the screening band is the performance range into which a world must fall; and a neutralized control is the previous version with the changed difficulty axis reset. The final Creator module library holds 25 modules (Figure 13). Eleven are pre-RSI modules, audits that form the initial module library of the Creator Harness and apply the host's formulas to recorded routes, claims, edits, or submissions: Commitment Cost

Arithmetic Audit, Witness and Action Ceiling Audit, Difficulty Axis Range and Feasibility Audit, Screening Band Direction Diagnosis, Layout Novelty Ledger, Effective Difficulty Change Audit, Single Change Attribution Audit, Repair Scope Gate, Claim and Execution Evidence Census, Bounded Search Declaration Audit, and Submission Budget and Repeated Rejection Guard. The other 14 are rule modules, predicate programs over one kind of host measurement learned from rejected worlds, each listed with the nodes at which it was formed: Production and Private Witness Isolation (d), Accepted Entry and Terminal Consistency (d→a), Reasoning Change and Direction Audit (a, c, d→a, e), Public Visual State and Probe Coverage Audit (a, b, c), Required Composition and Negative Route Audit (b, d→b), Target Attributes, Classes, and Acceptance Audit (a, b, c, d→b), Controlled Identity and Frame-State Alignment Audit (b, c, d→a), Action Polarity and Coordinate Basis Audit (a, b), Interaction Coverage and Decision-Time Visibility Audit (b, d), Neutralized Control Initialization and Completion Audit (b, d→a), Meaningful Exploratory Witness Pricing (e), Submission Artifact Completeness (a), Task Design Schema and Axis Type Handoff (c, d→a), and Authoritative Verifier Contract Handoff (all 11 nodes).

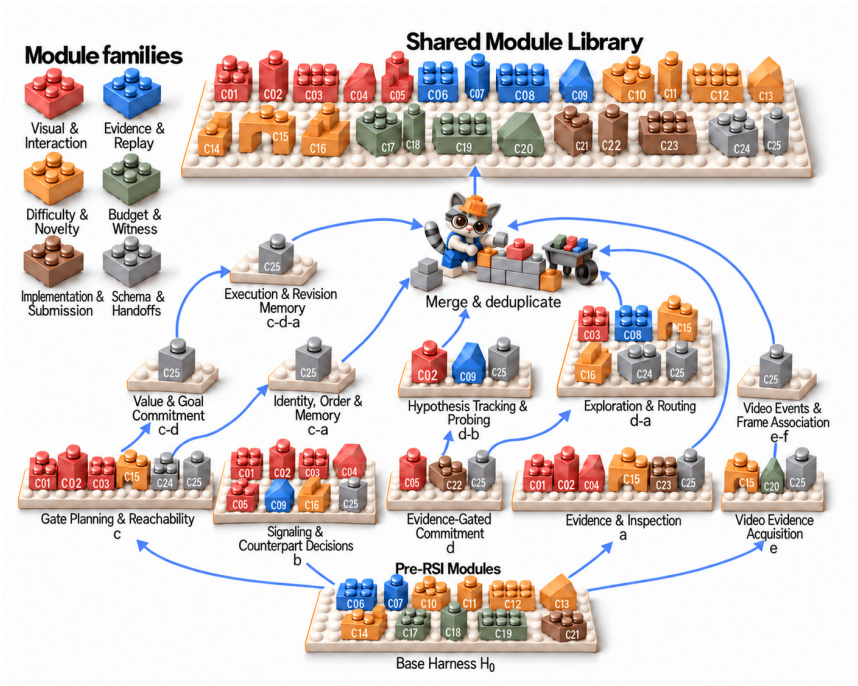

Figure 13 | Creator shared Harness. The root is the Base Harness H_0 with the pre-RSI modules. Each node shows the modules proposed or revised at that node. The modules of all nodes are merged and deduplicated into one shared library of 25 modules. Brick colours group the library by function.

D. Evaluation Details

Every run uses the same frozen backbone, Claude Opus 5 [84] with adaptive thinking at low effort, and the benchmark's native driver, which validates each action, returns the observation, and assigns the outcome. The Base Harness H_0 plays with no learned module and no skills; for $H^{(i)}$, each instance uses the best-scoring compatible node Harness under the Oracle setting, together with evaluation-time skills summarized from five preparation trials of that node Harness on the benchmark. Levels, action budgets, and scoring are fixed before the first run and shared by every Harness. Each benchmark reports performance on a 0–100 scale: task-completion, win, or solve rates where episodes pass or fail; the published bounded score where one exists; and, for ALE-Bench, the rank percentile among human entrants. The Codex Harness [85] and the DeepSeek Harness [86] ran the 16 unseen benchmarks other than the real-robot versions with the same backbone and settings through their native tool interfaces. Table 8 reports every score together with each benchmark's measure, number of distinct instances, and number of episodes; * marks environments rebuilt from source rules through agent web search. Train benchmarks use their fixed instance sets (Section 4.1), and ARC-AGI-3 covers the 25 games of the public set. Six unseen benchmarks run on a fixed subset of their official set: Pencil Puzzle Bench uses the official Golden 30 set, PHYRE the 50 test tasks of fold 0 in the within-template B tier, and Video-MME-v2 a stratified subset of 400 of the 800 videos (1,600 questions); DiG-bench uses 13 of the 21 public games, all games of tiers 1–3 and the first game of each of tiers 4–7; ALE-Bench uses 26 of the 40 problems and SciGym 126 of the 137 small systems, those on which the backbone's safety layer did not refuse the task inputs. NetHack is scored after a budget of 500 steps.

Table 8 | Full benchmark results. Scores on a 0–100 scale for the Base Harness H_0, the Harnesses $H^{(1)}$–$H^{(4)}$ after each RSI iteration, and the Codex and DeepSeek Harnesses on the unseen benchmarks; – marks runs that were not performed.

Benchmark	Measure, instances, episodes	Base H_0	$H^{(1)}$	$H^{(2)}$	$H^{(3)}$	$H^{(4)}$	Codex	DeepSeek
Train								
AdaPlanBench [54]	accuracy, 3 queries, 27 episodes	22.2	70.4	**74.1**	**74.1**	**74.1**	–	–
Among Us [72]	win rate, 3 games, 27 episodes	11.1	48.1	**63**	**63**	**63**	–	–
ARC-AGI-3 [36]	success rate, 25 games, 183 levels	22.4	41	45.9	50.3	**53.6**	–	–
BoxingGym [44]	success rate, 3 environments, 45 episodes	60	80	**84.4**	**84.4**	**84.4**	–	–
Chess Tactics* [50]	mate found, 3 positions, 27 episodes	66.7	92.6	96.3	96.3	**100**	–	–
Codenames [74]	win rate, 3 games, 27 episodes	33.3	66.7	**81.5**	**81.5**	**81.5**	–	–
Colonel Blotto [75]	win rate, 3 matches, 27 episodes	55.6	85.2	85.2	85.2	**88.9**	–	–
DiscoverPhysics [45]	pass rate, 2 worlds, 30 episodes	40	70	**73.3**	**73.3**	**73.3**	–	–
Eleusis* [41]	rule found, 6 rules, 90 episodes	43.3	**82.2**	81.1	81.1	81.1	–	–
Euro NCAP AEB* [63]	success rate, 3 scenarios, 9 episodes	66.7	77.8	**88.9**	**88.9**	**88.9**	–	–
GLEE Persuasion [76]	revenue ≥ 50%, 3 games, 27 episodes	33.3	**63**	**63**	**63**	**63**	–	–
Master Keying* [55]	success rate, 3 designs, 33 episodes	45.5	**84.8**	**84.8**	**84.8**	**84.8**	–	–
Mastermind [42]	solve rate, 3 codes, 45 episodes	53.3	**88.9**	**88.9**	**88.9**	**88.9**	–	–
Memory Maze [62]	success rate, 3 mazes, 9 episodes	0	**11.1**	**11.1**	**11.1**	**11.1**	–	–
Micromouse* [37]	success rate, 3 mazes, 27 episodes	55.6	66.7	66.7	**77.8**	**77.8**	–	–
Pallet Rack Inspection* [60]	success rate, 3 racks, 27 episodes	0	29.6	37	37	**59.3**	–	–
PInVerify [61]	accuracy, 3 pairs, 45 episodes	66.7	88.9	**93.3**	**93.3**	**93.3**	–	–
Pooltool Billiards [64, 65]	success rate, 3 seeds, 27 episodes	33.3	**66.7**	**66.7**	**66.7**	**66.7**	–	–
puzz.link / Penpa+ [51, 52]	solve rate, 3 puzzles, 9 episodes	33.3	**44.4**	**44.4**	**44.4**	**44.4**	–	–
PuzzleScript [46]	solve rate, 3 levels, 9 episodes	66.7	**77.8**	**77.8**	**77.8**	**77.8**	–	–
Saboteur* [73]	win rate, 3 games, 27 episodes	55.6	**77.8**	**77.8**	**77.8**	**77.8**	–	–
TextQuests [80]	success rate, 3 games, 9 episodes	0	11.1	11.1	11.1	**22.2**	–	–
The Long Dark* [81]	success rate, 3 runs, 27 episodes	77.8	**88.9**	**88.9**	**88.9**	**88.9**	–	–
Video-Index [66]	accuracy, 777 questions, 777 episodes	47.9	87.5	87.5	88.9	**89.4**	–	–
WCA Cube Solving* [53]	solve rate, 3 scrambles, 45 episodes	13.3	51.1	**80**	**80**	**80**	–	–
WILT [43]	accuracy, 8 rules, 120 episodes	80	**94.2**	**94.2**	**94.2**	**94.2**	–	–
SOON (simulation) [67]	success rate, 1 route, 10 episodes	0	20	20	**80**	**80**	–	–
CaP-X (simulation) [68]	success rate, 10 layouts, 10 episodes	10	20	30	40	**60**	–	–

continued on the next page

Benchmark	Measure, instances, episodes	Base H_0	$H^{(1)}$	$H^{(2)}$	$H^{(3)}$	$H^{(4)}$	Codex	DeepSeek
Unseen								
ActiView [69]	accuracy, 1,625 questions, 1,625 episodes	73.5	93.7	94.6	95.7	**97.4**	77.2	73.7
ALE-Bench [58]	percentile, 26 problems, 26 episodes	81.3	85.4	86.4	87.4	**91.9**	85	91.4
BuildArena [59]	pass rate, 9 levels, 45 episodes	73.3	93.3	95.6	95.6	**97.8**	77.8	80
Coup [75]	win rate, 40 games, 40 episodes	45	57.5	57.5	**65**	**65**	32.5	35
Crafter [83]	progress, 10 seeds, 10 episodes	51	56	57	58	60	**82.7**	70
DiG-bench [49]	levels passed, 13 games, 13 episodes	32.5	33.3	34.2	35.1	36.8	40.4	**54.4**
Entity Deduction Arena [47]	success rate, 200 entities, 200 episodes	64.5	**95**	**95**	94.5	**95**	72.5	67.5
Hanabi [77]	success rate, 40 games, 40 episodes	0	5	10	15	**20**	0	0
Heads-Up NL Hold'em [78, 79]	matches ahead, 20 matches, 20 episodes	60	80	80	80	**85**	30	35
MiniHack Boxoban [57]	solve rate, 10 levels, 10 episodes	0	10	20	20	**30**	10	0
MiniHack Maze / Corridor [57]	success rate, 20 levels, 20 episodes	55	80	80	80	**85**	60	75
NetHack [82]	progress, 5 seeds, 5 episodes, 500 steps	2.2	3	3	3	**4**	3.7	2.3
Pencil Puzzle Bench [56]	solve rate, 30 puzzles, 30 episodes	20	26.7	26.7	26.7	**30**	23.3	23.3
PHYRE [70]	solved, 50 tasks, 50 episodes	42	82	86	86	**90**	84	52
SciGym [48]	recovered, 126 systems, 126 episodes	25.4	45.2	47.6	50	**57.1**	26.2	33.3
Video-MME-v2 [71]	accuracy, 1,600 questions, 1,600 episodes	48.4	77.4	79.5	**80**	**80**	54.5	45.9
SOON (robot) [67]	success rate, 1 route, 10 episodes	0	20	20	**60**	**60**	–	–
CaP-X (robot) [68]	success rate, 10 layouts, 10 episodes	0	20	20	40	**60**	–	–
Train mean (26 benchmarks, excluding SOON and CaP-X)		41.68	67.17	71.03	71.68	**73.40**	–	–
Train mean (28 benchmarks)		39.06	63.80	67.75	70.85	**73.16**	–	–
Unseen mean (16 benchmarks, excluding SOON and CaP-X)		42.13	57.72	59.57	60.75	**64.06**	47.49	46.18
Unseen mean (18 benchmarks)		37.45	53.53	55.17	59.56	**63.61**	–	–

E. Visual Record

This appendix records what the agent sees and does. Appendix E.1 pairs H_0 and $H^{(4)}$ on the same configuration of each of the 44 benchmarks and shows where the two Harnesses part in reasoning and in outcome. Appendix E.2 follows the 11 capability nodes through four iterations of training levels, from the benchmarks that feed a node to the modules its levels produce. Appendix E.3 presents SOON and CaP-X in their reconstructed simulators and on the real robots, with the runs of H_0 and $H^{(4)}$ in both.

E.1. Interaction Process

Figures 14–57 show the actual decisions and outcomes of the agent on each of the 44 benchmarks (26 training, 16 unseen, and the two embodied benchmarks in simulation): on the same configuration, the different Harnesses lead H_0 and $H^{(4)}$ to different reasoning and judgements, and hence to different results.

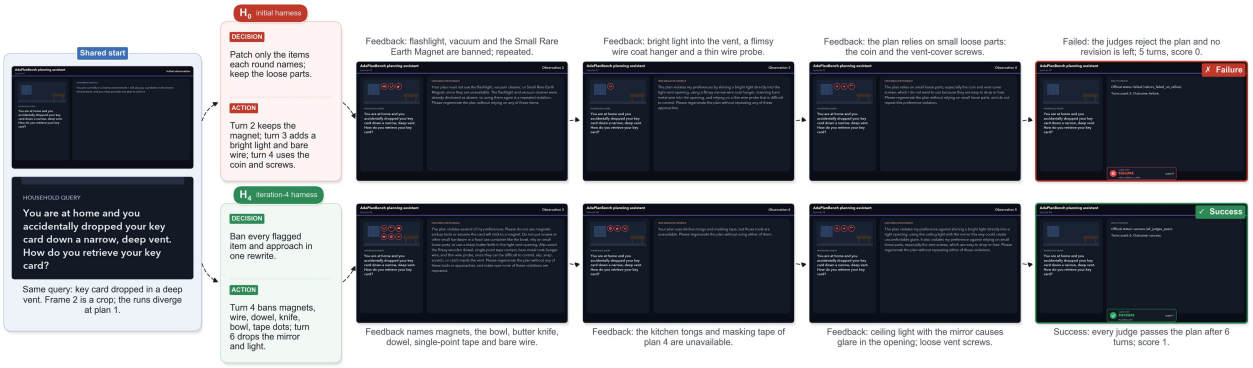

Figure 14 | AdaPlanBench (train)

Figure 15 | Among Us (train)

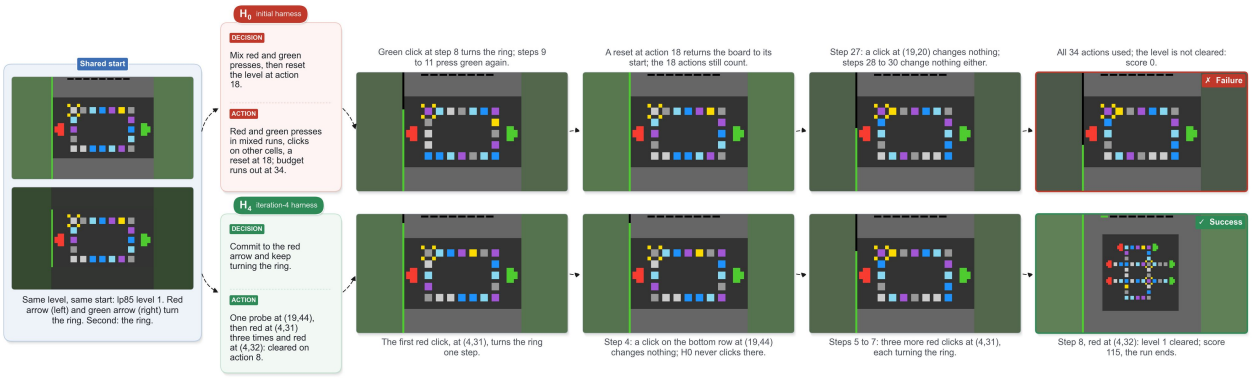

Figure 16 | ARC-AGI-3 (train)

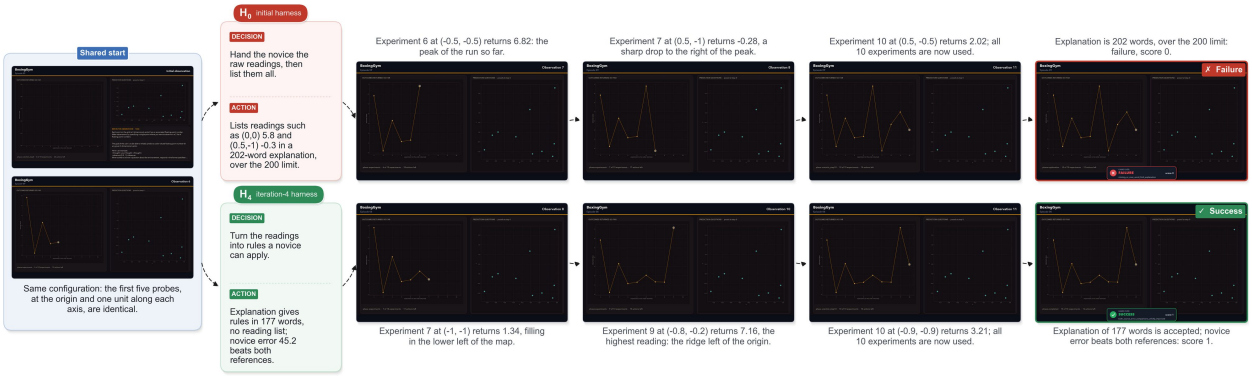

Figure 17 | BoxingGym (train)

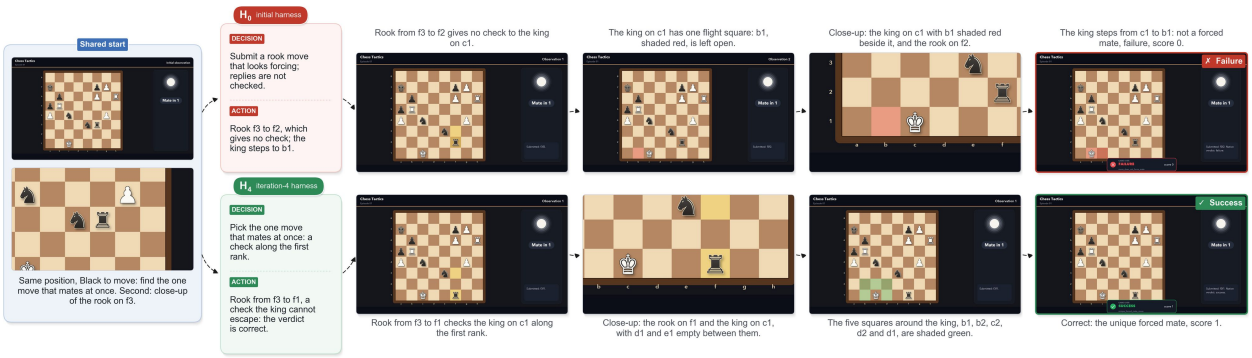

Figure 18 | Chess Tactics (train)

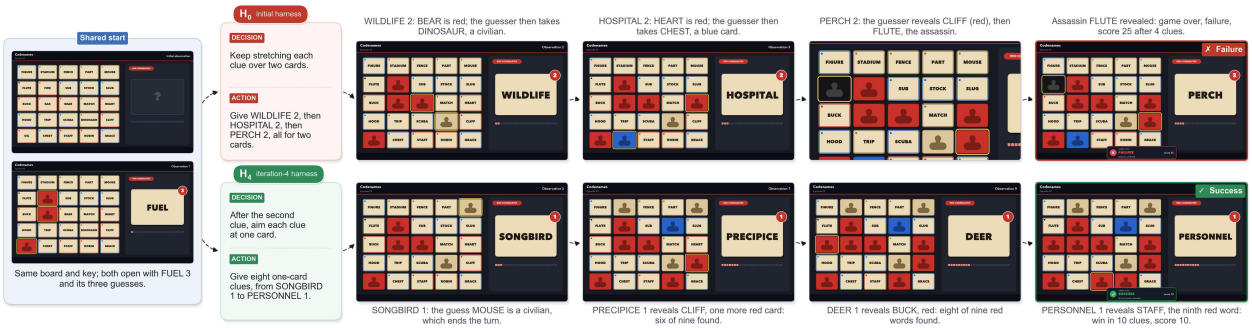

Figure 19 | Codenames (train)

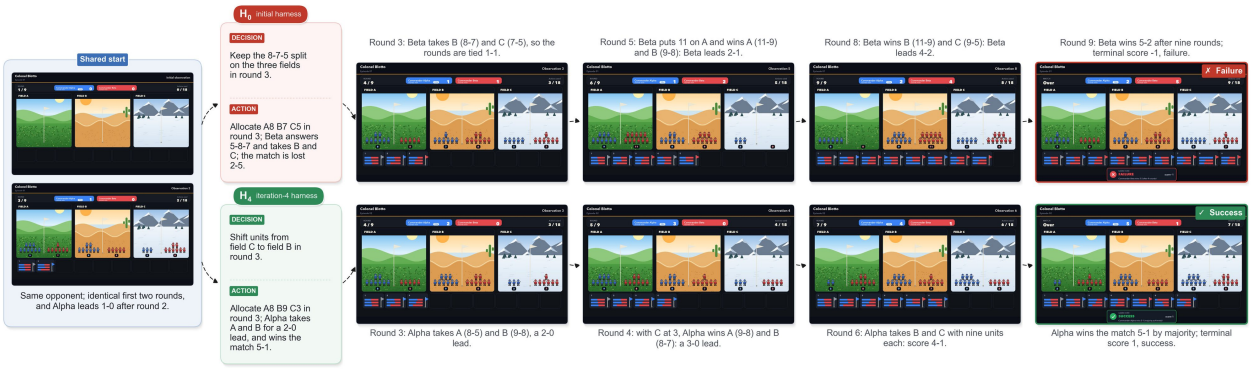

Figure 20 | **Colonel Blotto (train)**

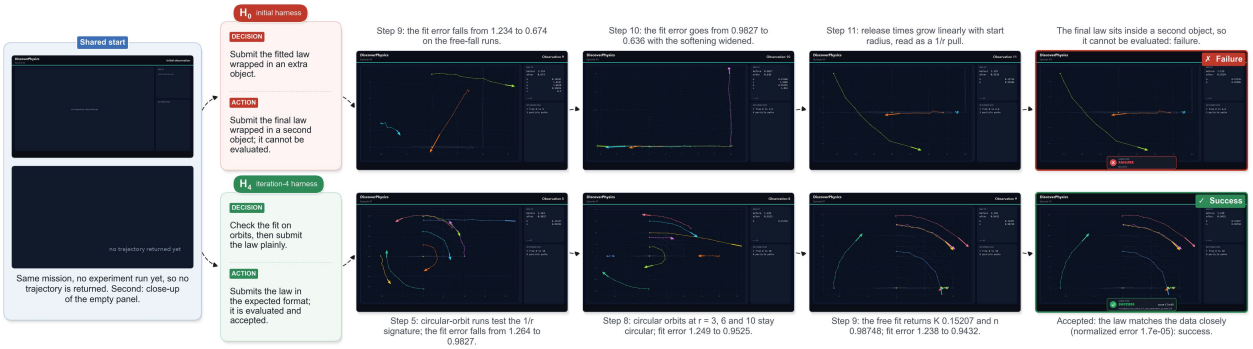

Figure 21 | **DiscoverPhysics (train)**

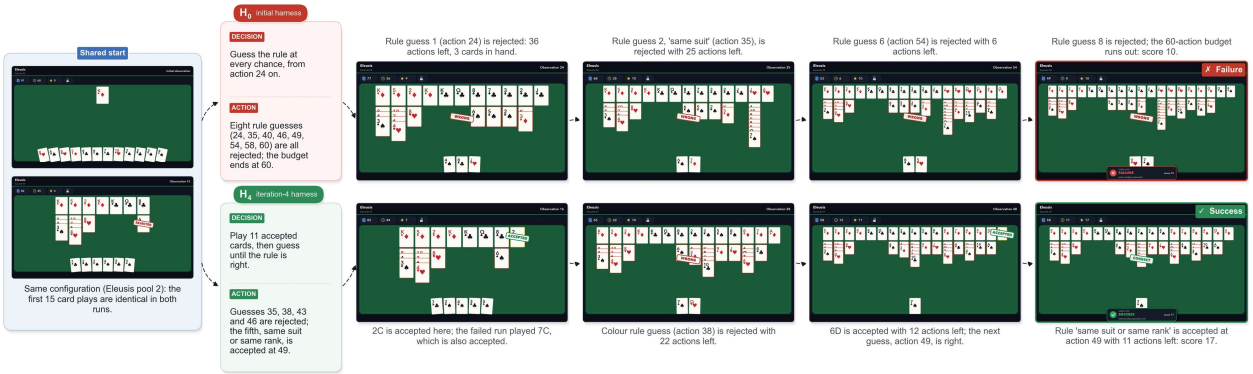

Figure 22 | **Eleusis (train)**

Figure 23 | **Euro NCAP AEB (train)**

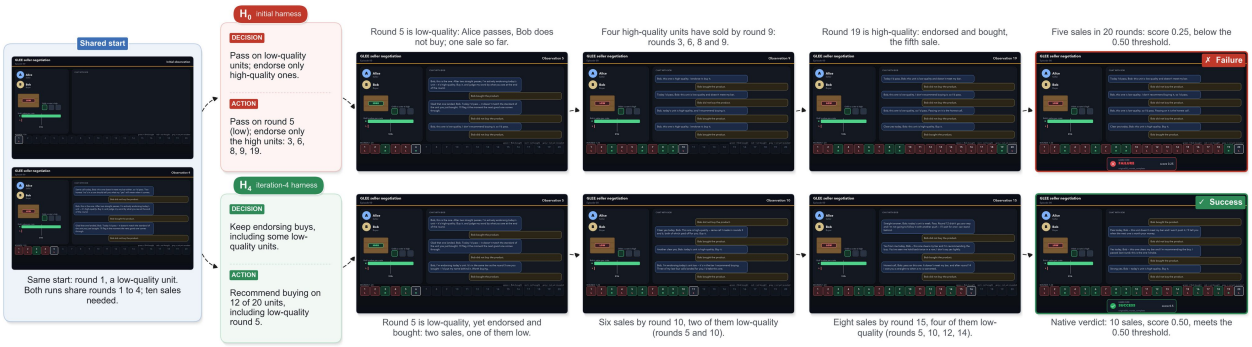

Figure 24 | GLEE Persuasion (train)

Figure 25 | Master Keying (train)

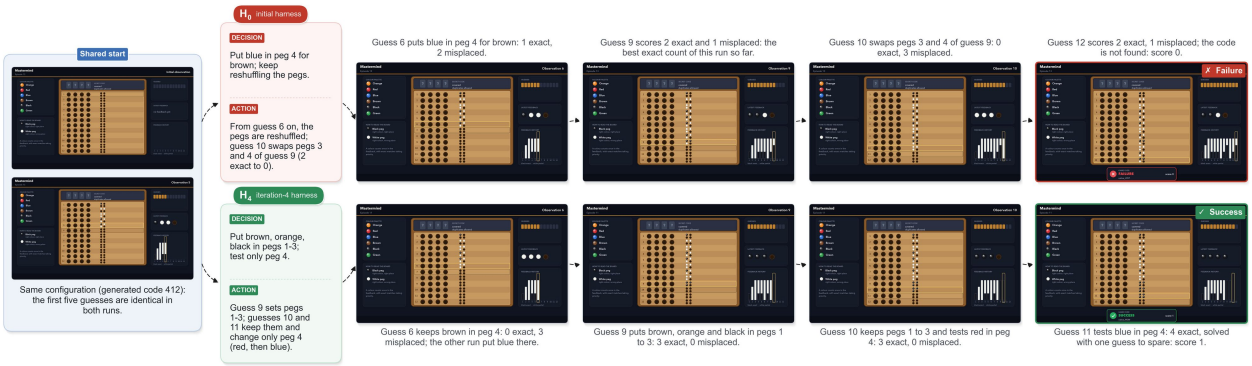

Figure 26 | Mastermind (train)

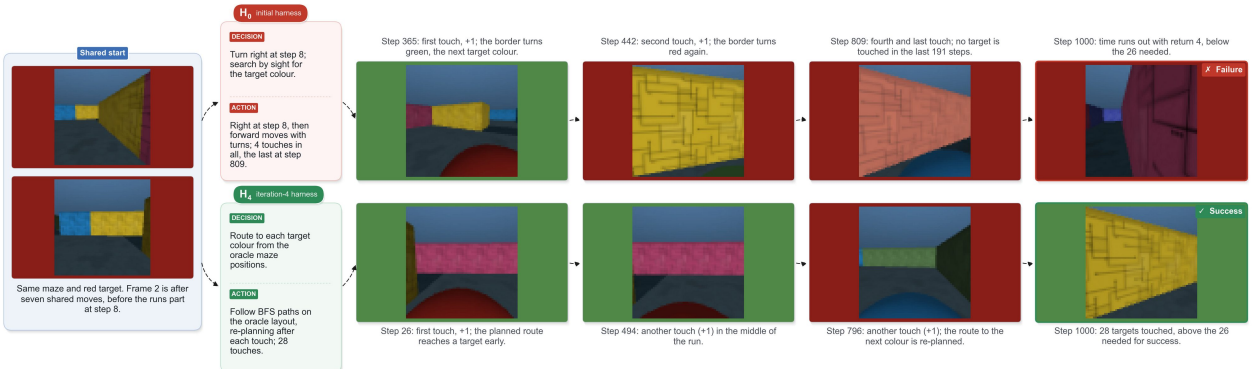

Figure 27 | Memory Maze (train)

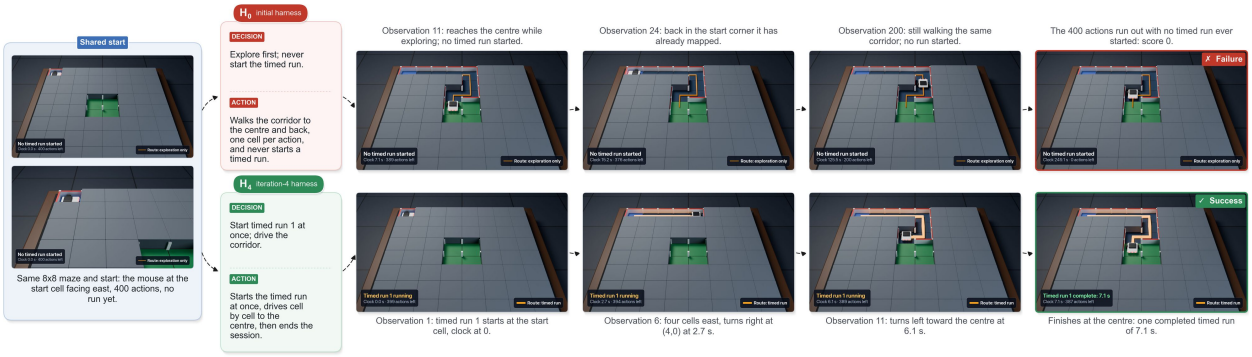

Figure 28 | Micromouse (train)

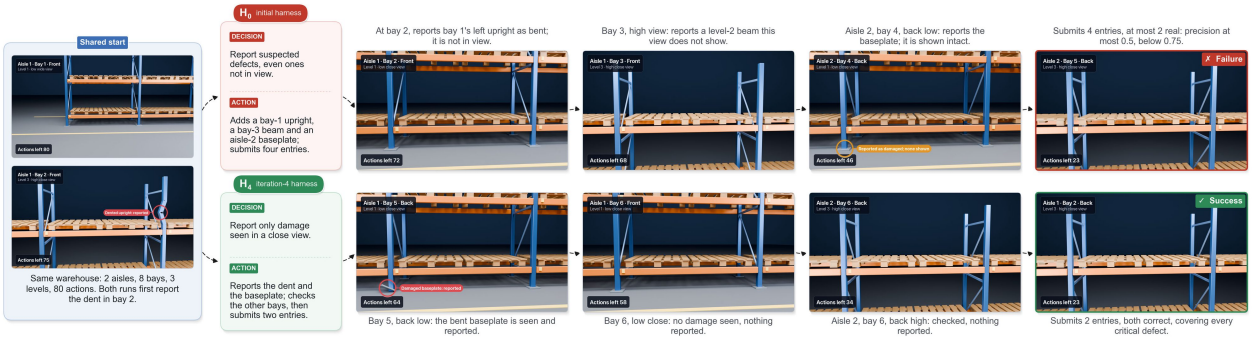

Figure 29 | Pallet Rack Inspection (train)

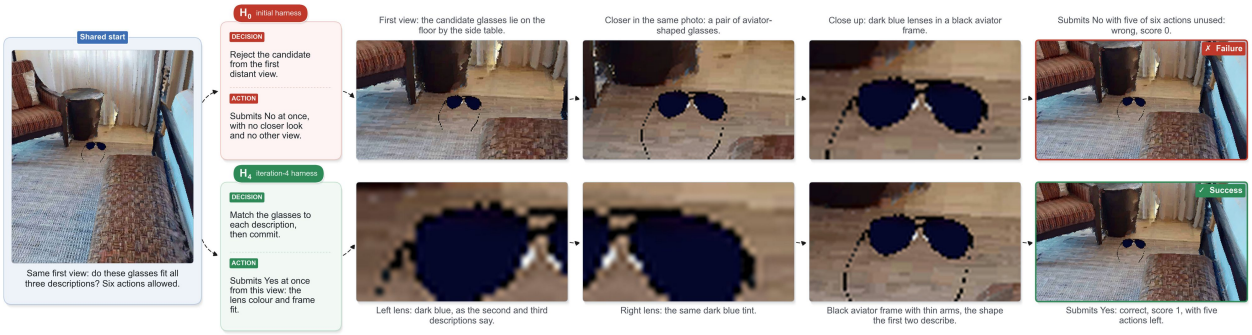

Figure 30 | PInVerify (train)

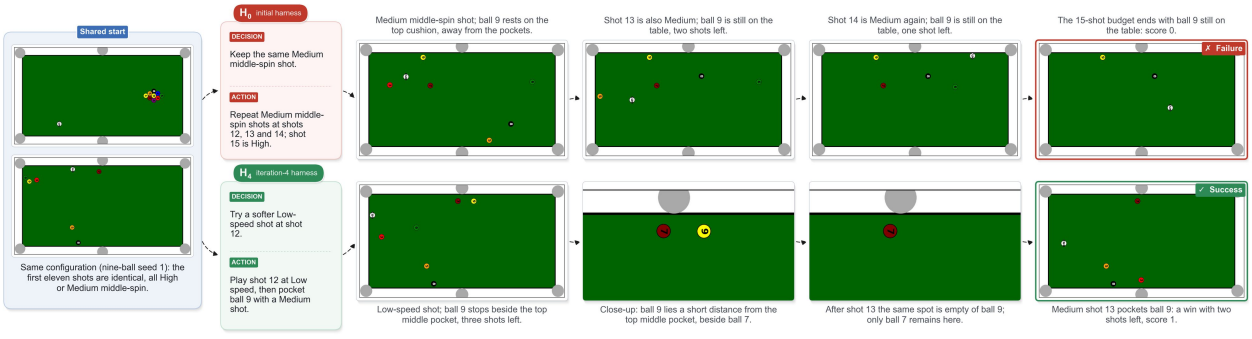

Figure 31 | Pooltool Billiards (train)

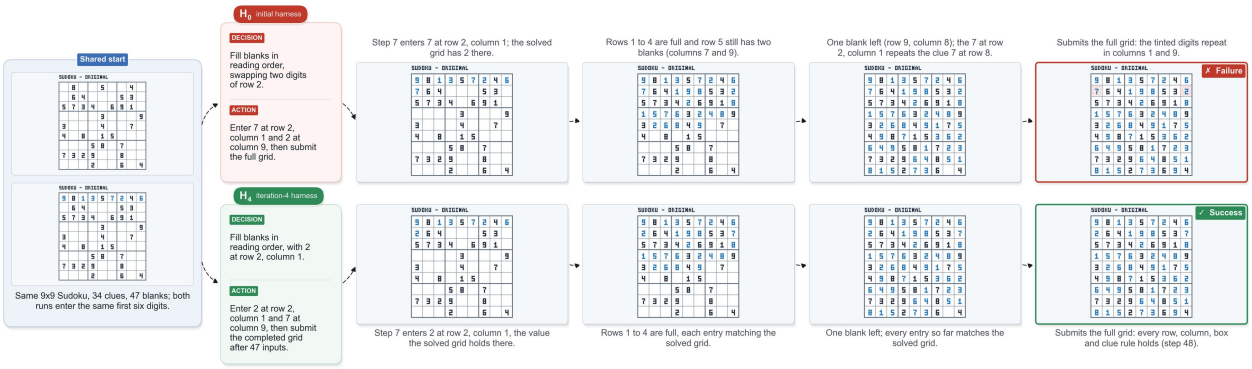

Figure 32 | **puzz.link / Penpa+ (train)**

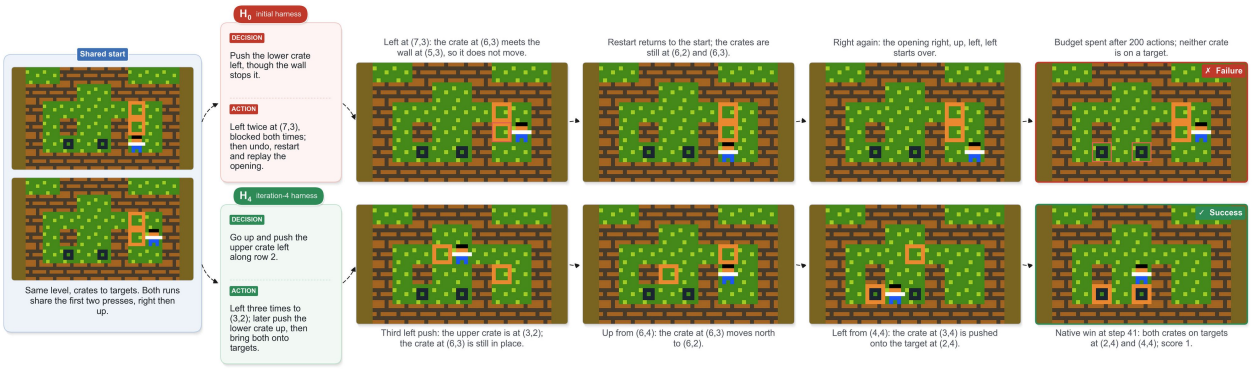

Figure 33 | **PuzzleScript (train)**

Figure 34 | **Saboteur (train)**

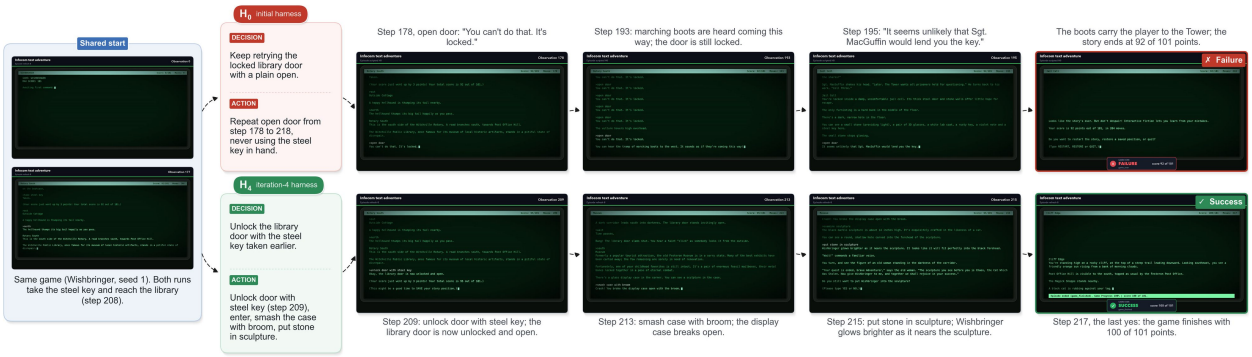

Figure 35 | **TextQuests (train)**

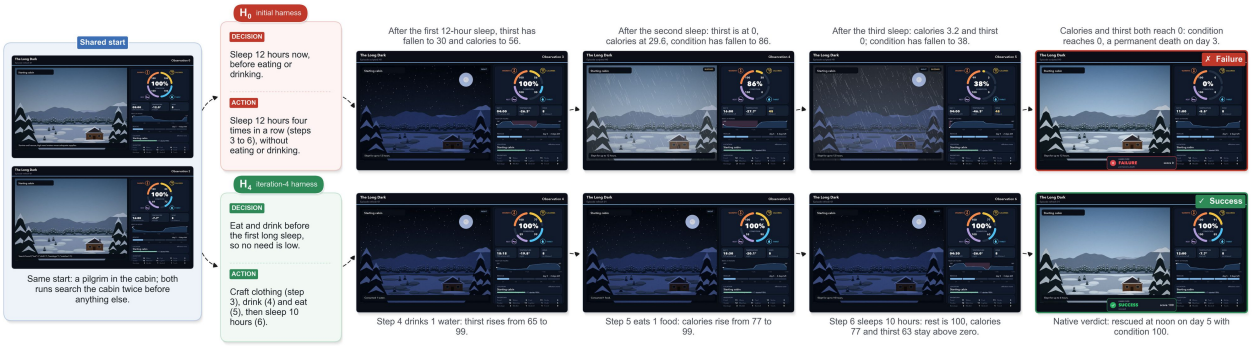

Figure 36 | The Long Dark (train)

Figure 37 | Video-Index (train)

Figure 38 | WCA Cube Solving (train)

Figure 39 | WILT (train)

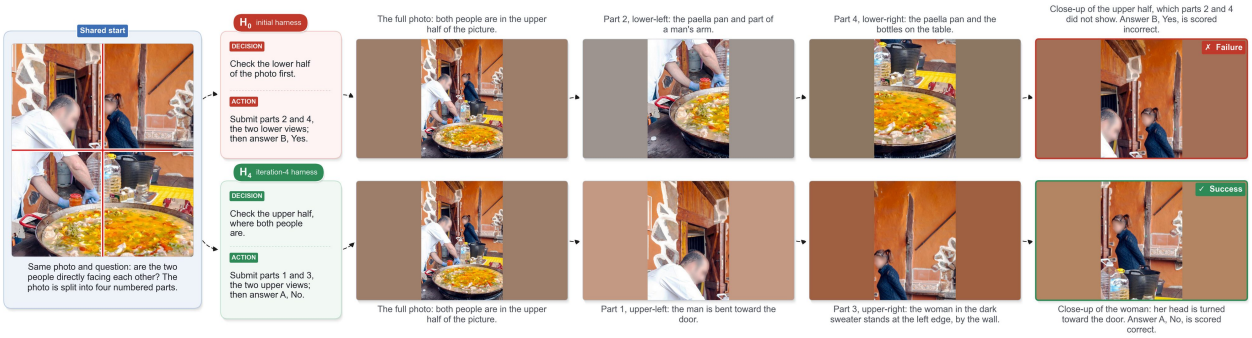

Figure 40 | **ActiView (unseen)**

Figure 41 | **ALE-Bench (unseen)**

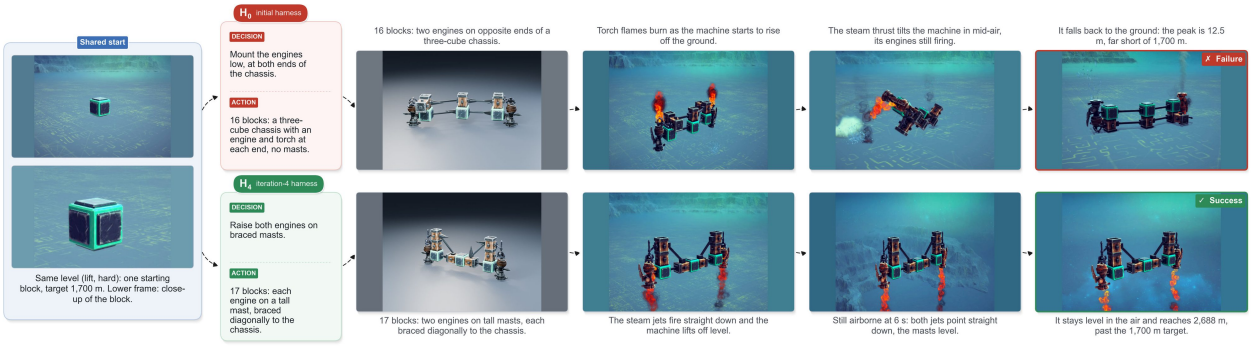

Figure 42 | **BuildArena (unseen)**

Figure 43 | **Coup (unseen)**

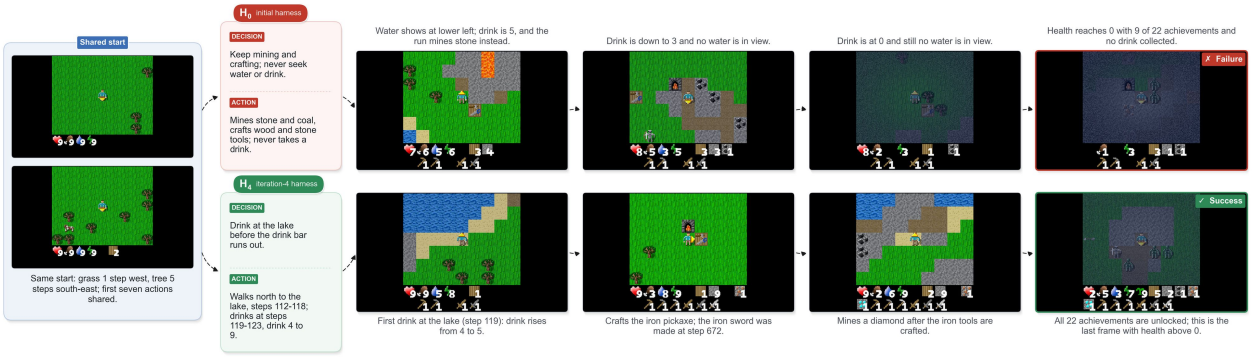

Figure 44 | Crafter (unseen)

Figure 45 | DiG-bench (unseen)

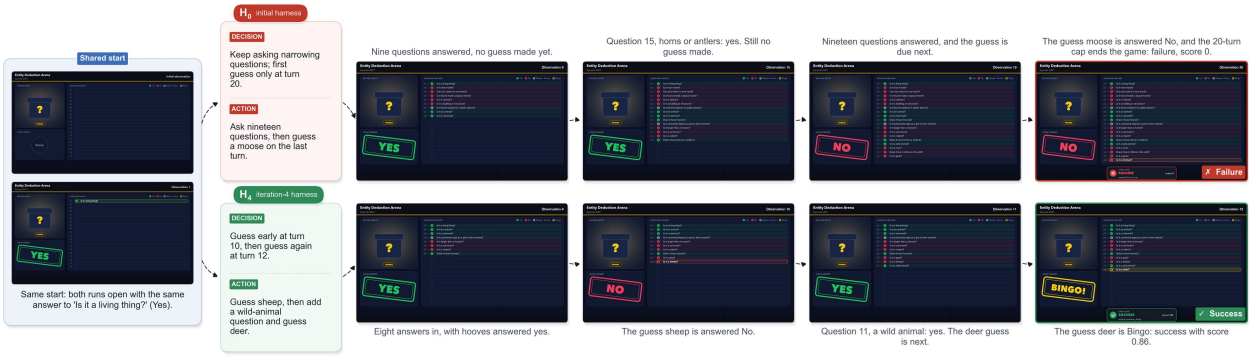

Figure 46 | Entity Deduction Arena (unseen)

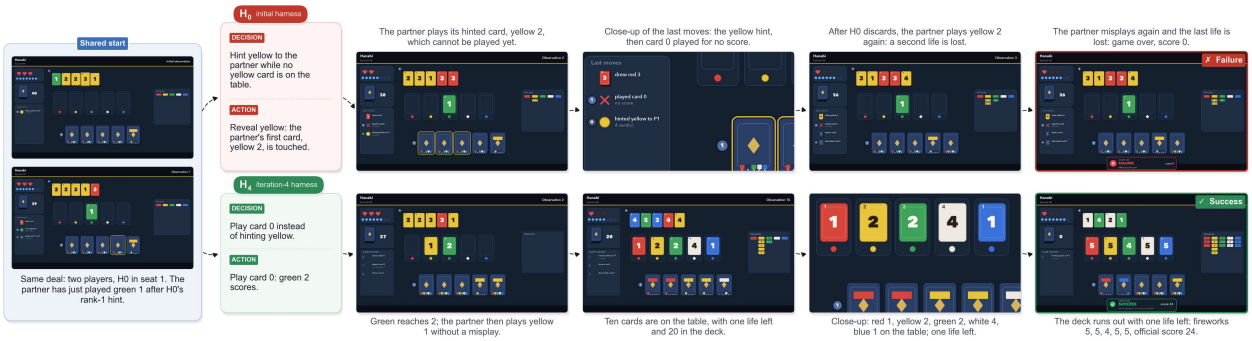

Figure 47 | Hanabi (unseen)

Figure 48 | Heads-Up NL Hold'em (unseen)

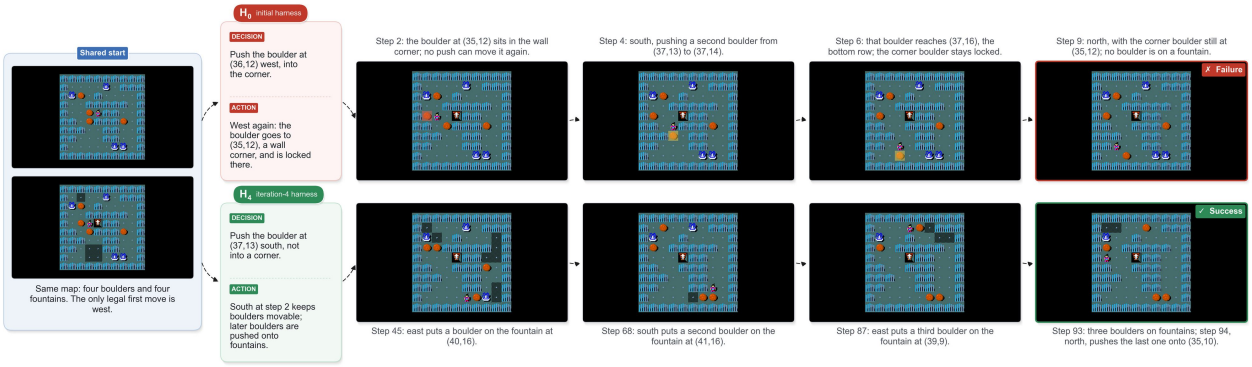

Figure 49 | MiniHack Boxoban (unseen)

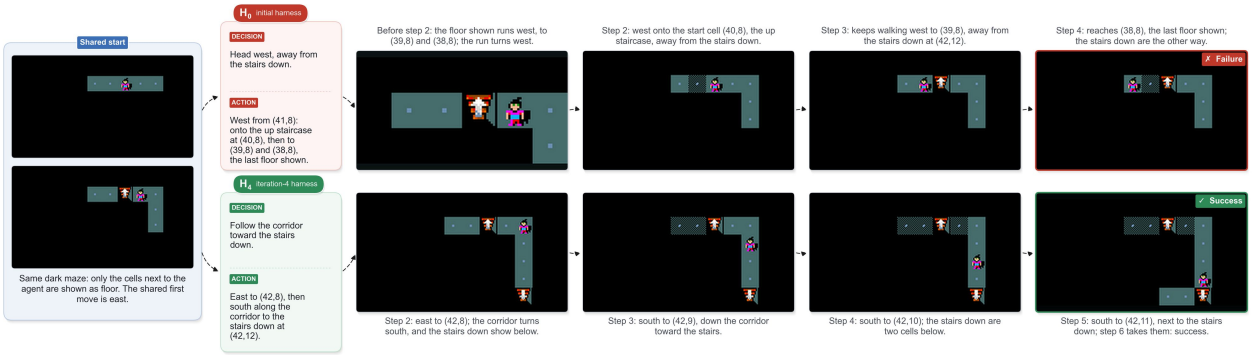

Figure 50 | MiniHack Maze / Corridor (unseen)

Figure 51 | NetHack (unseen)

Figure 52 | Pencil Puzzle Bench (unseen)

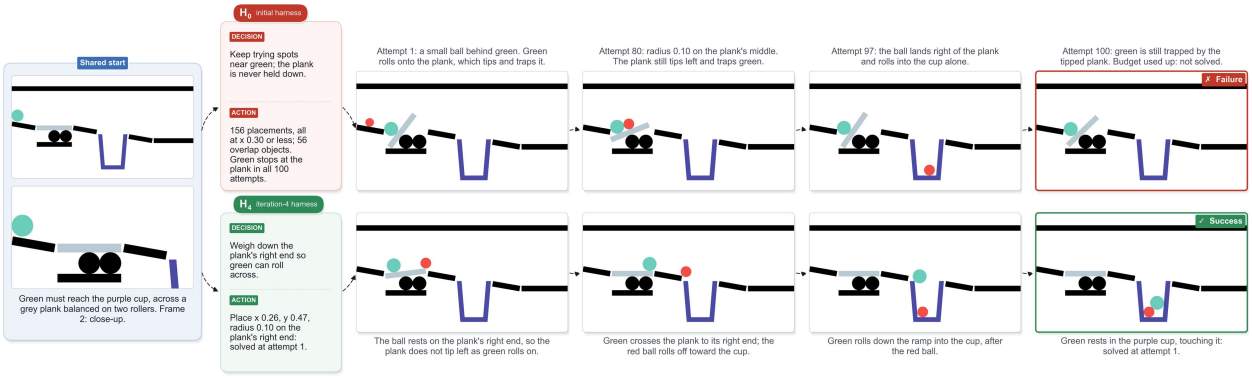

Figure 53 | PHYRE (unseen)

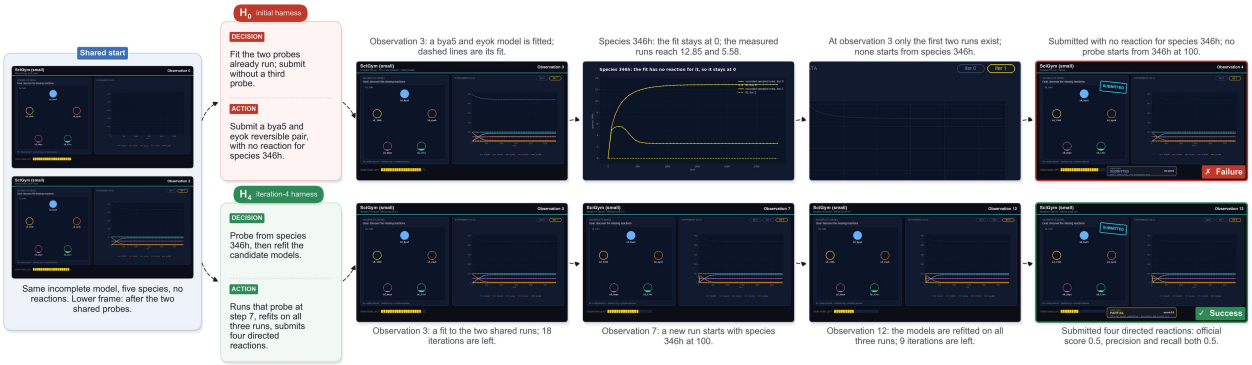

Figure 54 | SciGym (unseen)

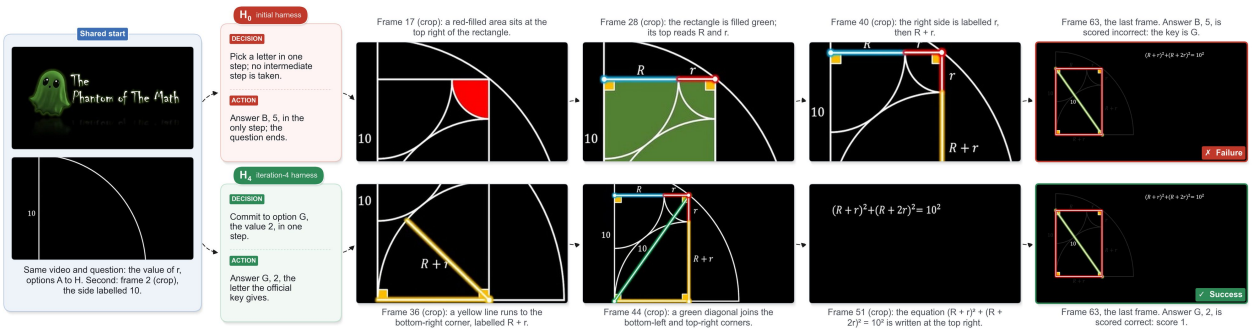

Figure 55 | Video-MME-v2 (unseen)

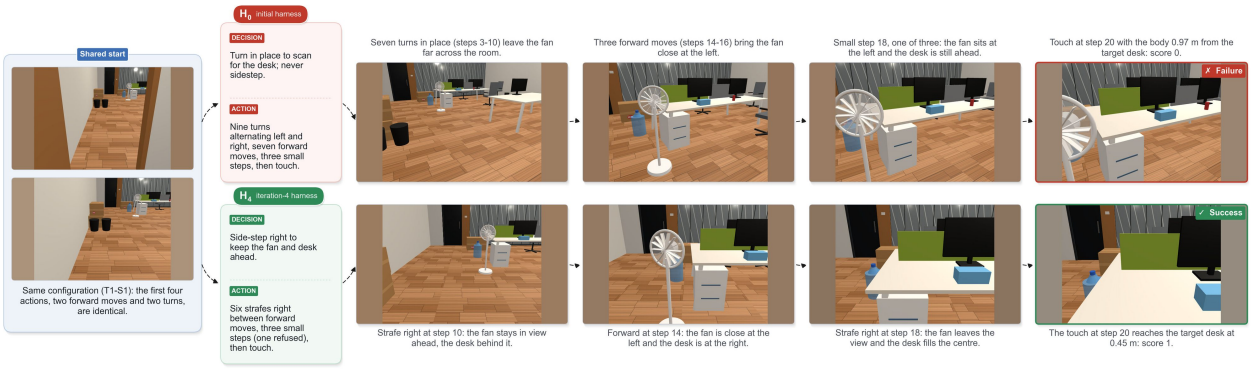

Figure 56 | SOON (embodied, simulation)

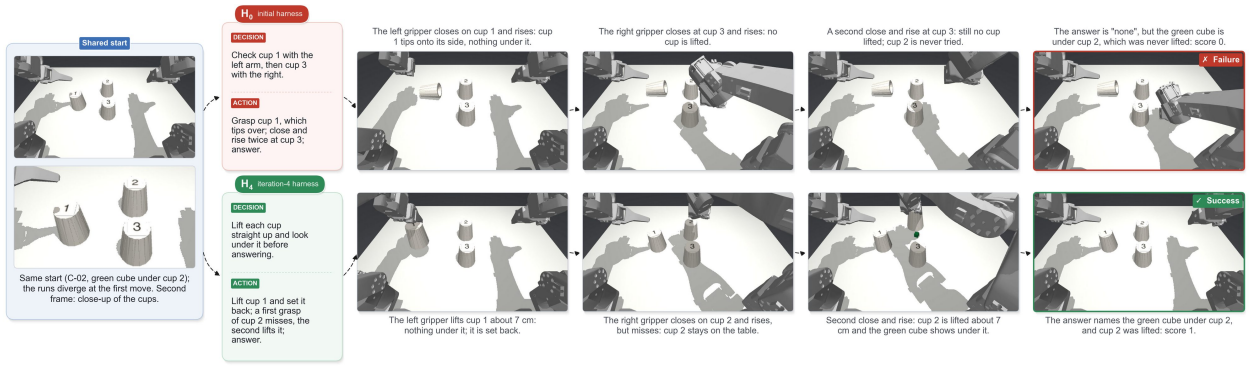

Figure 57 | CaP-X (embodied, simulation)

E.2. Node Training Levels

At every capability node, Evolver designs short training levels from the node's requirement, and Creator implements and revises them under the Creator Harness of Appendix C.2; each level is a simulation world with a 3D scene, public rules, a fixed action budget, and a scorer, and one level per node and iteration gives 44 levels. Because e and e→f form a separate path for video understanding, Video-Index, the only training benchmark with video input, feeds both nodes. Figures 58–68 show how each node evolves: on the left are the training benchmarks that form the node's input, and on the right its four levels are played in turn, each tracing how the failure on that level is distilled into a new or revised module and how the next run with that module completes the level. The levels of a node keep one task family and raise the difficulty from iteration to iteration. At c, two robots deliver crates through gates that open only while the partner stands on a linked plate (M07, M08). At c→d, a factory must meet orders due on fixed shifts under staged shortages (M12, M13, M14). At c→d→a, ring-marked cells must satisfy relations set in stages (M15, M16). At c→a, similar jars must be collected in a revealed order (M10, M09, M11). At d, a claim may be submitted only once an object's hidden faces have been seen (M17, M18). At d→a, stamps and a parcel must be brought back while gates close on the return (M19, M20, M21). At d→b, a hidden pattern with repeated symbols must be identified by probes (M22, M23). At a, structural members are inspected for defects beside look-alikes (M01, M02, M03). At b, tokens are allocated over three fields against a rotating formation (M06, M04, M05). At e, the segments holding a parcel's hand-offs must be bought within a budget (M25, M24, M26). At e→f, the parcel must be followed across hand-offs to its endpoint (M28, M27).

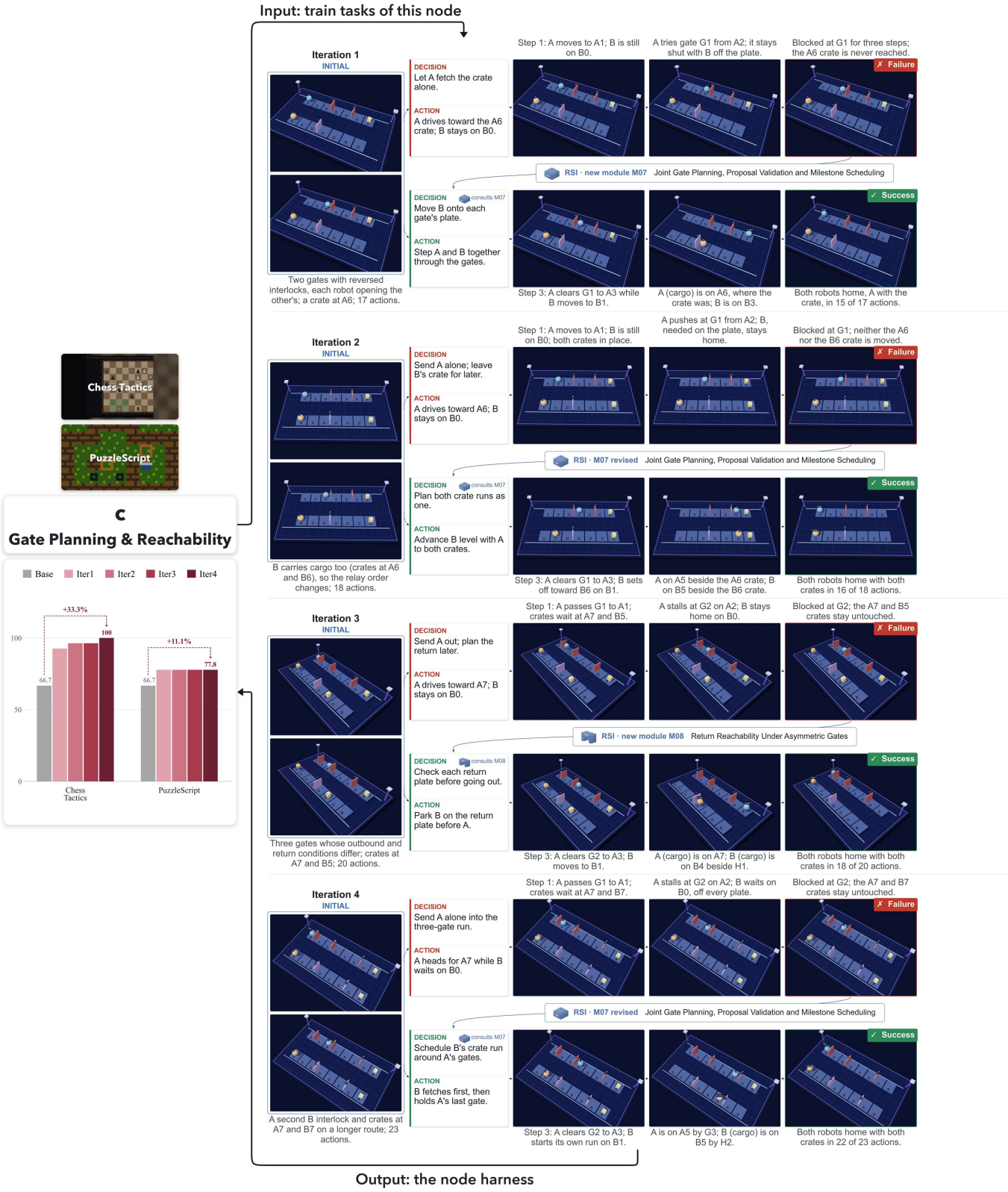

Figure 58 | c: Gate Planning & Reachability

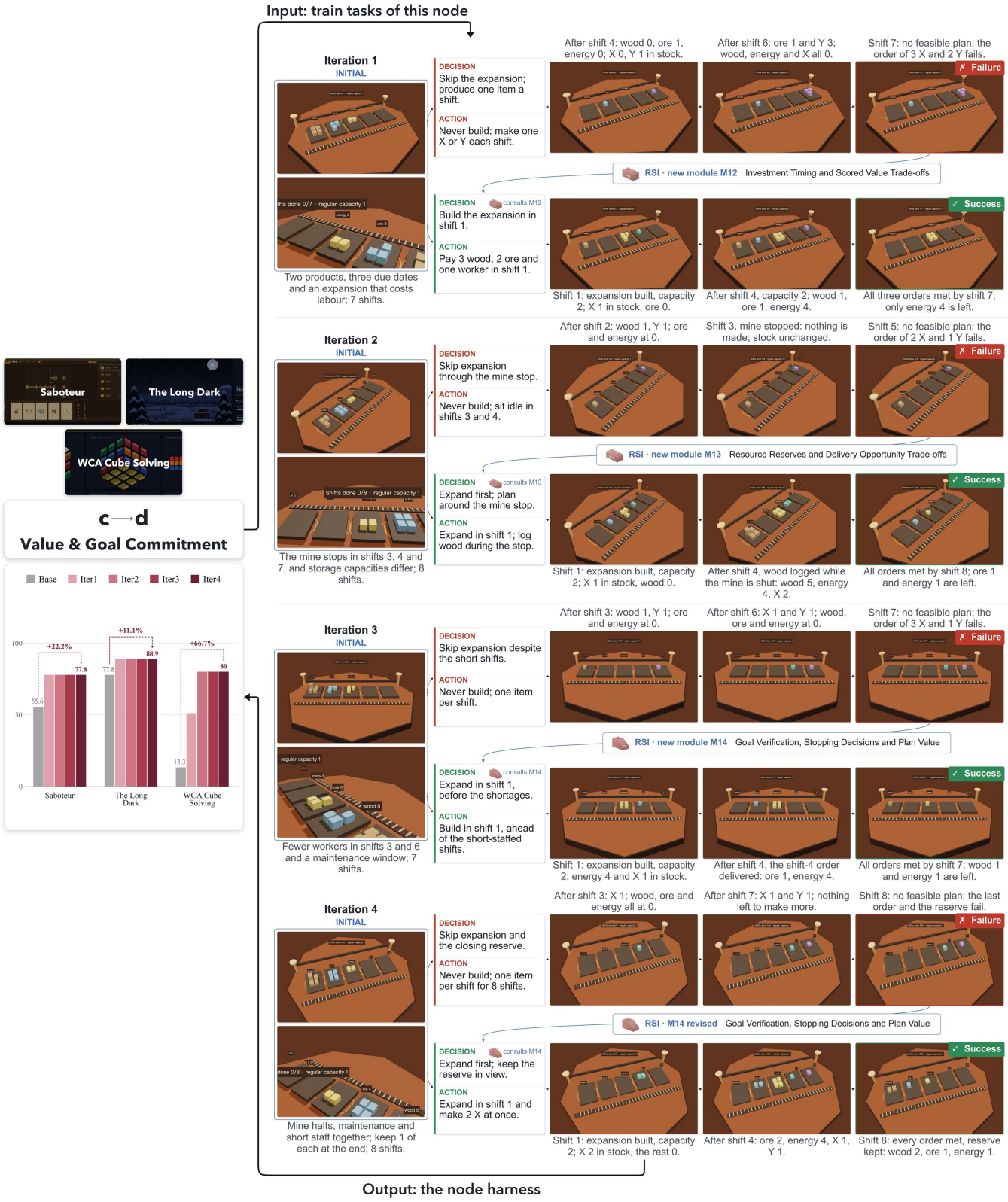

Figure 59 | c→d: Value & Goal Commitment

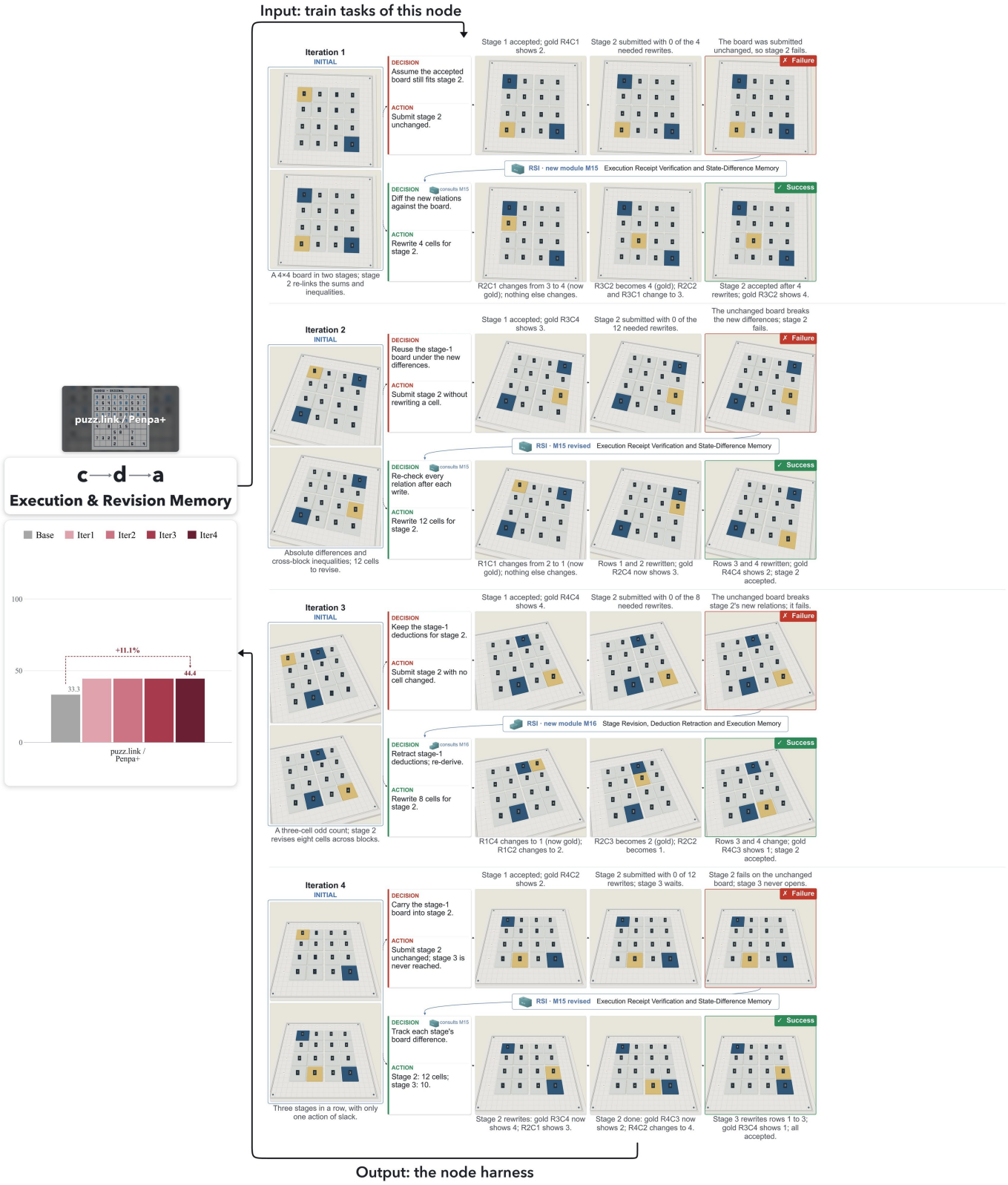

Figure 60 | c→d→a: Execution & Revision Memory

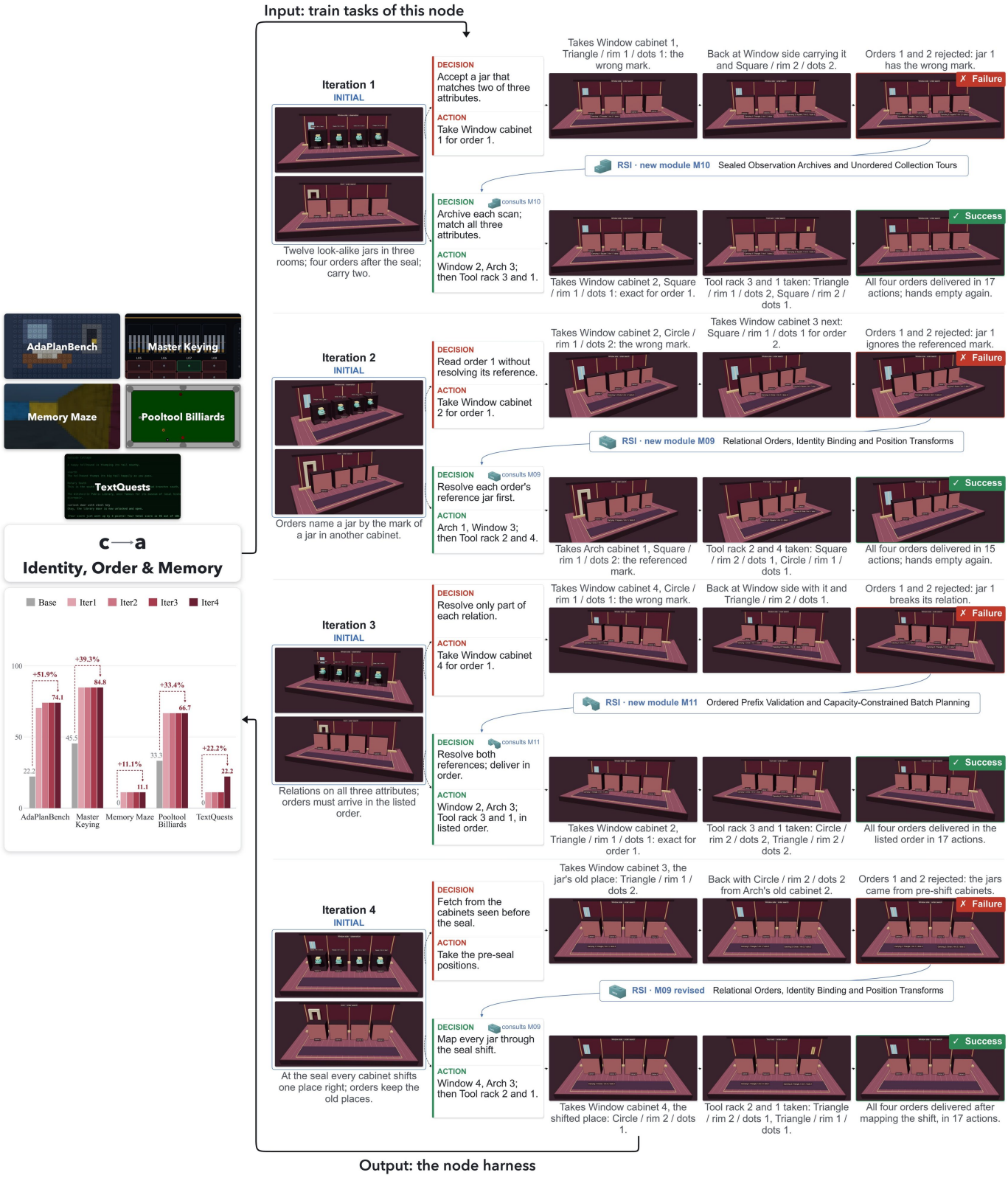

Figure 61 | c→a: Identity, Order & Memory

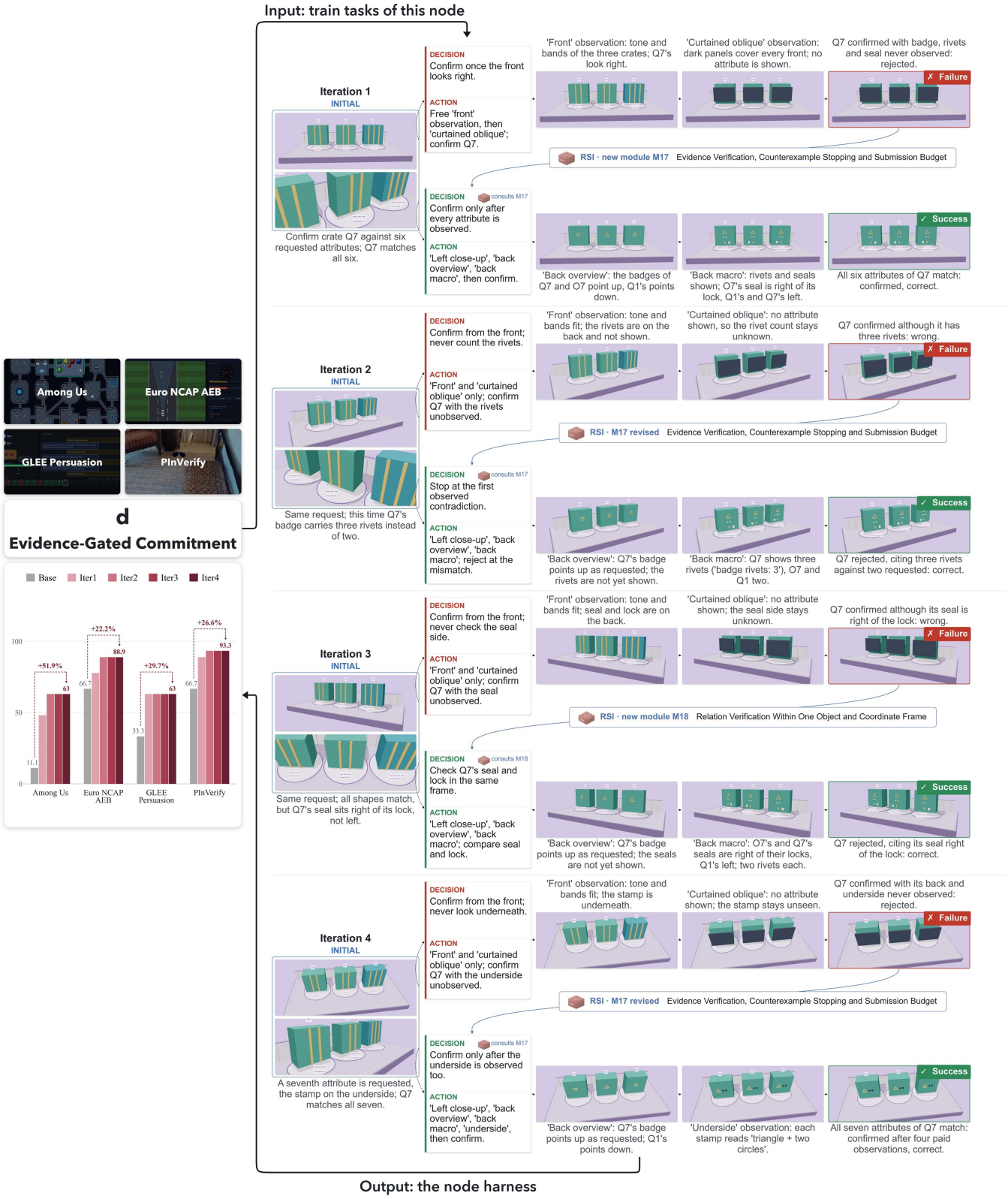

Figure 62 | d: Evidence-Gated Commitment

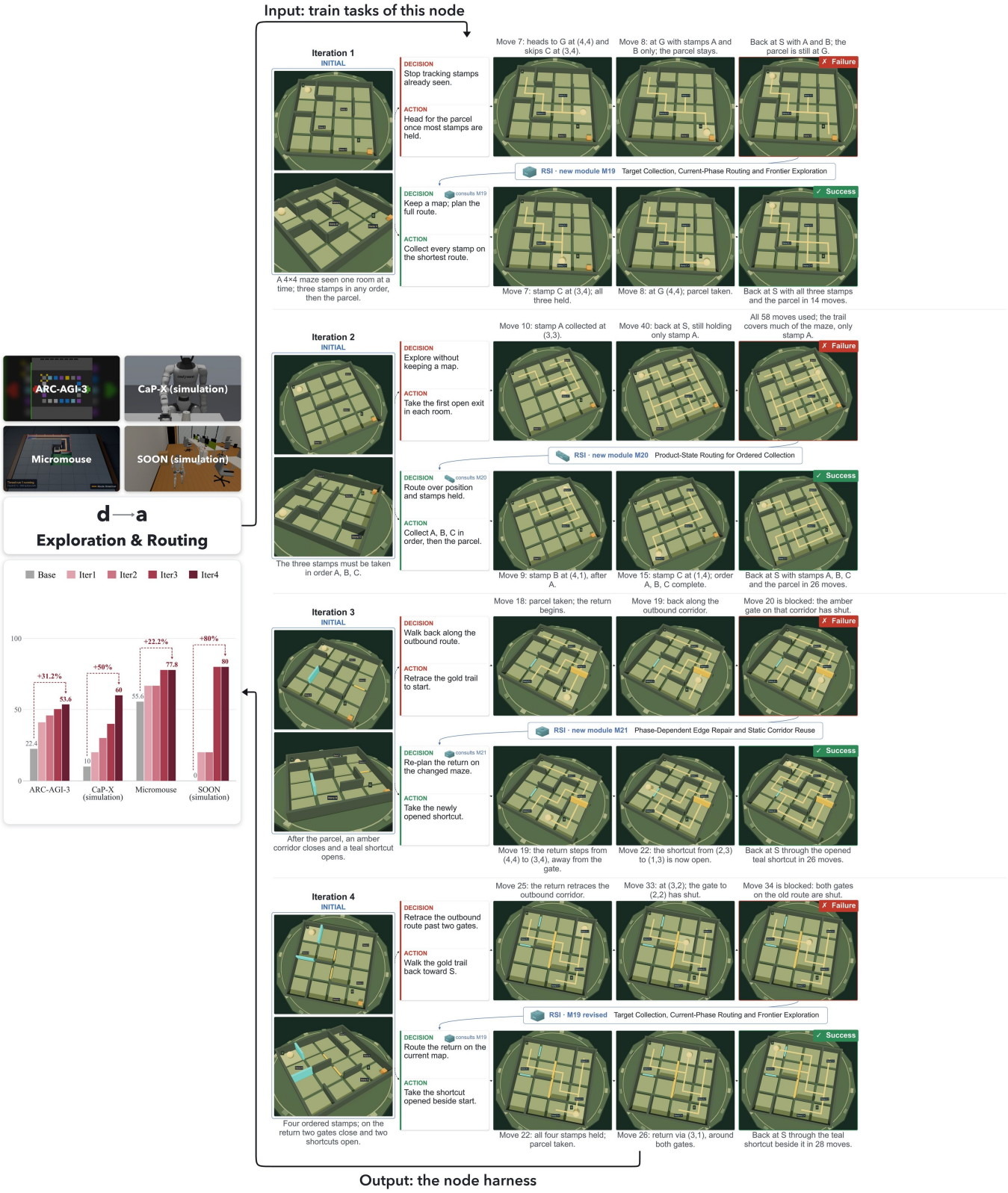

Figure 63 | d→a: Exploration & Routing

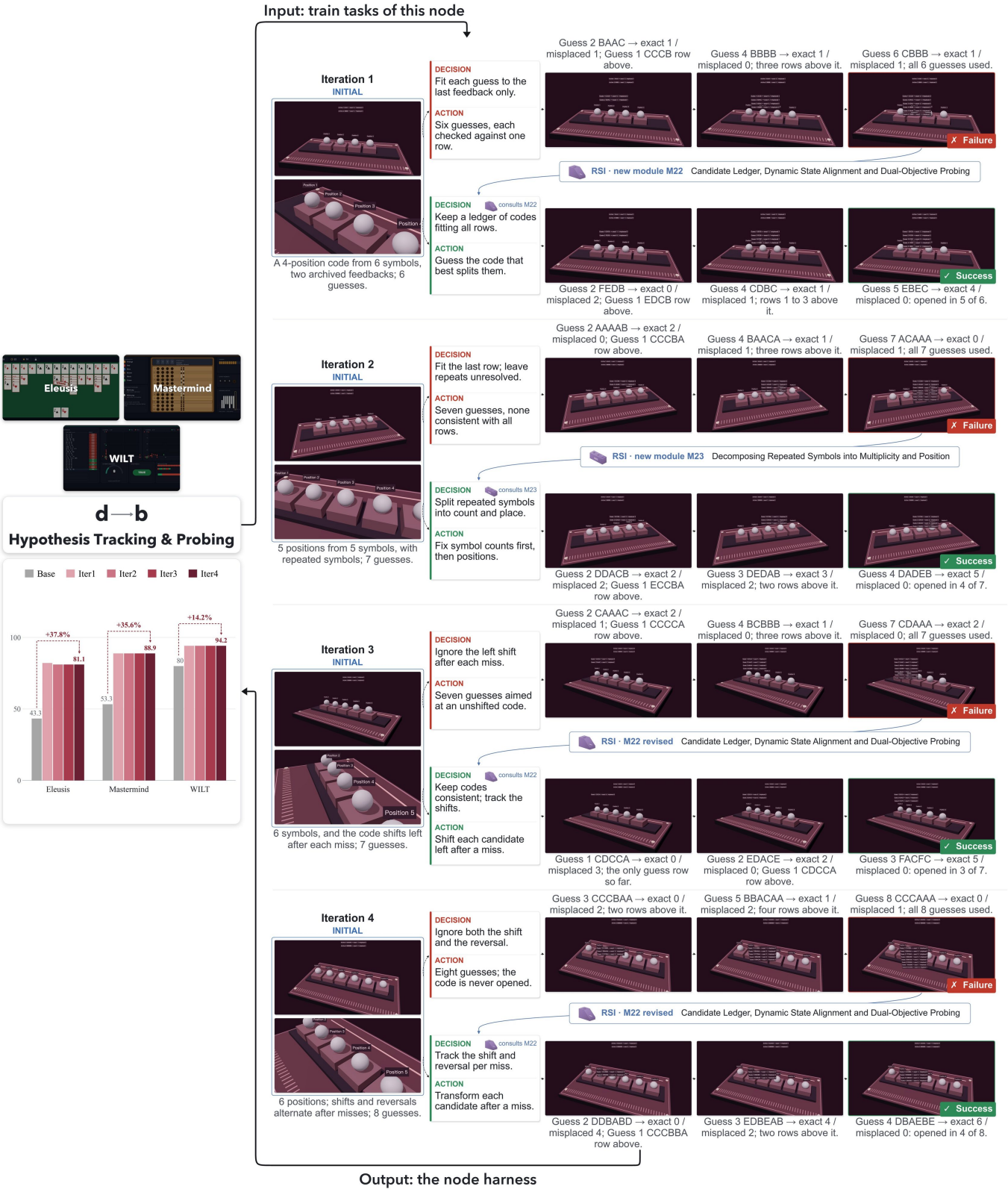

Figure 64 | d→b: Hypothesis Tracking & Probing

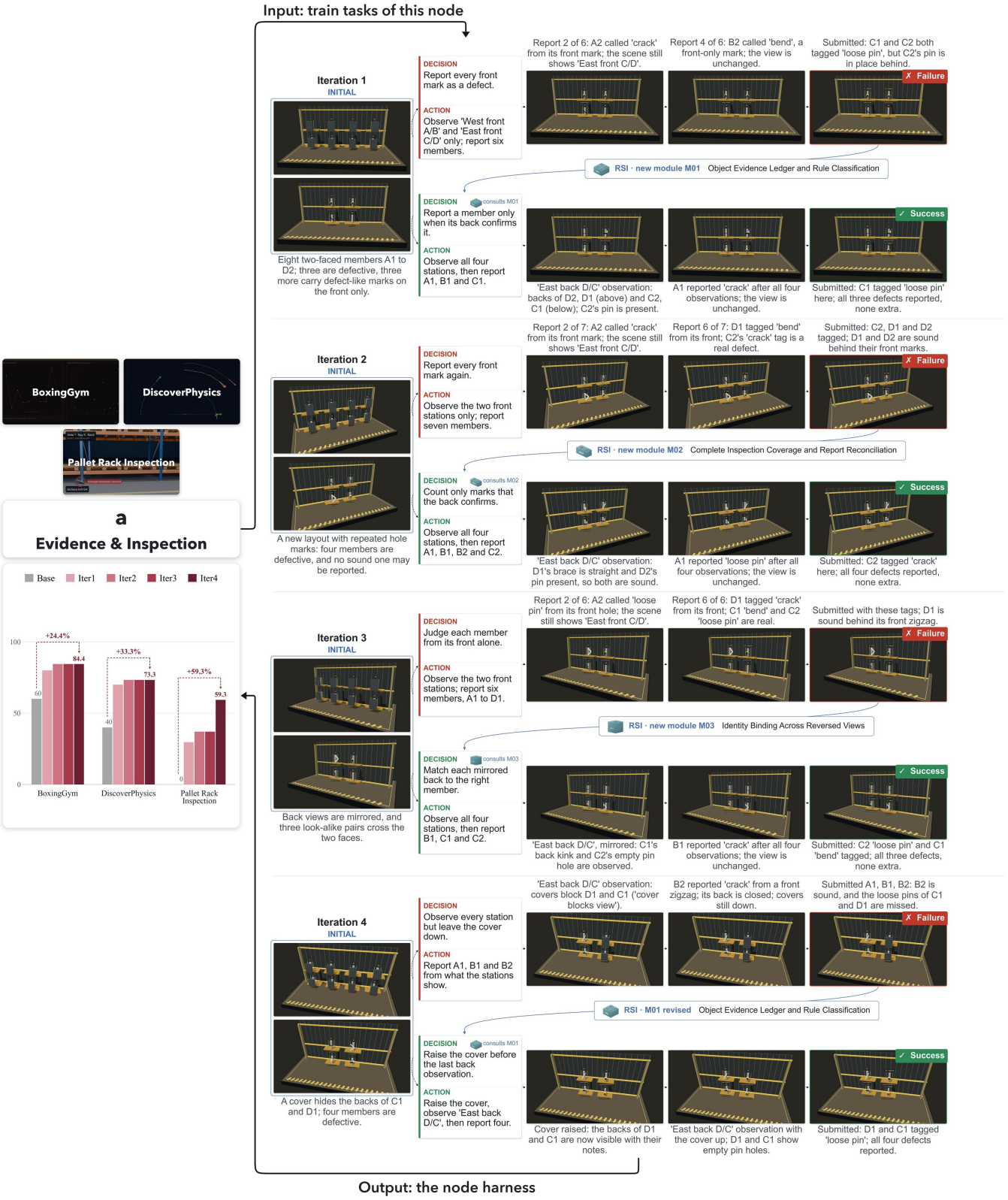

Figure 65 | a: Evidence & Inspection

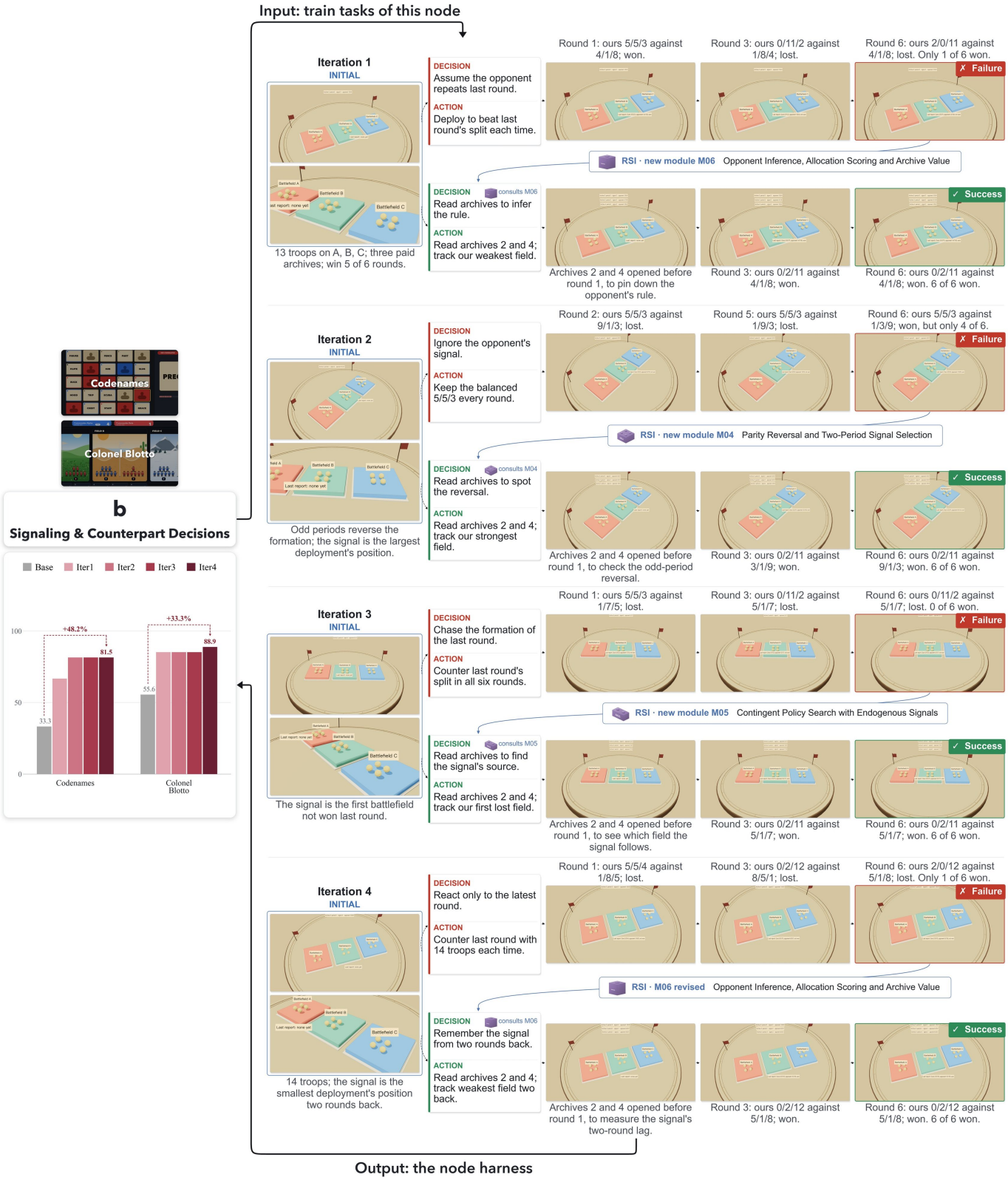

Figure 66 | b: Signaling & Counterpart Decisions

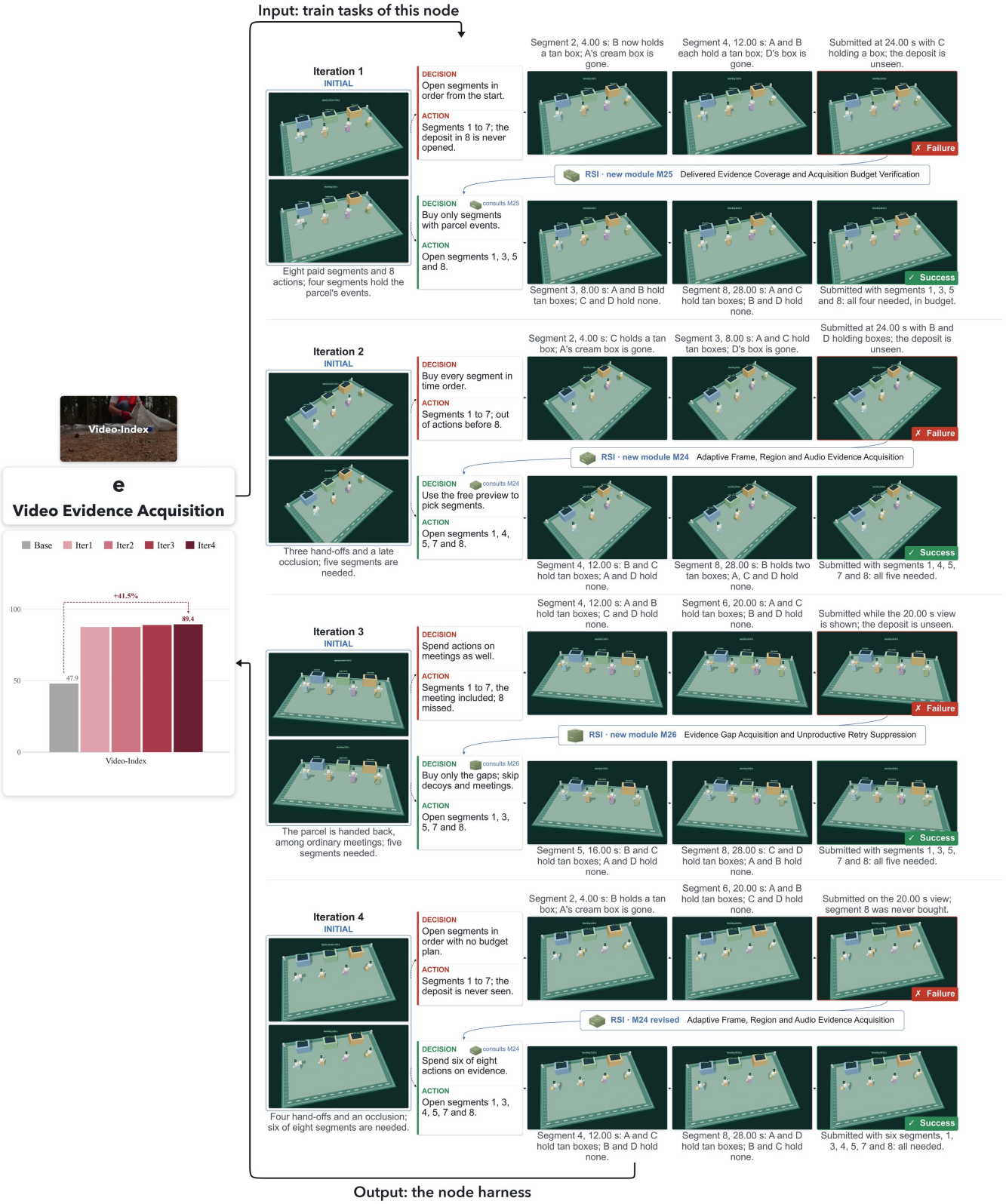

Figure 67 | e: Video Evidence Acquisition

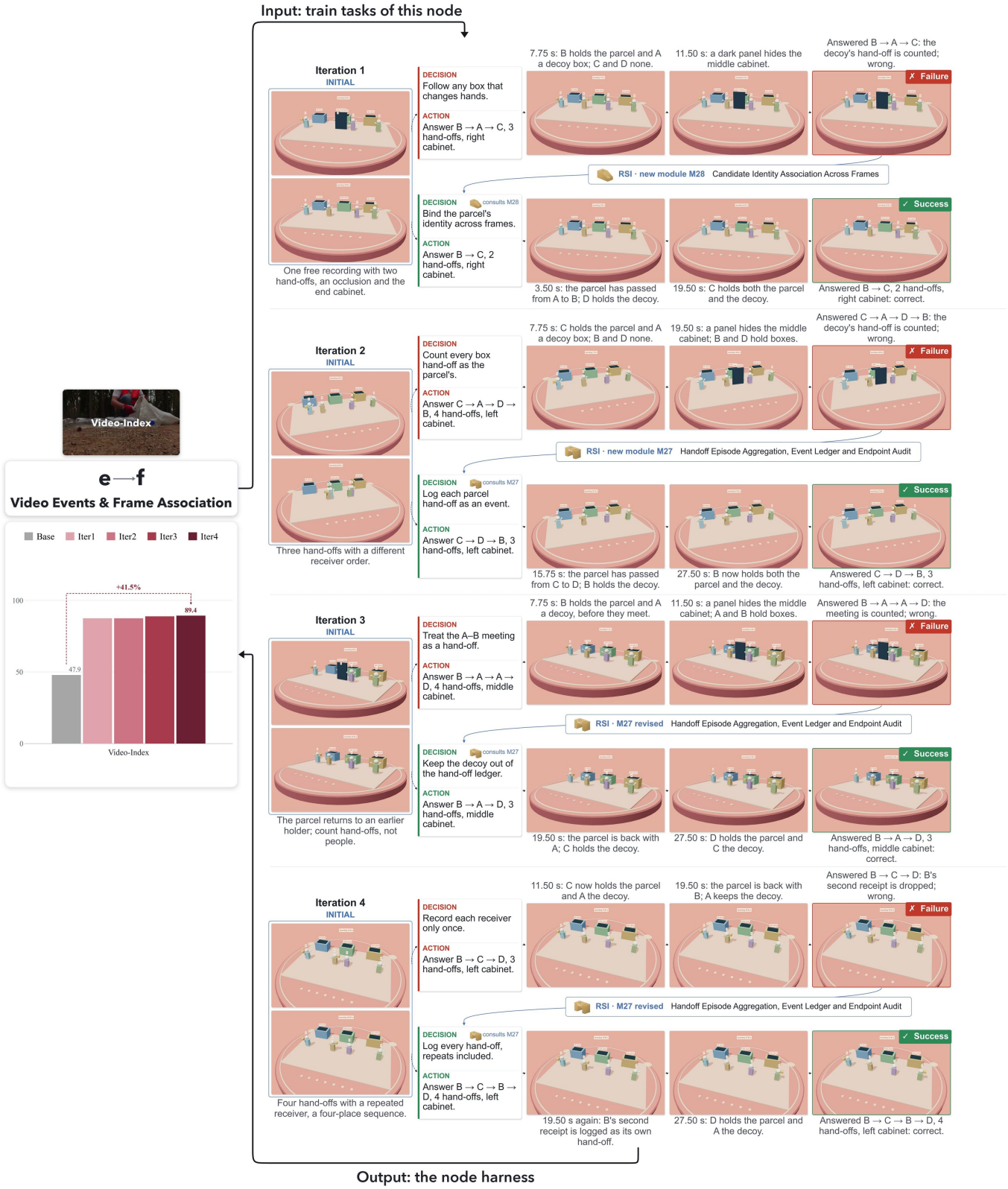

Figure 68 | e→f: Video Events & Frame Association

E.3. Embodied Benchmarks

The simulation of each embodied benchmark runs in an environment built from its real scene, which closely reproduces the layout and furnishings of the site, with a model of the same robot as the real runs: a Unitree G1 for SOON and an AlohaMini2 for CaP-X. For SOON we rebuilt the office from its floor plan and from photographs and video of the site: the side room where the robot starts, the doorway into the office, the rows of desks with their chairs and cabinets, and the white pedestal fan beside the target desk. The office is a 3D scene in which the walls and furniture are collision obstacles: each action moves the robot by the nominal distance or angle of the real robot's motion profile, a move that hits an obstacle counts as a collision and ends the episode as a failure, and after every action the agent receives the head-camera image rendered from the robot's new pose with the height, downward pitch, and field of view of the real camera. On the real robot a person judges every episode on site, and an episode in which the robot collides with an obstacle is a failure. The G1 starts in the side room, walks through the doorway into the office, and must stop at the desk next to the white fan and touch it within 30 movement actions. For CaP-X we built the table scene in the MuJoCo physics simulator [125] with the official AlohaMini2 model, and with objects of the same size and colour as the real ones: three paper cups, 9 cm tall, 7 cm across the mouth, and 5 cm across the closed bottom, stand upside down on a white table, and in most layouts a 2 cm cube, red, green, or blue, lies under one of them. The AlohaMini2 stands at the near edge of the table with its two six-joint arms; its mobile base and lift stay still. After every action the agent sees the head camera, the wrist cameras on both grippers, and a fixed side camera; it moves a gripper in small steps while the host solves the arm's inverse kinematics, and within 30 actions it must report whether a cube is present and, if it is, its colour and the cup it is under. Figure 69 compares the real and simulated scenes directly.

Figure 69 | **Embodied environments.** Real scenes appear above their simulated counterparts, with SOON on the left and CaP-X on the right.

Figures 70–73 compare H_0 and $H^{(4)}$ under the same task settings. In SOON, the robot must judge the distance to the target desk from head-camera images alone. H_0 looks around and reaches out as

soon as the desk looks close, stopping about 0.5 m short in simulation and out of reach on the real robot. $H^{(4)}$ turns towards the desk right after the doorway and advances in small steps until the desk is directly ahead, so that in both settings the hand ends within the range judged as a successful touch. In CaP-X, the cube is hidden under upside-down cups and can only be found by uncovering them. H_0 grasps clumsily, knocks cup 1 over, tries cup 3, and then answers that there is no cube without ever examining cup 2. $H^{(4)}$ checks the cups one by one and answers only after seeing the cube: in simulation it lifts cup 1, finds it empty, retries a missed grasp on cup 2, and lifts it to reveal the green cube; on the real robot it pushes the cups aside until the cube lies uncovered. In both benchmarks, $H^{(4)}$ seeks direct evidence before committing and thus completes the task.

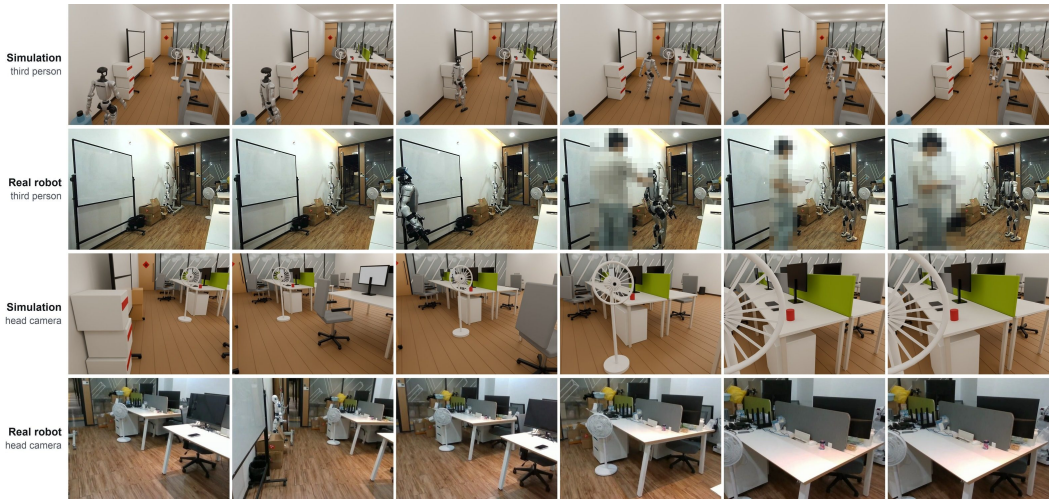

Figure 70 | SOON with the Base Harness. Rows show simulation and real-robot views from the fixed office camera, followed by the head camera. The robot walks from the side room along the aisle, looks around, and turns towards the desk beside the fan, but reaches out before the desk is within reach in both settings.

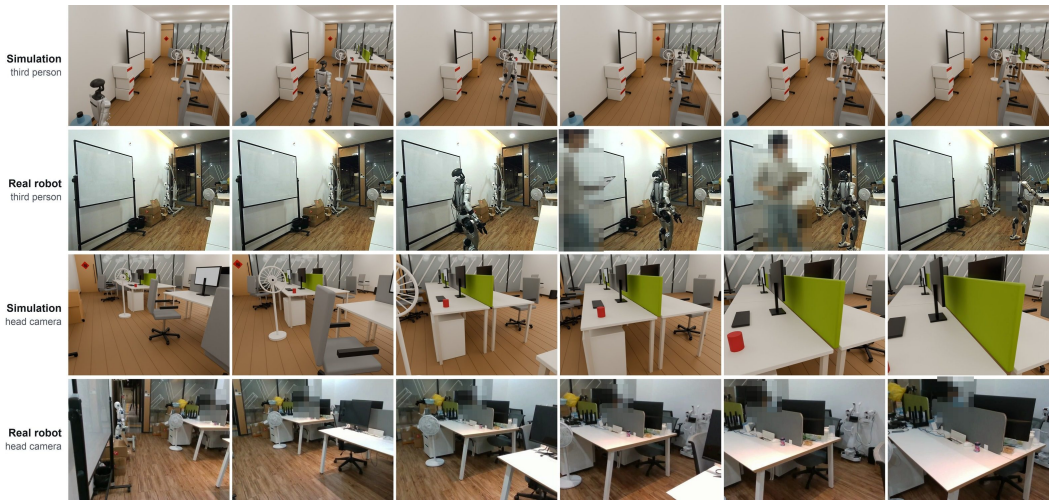

Figure 71 | SOON with the evolved Harness. Rows follow Figure 70. In simulation the robot turns right past the doorway, walks diagonally towards the desk beside the fan, and slows to small steps before touching it. The real robot also approaches in small steps and completes the task.

Figure 72 | CaP-X with the Base Harness. Rows show head, side, and wrist cameras in simulation, followed by the real robot. In both settings the robot manipulates other cups without inspecting the target cup and incorrectly reports that no cube is present.

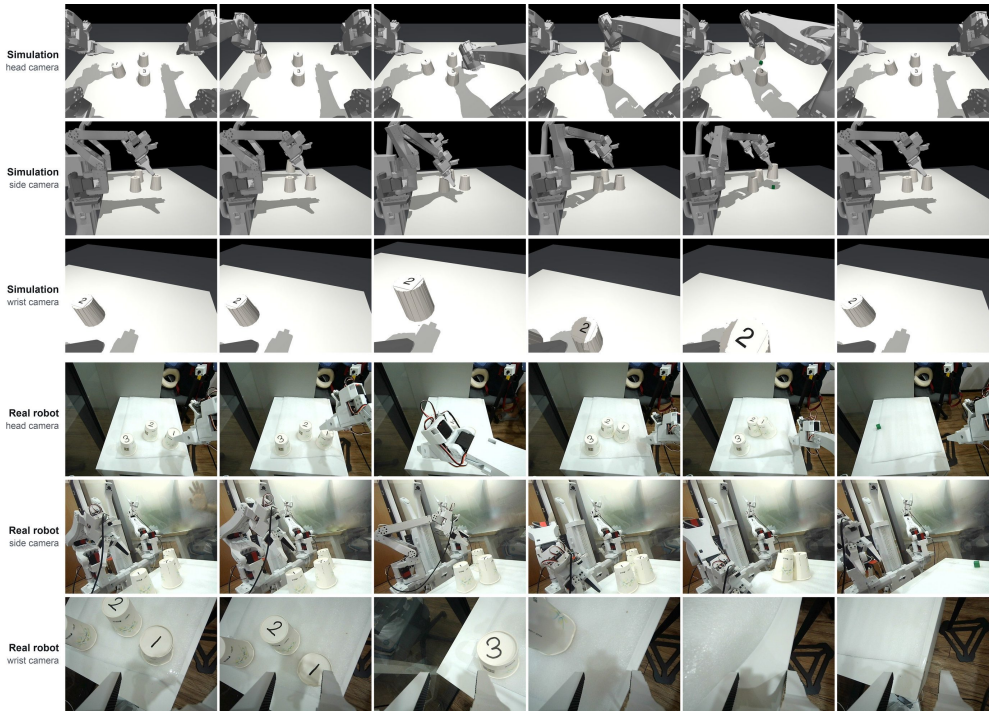

Figure 73 | CaP-X with the evolved Harness. Rows follow Figure 72. In simulation the robot checks the cups and retries a missed grasp until the cube is visible. On the real robot it sweeps the cups aside to uncover the cube; both runs then report its colour and location.